Untitled

September 20, 2025

1 ANN

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("Churn_Modelling.csv")
X=dataset.iloc[:,3:13]
y=dataset.iloc[:,13]

#create dummy variables
geography=pd.get_dummies(X["Geography"],drop_first=True)
gender=pd.get_dummies(X["Gender"],drop_first=True)

#concat dataframes
X=pd.concat([X,geography,gender],axis=1)
#drop unecessary cols
X=X.drop(['Geography','Gender'],axis=1)

#split train test
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)

#feature scaling

from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)

##ANN

import keras
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LeakyReLU,PReLU,ELU
from keras.layers import Dropout
```

```
#initialise ANN
classifier=Sequential()

#add input and 1st hidden layer
#classifier.
 ↪add(Dense(output_dim=6,init="he_uniform",activation="relu",input_dim=11))
classifier.
 ↪add(Dense(units=6,kernel_initializer="he_uniform",activation="relu",input_dim=11))

#add 2nd hidden layer
classifier.add(Dense(units=6,kernel_initializer="he_uniform",activation="relu"))

#add output layer
classifier.
 ↪add(Dense(units=1,kernel_initializer="glorot_uniform",activation="sigmoid"))

#compile ANN
classifier.
 ↪compile(optimizer="Adamax",loss="binary_crossentropy",metrics=["accuracy"])

#fit ANN to training set
model_history=classifier.fit(X_train,y_train,validation_split=0.
 ↪33,batch_size=10,epochs=100)
classifier.summary()

#list all data in history
print(model_history.history.keys())

#summarize for accuracy
plt.plot(model_history.history['accuracy'])
plt.plot(model_history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train','test'],loc='upper left')
plt.show()

#summarize for loss
plt.plot(model_history.history['loss'])
plt.plot(model_history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train','test'],loc='upper left')
plt.show()
```

```
y_pred=classifier.predict(X_test)
y_pred=(y_pred>0.5)

from sklearn.metrics import confusion_matrix,accuracy_score
cm=confusion_matrix(y_test,y_pred)
score=accuracy_score(y_pred,y_test)

print(cm)
print(score)
```

```
2024-12-14 17:42:33.095460: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
2024-12-14 17:42:33.659183: E
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8454] Unable to register
cuDNN factory: Attempting to register factory for plugin cuDNN when one has
already been registered
2024-12-14 17:42:33.839270: E
external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1452] Unable to
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
2024-12-14 17:42:35.100378: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2024-12-14 17:42:42.259088: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not
find TensorRT
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/100
536/536              4s 4ms/step -
accuracy: 0.2738 - loss: 1.2502 - val_accuracy: 0.6717 - val_loss: 0.6313
Epoch 2/100
536/536              2s 3ms/step -
accuracy: 0.7339 - loss: 0.5762 - val_accuracy: 0.7982 - val_loss: 0.4881
Epoch 3/100
536/536              2s 3ms/step -
accuracy: 0.8010 - loss: 0.4733 - val_accuracy: 0.8012 - val_loss: 0.4652
Epoch 4/100
536/536              2s 3ms/step -
```

```
accuracy: 0.7985 - loss: 0.4564 - val_accuracy: 0.8031 - val_loss: 0.4522
Epoch 5/100
536/536             2s 3ms/step -
accuracy: 0.8090 - loss: 0.4431 - val_accuracy: 0.8069 - val_loss: 0.4412
Epoch 6/100
536/536             2s 3ms/step -
accuracy: 0.8118 - loss: 0.4289 - val_accuracy: 0.8129 - val_loss: 0.4326
Epoch 7/100
536/536             2s 4ms/step -
accuracy: 0.8131 - loss: 0.4224 - val_accuracy: 0.8141 - val_loss: 0.4251
Epoch 8/100
536/536             2s 3ms/step -
accuracy: 0.8062 - loss: 0.4215 - val_accuracy: 0.8129 - val_loss: 0.4195
Epoch 9/100
536/536             2s 3ms/step -
accuracy: 0.8201 - loss: 0.4112 - val_accuracy: 0.8183 - val_loss: 0.4155
Epoch 10/100
536/536             3s 3ms/step -
accuracy: 0.8252 - loss: 0.3933 - val_accuracy: 0.8217 - val_loss: 0.4118
Epoch 11/100
536/536             2s 3ms/step -
accuracy: 0.8221 - loss: 0.4087 - val_accuracy: 0.8217 - val_loss: 0.4093
Epoch 12/100
536/536             2s 3ms/step -
accuracy: 0.8289 - loss: 0.3964 - val_accuracy: 0.8239 - val_loss: 0.4065
Epoch 13/100
536/536             1s 3ms/step -
accuracy: 0.8276 - loss: 0.3982 - val_accuracy: 0.8239 - val_loss: 0.4042
Epoch 14/100
536/536             1s 3ms/step -
accuracy: 0.8393 - loss: 0.3749 - val_accuracy: 0.8258 - val_loss: 0.4022
Epoch 15/100
536/536             1s 3ms/step -
accuracy: 0.8365 - loss: 0.3838 - val_accuracy: 0.8247 - val_loss: 0.4005
Epoch 16/100
536/536             1s 3ms/step -
accuracy: 0.8445 - loss: 0.3726 - val_accuracy: 0.8277 - val_loss: 0.3985
Epoch 17/100
536/536             2s 3ms/step -
accuracy: 0.8382 - loss: 0.3804 - val_accuracy: 0.8266 - val_loss: 0.3971
Epoch 18/100
536/536             2s 3ms/step -
accuracy: 0.8404 - loss: 0.3695 - val_accuracy: 0.8270 - val_loss: 0.3959
Epoch 19/100
536/536             1s 3ms/step -
accuracy: 0.8416 - loss: 0.3752 - val_accuracy: 0.8281 - val_loss: 0.3946
Epoch 20/100
536/536             1s 2ms/step -
```

```
accuracy: 0.8368 - loss: 0.3743 - val_accuracy: 0.8292 - val_loss: 0.3935
Epoch 21/100
536/536            1s 3ms/step -
accuracy: 0.8446 - loss: 0.3682 - val_accuracy: 0.8300 - val_loss: 0.3925
Epoch 22/100
536/536            1s 3ms/step -
accuracy: 0.8471 - loss: 0.3645 - val_accuracy: 0.8289 - val_loss: 0.3914
Epoch 23/100
536/536            1s 3ms/step -
accuracy: 0.8528 - loss: 0.3464 - val_accuracy: 0.8307 - val_loss: 0.3907
Epoch 24/100
536/536            2s 3ms/step -
accuracy: 0.8458 - loss: 0.3649 - val_accuracy: 0.8304 - val_loss: 0.3900
Epoch 25/100
536/536            2s 3ms/step -
accuracy: 0.8400 - loss: 0.3643 - val_accuracy: 0.8311 - val_loss: 0.3889
Epoch 26/100
536/536            1s 3ms/step -
accuracy: 0.8447 - loss: 0.3630 - val_accuracy: 0.8319 - val_loss: 0.3885
Epoch 27/100
536/536            1s 2ms/step -
accuracy: 0.8467 - loss: 0.3600 - val_accuracy: 0.8323 - val_loss: 0.3877
Epoch 28/100
536/536            1s 3ms/step -
accuracy: 0.8533 - loss: 0.3553 - val_accuracy: 0.8334 - val_loss: 0.3872
Epoch 29/100
536/536            1s 3ms/step -
accuracy: 0.8422 - loss: 0.3769 - val_accuracy: 0.8353 - val_loss: 0.3863
Epoch 30/100
536/536            1s 3ms/step -
accuracy: 0.8436 - loss: 0.3608 - val_accuracy: 0.8360 - val_loss: 0.3857
Epoch 31/100
536/536            1s 3ms/step -
accuracy: 0.8512 - loss: 0.3631 - val_accuracy: 0.8360 - val_loss: 0.3853
Epoch 32/100
536/536            1s 3ms/step -
accuracy: 0.8520 - loss: 0.3480 - val_accuracy: 0.8360 - val_loss: 0.3846
Epoch 33/100
536/536            1s 3ms/step -
accuracy: 0.8465 - loss: 0.3586 - val_accuracy: 0.8353 - val_loss: 0.3839
Epoch 34/100
536/536            1s 3ms/step -
accuracy: 0.8538 - loss: 0.3560 - val_accuracy: 0.8360 - val_loss: 0.3834
Epoch 35/100
536/536            1s 3ms/step -
accuracy: 0.8483 - loss: 0.3564 - val_accuracy: 0.8338 - val_loss: 0.3831
Epoch 36/100
536/536            2s 3ms/step -
```

```
accuracy: 0.8479 - loss: 0.3672 - val_accuracy: 0.8368 - val_loss: 0.3825
Epoch 37/100
536/536              2s 3ms/step -
accuracy: 0.8483 - loss: 0.3541 - val_accuracy: 0.8353 - val_loss: 0.3821
Epoch 38/100
536/536              2s 4ms/step -
accuracy: 0.8523 - loss: 0.3500 - val_accuracy: 0.8360 - val_loss: 0.3818
Epoch 39/100
536/536              1s 3ms/step -
accuracy: 0.8476 - loss: 0.3584 - val_accuracy: 0.8357 - val_loss: 0.3813
Epoch 40/100
536/536              1s 3ms/step -
accuracy: 0.8490 - loss: 0.3552 - val_accuracy: 0.8360 - val_loss: 0.3812
Epoch 41/100
536/536              2s 3ms/step -
accuracy: 0.8469 - loss: 0.3536 - val_accuracy: 0.8353 - val_loss: 0.3809
Epoch 42/100
536/536              2s 3ms/step -
accuracy: 0.8567 - loss: 0.3398 - val_accuracy: 0.8376 - val_loss: 0.3805
Epoch 43/100
536/536              2s 3ms/step -
accuracy: 0.8492 - loss: 0.3503 - val_accuracy: 0.8383 - val_loss: 0.3801
Epoch 44/100
536/536              2s 3ms/step -
accuracy: 0.8459 - loss: 0.3515 - val_accuracy: 0.8372 - val_loss: 0.3800
Epoch 45/100
536/536              1s 3ms/step -
accuracy: 0.8499 - loss: 0.3584 - val_accuracy: 0.8383 - val_loss: 0.3796
Epoch 46/100
536/536              1s 3ms/step -
accuracy: 0.8522 - loss: 0.3472 - val_accuracy: 0.8379 - val_loss: 0.3793
Epoch 47/100
536/536              1s 3ms/step -
accuracy: 0.8522 - loss: 0.3485 - val_accuracy: 0.8387 - val_loss: 0.3793
Epoch 48/100
536/536              1s 3ms/step -
accuracy: 0.8429 - loss: 0.3602 - val_accuracy: 0.8402 - val_loss: 0.3788
Epoch 49/100
536/536              1s 3ms/step -
accuracy: 0.8594 - loss: 0.3390 - val_accuracy: 0.8379 - val_loss: 0.3787
Epoch 50/100
536/536              2s 3ms/step -
accuracy: 0.8576 - loss: 0.3433 - val_accuracy: 0.8413 - val_loss: 0.3783
Epoch 51/100
536/536              1s 3ms/step -
accuracy: 0.8547 - loss: 0.3518 - val_accuracy: 0.8448 - val_loss: 0.3785
Epoch 52/100
536/536              1s 3ms/step -
```

```
accuracy: 0.8585 - loss: 0.3358 - val_accuracy: 0.8398 - val_loss: 0.3783
Epoch 53/100
536/536            1s 3ms/step -
accuracy: 0.8498 - loss: 0.3499 - val_accuracy: 0.8429 - val_loss: 0.3775
Epoch 54/100
536/536            2s 3ms/step -
accuracy: 0.8500 - loss: 0.3521 - val_accuracy: 0.8432 - val_loss: 0.3773
Epoch 55/100
536/536            2s 3ms/step -
accuracy: 0.8588 - loss: 0.3353 - val_accuracy: 0.8421 - val_loss: 0.3769
Epoch 56/100
536/536            2s 3ms/step -
accuracy: 0.8531 - loss: 0.3556 - val_accuracy: 0.8410 - val_loss: 0.3766
Epoch 57/100
536/536            2s 3ms/step -
accuracy: 0.8552 - loss: 0.3468 - val_accuracy: 0.8440 - val_loss: 0.3762
Epoch 58/100
536/536            2s 3ms/step -
accuracy: 0.8569 - loss: 0.3424 - val_accuracy: 0.8429 - val_loss: 0.3759
Epoch 59/100
536/536            2s 3ms/step -
accuracy: 0.8517 - loss: 0.3433 - val_accuracy: 0.8448 - val_loss: 0.3755
Epoch 60/100
536/536            1s 3ms/step -
accuracy: 0.8584 - loss: 0.3342 - val_accuracy: 0.8432 - val_loss: 0.3753
Epoch 61/100
536/536            1s 3ms/step -
accuracy: 0.8553 - loss: 0.3308 - val_accuracy: 0.8421 - val_loss: 0.3750
Epoch 62/100
536/536            1s 3ms/step -
accuracy: 0.8552 - loss: 0.3386 - val_accuracy: 0.8444 - val_loss: 0.3746
Epoch 63/100
536/536            1s 3ms/step -
accuracy: 0.8577 - loss: 0.3406 - val_accuracy: 0.8444 - val_loss: 0.3744
Epoch 64/100
536/536            3s 3ms/step -
accuracy: 0.8582 - loss: 0.3402 - val_accuracy: 0.8440 - val_loss: 0.3743
Epoch 65/100
536/536            1s 3ms/step -
accuracy: 0.8627 - loss: 0.3355 - val_accuracy: 0.8440 - val_loss: 0.3742
Epoch 66/100
536/536            1s 3ms/step -
accuracy: 0.8537 - loss: 0.3387 - val_accuracy: 0.8455 - val_loss: 0.3740
Epoch 67/100
536/536            2s 3ms/step -
accuracy: 0.8621 - loss: 0.3386 - val_accuracy: 0.8455 - val_loss: 0.3740
Epoch 68/100
536/536            2s 3ms/step -
```

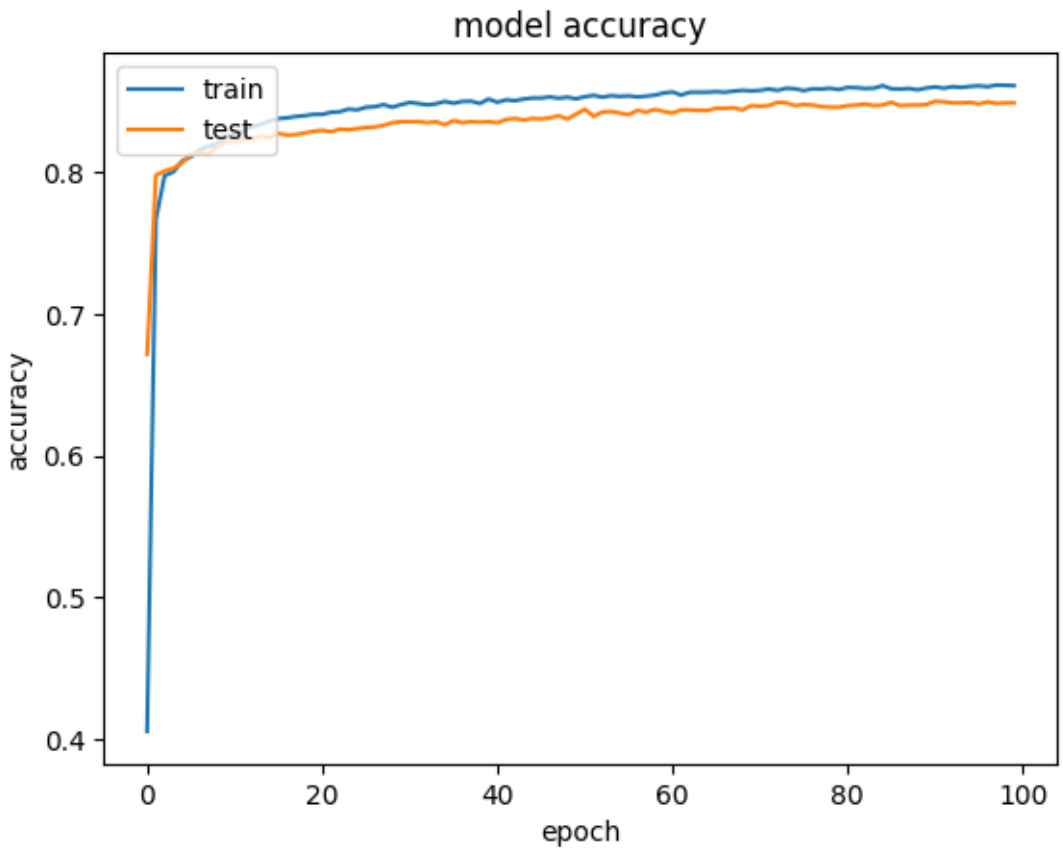
```
accuracy: 0.8508 - loss: 0.3506 - val_accuracy: 0.8459 - val_loss: 0.3737
Epoch 69/100
536/536            1s 3ms/step -
accuracy: 0.8562 - loss: 0.3435 - val_accuracy: 0.8444 - val_loss: 0.3745
Epoch 70/100
536/536            1s 3ms/step -
accuracy: 0.8555 - loss: 0.3442 - val_accuracy: 0.8474 - val_loss: 0.3732
Epoch 71/100
536/536            3s 3ms/step -
accuracy: 0.8600 - loss: 0.3347 - val_accuracy: 0.8470 - val_loss: 0.3732
Epoch 72/100
536/536            1s 3ms/step -
accuracy: 0.8627 - loss: 0.3311 - val_accuracy: 0.8474 - val_loss: 0.3732
Epoch 73/100
536/536            2s 3ms/step -
accuracy: 0.8526 - loss: 0.3466 - val_accuracy: 0.8497 - val_loss: 0.3729
Epoch 74/100
536/536            1s 3ms/step -
accuracy: 0.8644 - loss: 0.3293 - val_accuracy: 0.8493 - val_loss: 0.3726
Epoch 75/100
536/536            1s 3ms/step -
accuracy: 0.8644 - loss: 0.3315 - val_accuracy: 0.8474 - val_loss: 0.3725
Epoch 76/100
536/536            1s 3ms/step -
accuracy: 0.8609 - loss: 0.3371 - val_accuracy: 0.8482 - val_loss: 0.3723
Epoch 77/100
536/536            1s 3ms/step -
accuracy: 0.8605 - loss: 0.3425 - val_accuracy: 0.8478 - val_loss: 0.3722
Epoch 78/100
536/536            1s 3ms/step -
accuracy: 0.8666 - loss: 0.3320 - val_accuracy: 0.8466 - val_loss: 0.3721
Epoch 79/100
536/536            1s 3ms/step -
accuracy: 0.8647 - loss: 0.3288 - val_accuracy: 0.8463 - val_loss: 0.3719
Epoch 80/100
536/536            3s 3ms/step -
accuracy: 0.8552 - loss: 0.3445 - val_accuracy: 0.8463 - val_loss: 0.3720
Epoch 81/100
536/536            2s 3ms/step -
accuracy: 0.8600 - loss: 0.3403 - val_accuracy: 0.8474 - val_loss: 0.3719
Epoch 82/100
536/536            2s 3ms/step -
accuracy: 0.8625 - loss: 0.3314 - val_accuracy: 0.8478 - val_loss: 0.3718
Epoch 83/100
536/536            1s 3ms/step -
accuracy: 0.8551 - loss: 0.3390 - val_accuracy: 0.8485 - val_loss: 0.3714
Epoch 84/100
536/536            1s 3ms/step -
```

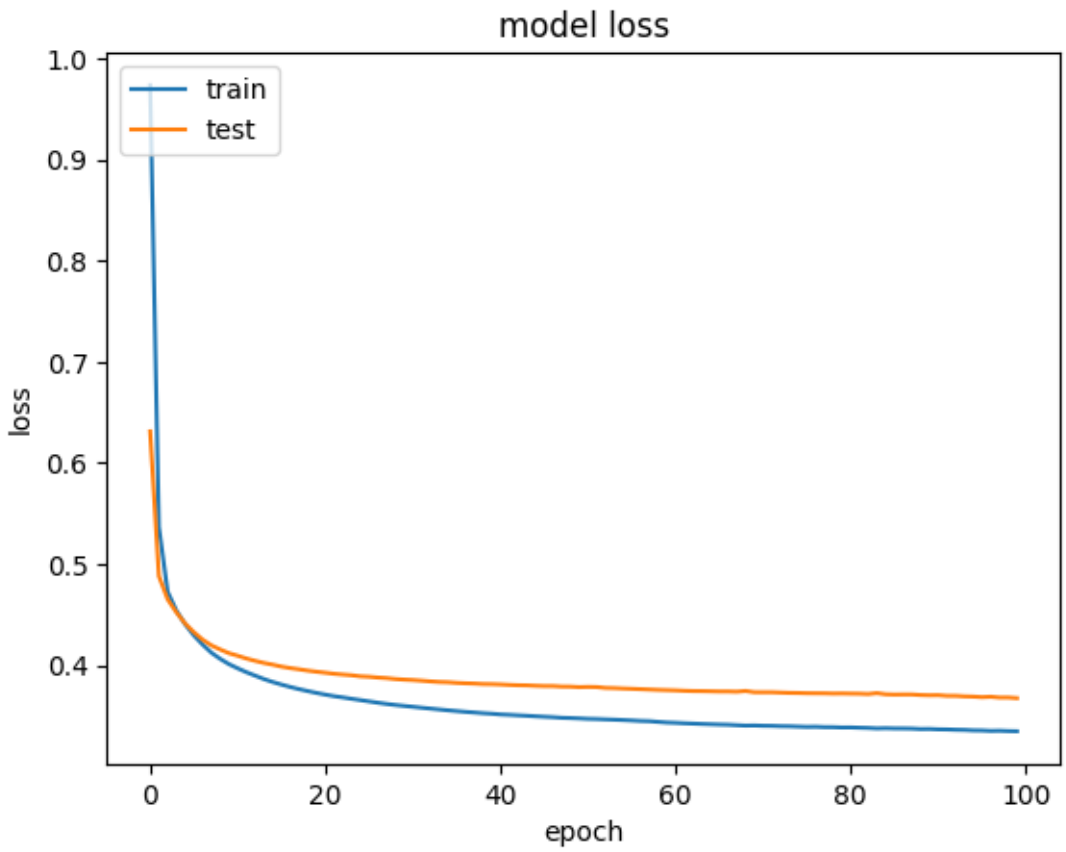
```
accuracy: 0.8641 - loss: 0.3266 - val_accuracy: 0.8474 - val_loss: 0.3723
Epoch 85/100
536/536            1s 3ms/step -
accuracy: 0.8584 - loss: 0.3432 - val_accuracy: 0.8478 - val_loss: 0.3712
Epoch 86/100
536/536            1s 3ms/step -
accuracy: 0.8582 - loss: 0.3398 - val_accuracy: 0.8497 - val_loss: 0.3709
Epoch 87/100
536/536            1s 3ms/step -
accuracy: 0.8597 - loss: 0.3375 - val_accuracy: 0.8474 - val_loss: 0.3710
Epoch 88/100
536/536            3s 3ms/step -
accuracy: 0.8669 - loss: 0.3217 - val_accuracy: 0.8478 - val_loss: 0.3710
Epoch 89/100
536/536            1s 3ms/step -
accuracy: 0.8578 - loss: 0.3368 - val_accuracy: 0.8478 - val_loss: 0.3705
Epoch 90/100
536/536            1s 3ms/step -
accuracy: 0.8640 - loss: 0.3305 - val_accuracy: 0.8482 - val_loss: 0.3702
Epoch 91/100
536/536            2s 3ms/step -
accuracy: 0.8660 - loss: 0.3368 - val_accuracy: 0.8504 - val_loss: 0.3703
Epoch 92/100
536/536            2s 3ms/step -
accuracy: 0.8667 - loss: 0.3199 - val_accuracy: 0.8501 - val_loss: 0.3696
Epoch 93/100
536/536            2s 3ms/step -
accuracy: 0.8619 - loss: 0.3356 - val_accuracy: 0.8493 - val_loss: 0.3696
Epoch 94/100
536/536            2s 3ms/step -
accuracy: 0.8616 - loss: 0.3389 - val_accuracy: 0.8493 - val_loss: 0.3691
Epoch 95/100
536/536            1s 3ms/step -
accuracy: 0.8603 - loss: 0.3366 - val_accuracy: 0.8497 - val_loss: 0.3689
Epoch 96/100
536/536            1s 3ms/step -
accuracy: 0.8593 - loss: 0.3357 - val_accuracy: 0.8485 - val_loss: 0.3684
Epoch 97/100
536/536            1s 3ms/step -
accuracy: 0.8602 - loss: 0.3318 - val_accuracy: 0.8501 - val_loss: 0.3687
Epoch 98/100
536/536            1s 3ms/step -
accuracy: 0.8552 - loss: 0.3461 - val_accuracy: 0.8489 - val_loss: 0.3680
Epoch 99/100
536/536            1s 3ms/step -
accuracy: 0.8667 - loss: 0.3251 - val_accuracy: 0.8493 - val_loss: 0.3680
Epoch 100/100
536/536            2s 3ms/step -
```

```
accuracy: 0.8701 - loss: 0.3198 - val_accuracy: 0.8493 - val_loss: 0.3674
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6)	72
dense_1 (Dense)	(None, 6)	42
dense_2 (Dense)	(None, 1)	7

```
Total params: 365 (1.43 KB)

Trainable params: 121 (484.00 B)

Non-trainable params: 0 (0.00 B)

Optimizer params: 244 (980.00 B)

dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```


model accuracy
train
test
accuracy
0.8
0.7
0.6
0.5
0.4
0
20
40
60
80
100
epoch




```
63/63              0s 2ms/step
[[1519   76]
 [ 215  190]]
0.8545
```

ANN-Dropout

[18]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("Churn_Modelling.csv")
X=dataset.iloc[:,3:13]
y=dataset.iloc[:,13]

#create dummy variables
geography=pd.get_dummies(X["Geography"],drop_first=True)
gender=pd.get_dummies(X["Gender"],drop_first=True)

#concat dataframes
X=pd.concat([X,geography,gender],axis=1)
```

```python
#drop unecessary cols
X=X.drop(['Geography','Gender'],axis=1)

#split train test
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)

#feature scaling

from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)

##ANN

import keras
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LeakyReLU,PReLU,ELU
from keras.layers import Dropout

#initialise ANN
classifier=Sequential()

#add input and 1st hidden layer
#classifier.
 ↪add(Dense(output_dim=6,init="he_uniform",activation="relu",input_dim=11))
classifier.
 ↪add(Dense(units=10,kernel_initializer="he_normal",activation="relu",input_dim=11))
classifier.add(Dropout(0.3))

#add 2nd hidden layer
classifier.add(Dense(units=20,kernel_initializer="he_normal",activation="relu"))
classifier.add(Dropout(0.4))

#add 3rd hidden layer
classifier.add(Dense(units=15,kernel_initializer="he_normal",activation="relu"))
classifier.add(Dropout(0.2))

#add output layer
classifier.
 ↪add(Dense(units=1,kernel_initializer="glorot_uniform",activation="sigmoid"))

#compile ANN
classifier.
 ↪compile(optimizer="Adamax",loss="binary_crossentropy",metrics=["accuracy"])
```

```python
#fit ANN to training set
model_history=classifier.fit(X_train,y_train,validation_split=0.
↪33,batch_size=10,epochs=100)
classifier.summary()

#list all data in history
print(model_history.history.keys())

#summarize for accuracy
plt.plot(model_history.history['accuracy'])
plt.plot(model_history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train','test'],loc='upper left')
plt.show()

#summarize for loss
plt.plot(model_history.history['loss'])
plt.plot(model_history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train','test'],loc='upper left')
plt.show()


y_pred=classifier.predict(X_test)
y_pred=(y_pred>0.5)

from sklearn.metrics import confusion_matrix,accuracy_score
cm=confusion_matrix(y_test,y_pred)
score=accuracy_score(y_pred,y_test)

print(cm,score)
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/100
536/536              6s 5ms/step -
accuracy: 0.7158 - loss: 0.6460 - val_accuracy: 0.7955 - val_loss: 0.5621
Epoch 2/100
536/536              2s 3ms/step -
```

```
accuracy: 0.7899 - loss: 0.5803 - val_accuracy: 0.7955 - val_loss: 0.5366
Epoch 3/100
536/536            2s 3ms/step -
accuracy: 0.7933 - loss: 0.5540 - val_accuracy: 0.7955 - val_loss: 0.5205
Epoch 4/100
536/536            2s 3ms/step -
accuracy: 0.7906 - loss: 0.5348 - val_accuracy: 0.7955 - val_loss: 0.5068
Epoch 5/100
536/536            2s 3ms/step -
accuracy: 0.7978 - loss: 0.5141 - val_accuracy: 0.7955 - val_loss: 0.4993
Epoch 6/100
536/536            2s 3ms/step -
accuracy: 0.7924 - loss: 0.5238 - val_accuracy: 0.7955 - val_loss: 0.4918
Epoch 7/100
536/536            2s 3ms/step -
accuracy: 0.7995 - loss: 0.5053 - val_accuracy: 0.7955 - val_loss: 0.4860
Epoch 8/100
536/536            2s 3ms/step -
accuracy: 0.8059 - loss: 0.4897 - val_accuracy: 0.7955 - val_loss: 0.4814
Epoch 9/100
536/536            2s 4ms/step -
accuracy: 0.7880 - loss: 0.5091 - val_accuracy: 0.7955 - val_loss: 0.4771
Epoch 10/100
536/536            2s 3ms/step -
accuracy: 0.7859 - loss: 0.5075 - val_accuracy: 0.7955 - val_loss: 0.4720
Epoch 11/100
536/536            2s 3ms/step -
accuracy: 0.7898 - loss: 0.4979 - val_accuracy: 0.7955 - val_loss: 0.4693
Epoch 12/100
536/536            3s 3ms/step -
accuracy: 0.7984 - loss: 0.4818 - val_accuracy: 0.7959 - val_loss: 0.4653
Epoch 13/100
536/536            2s 4ms/step -
accuracy: 0.7989 - loss: 0.4847 - val_accuracy: 0.7963 - val_loss: 0.4634
Epoch 14/100
536/536            4s 4ms/step -
accuracy: 0.7919 - loss: 0.4825 - val_accuracy: 0.7963 - val_loss: 0.4593
Epoch 15/100
536/536            3s 4ms/step -
accuracy: 0.7845 - loss: 0.4939 - val_accuracy: 0.7963 - val_loss: 0.4562
Epoch 16/100
536/536            2s 3ms/step -
accuracy: 0.7943 - loss: 0.4867 - val_accuracy: 0.7963 - val_loss: 0.4559
Epoch 17/100
536/536            2s 4ms/step -
accuracy: 0.7952 - loss: 0.4766 - val_accuracy: 0.7959 - val_loss: 0.4517
Epoch 18/100
536/536            2s 4ms/step -
```

```
accuracy: 0.7955 - loss: 0.4708 - val_accuracy: 0.7967 - val_loss: 0.4501
Epoch 19/100
536/536              2s 4ms/step -
accuracy: 0.8046 - loss: 0.4634 - val_accuracy: 0.7978 - val_loss: 0.4470
Epoch 20/100
536/536              5s 5ms/step -
accuracy: 0.8018 - loss: 0.4712 - val_accuracy: 0.7978 - val_loss: 0.4457
Epoch 21/100
536/536              3s 6ms/step -
accuracy: 0.7950 - loss: 0.4756 - val_accuracy: 0.7978 - val_loss: 0.4444
Epoch 22/100
536/536              3s 5ms/step -
accuracy: 0.8046 - loss: 0.4555 - val_accuracy: 0.7986 - val_loss: 0.4431
Epoch 23/100
536/536              3s 5ms/step -
accuracy: 0.8094 - loss: 0.4521 - val_accuracy: 0.7982 - val_loss: 0.4425
Epoch 24/100
536/536              5s 4ms/step -
accuracy: 0.8017 - loss: 0.4600 - val_accuracy: 0.8020 - val_loss: 0.4404
Epoch 25/100
536/536              2s 3ms/step -
accuracy: 0.7976 - loss: 0.4709 - val_accuracy: 0.7997 - val_loss: 0.4397
Epoch 26/100
536/536              3s 4ms/step -
accuracy: 0.8003 - loss: 0.4632 - val_accuracy: 0.8020 - val_loss: 0.4373
Epoch 27/100
536/536              2s 3ms/step -
accuracy: 0.7991 - loss: 0.4615 - val_accuracy: 0.8042 - val_loss: 0.4357
Epoch 28/100
536/536              2s 4ms/step -
accuracy: 0.8037 - loss: 0.4490 - val_accuracy: 0.8023 - val_loss: 0.4356
Epoch 29/100
536/536              2s 3ms/step -
accuracy: 0.7918 - loss: 0.4616 - val_accuracy: 0.8016 - val_loss: 0.4349
Epoch 30/100
536/536              2s 3ms/step -
accuracy: 0.8033 - loss: 0.4559 - val_accuracy: 0.8035 - val_loss: 0.4341
Epoch 31/100
536/536              2s 4ms/step -
accuracy: 0.8033 - loss: 0.4435 - val_accuracy: 0.8050 - val_loss: 0.4336
Epoch 32/100
536/536              2s 4ms/step -
accuracy: 0.8024 - loss: 0.4546 - val_accuracy: 0.8035 - val_loss: 0.4325
Epoch 33/100
536/536              2s 4ms/step -
accuracy: 0.8069 - loss: 0.4377 - val_accuracy: 0.8050 - val_loss: 0.4317
Epoch 34/100
536/536              2s 4ms/step -
```

```
accuracy: 0.8018 - loss: 0.4556 - val_accuracy: 0.8050 - val_loss: 0.4308
Epoch 35/100
536/536            2s 4ms/step -
accuracy: 0.8012 - loss: 0.4580 - val_accuracy: 0.8050 - val_loss: 0.4305
Epoch 36/100
536/536            2s 4ms/step -
accuracy: 0.8159 - loss: 0.4302 - val_accuracy: 0.8069 - val_loss: 0.4288
Epoch 37/100
536/536            2s 4ms/step -
accuracy: 0.7989 - loss: 0.4471 - val_accuracy: 0.8050 - val_loss: 0.4288
Epoch 38/100
536/536            2s 4ms/step -
accuracy: 0.8019 - loss: 0.4417 - val_accuracy: 0.8061 - val_loss: 0.4280
Epoch 39/100
536/536            3s 4ms/step -
accuracy: 0.8053 - loss: 0.4351 - val_accuracy: 0.8092 - val_loss: 0.4267
Epoch 40/100
536/536            2s 4ms/step -
accuracy: 0.8080 - loss: 0.4416 - val_accuracy: 0.8107 - val_loss: 0.4256
Epoch 41/100
536/536            2s 4ms/step -
accuracy: 0.8096 - loss: 0.4376 - val_accuracy: 0.8069 - val_loss: 0.4250
Epoch 42/100
536/536            2s 4ms/step -
accuracy: 0.8017 - loss: 0.4425 - val_accuracy: 0.8084 - val_loss: 0.4245
Epoch 43/100
536/536            3s 4ms/step -
accuracy: 0.7962 - loss: 0.4502 - val_accuracy: 0.8092 - val_loss: 0.4239
Epoch 44/100
536/536            3s 5ms/step -
accuracy: 0.8086 - loss: 0.4433 - val_accuracy: 0.8092 - val_loss: 0.4238
Epoch 45/100
536/536            3s 5ms/step -
accuracy: 0.8149 - loss: 0.4356 - val_accuracy: 0.8092 - val_loss: 0.4232
Epoch 46/100
536/536            2s 5ms/step -
accuracy: 0.8116 - loss: 0.4396 - val_accuracy: 0.8111 - val_loss: 0.4223
Epoch 47/100
536/536            2s 4ms/step -
accuracy: 0.8081 - loss: 0.4376 - val_accuracy: 0.8095 - val_loss: 0.4218
Epoch 48/100
536/536            2s 4ms/step -
accuracy: 0.8155 - loss: 0.4266 - val_accuracy: 0.8122 - val_loss: 0.4207
Epoch 49/100
536/536            2s 4ms/step -
accuracy: 0.8091 - loss: 0.4311 - val_accuracy: 0.8114 - val_loss: 0.4200
Epoch 50/100
536/536            3s 4ms/step -
```

```
accuracy: 0.8189 - loss: 0.4293 - val_accuracy: 0.8122 - val_loss: 0.4193
Epoch 51/100
536/536            2s 4ms/step -
accuracy: 0.8135 - loss: 0.4337 - val_accuracy: 0.8118 - val_loss: 0.4189
Epoch 52/100
536/536            2s 3ms/step -
accuracy: 0.8063 - loss: 0.4306 - val_accuracy: 0.8133 - val_loss: 0.4178
Epoch 53/100
536/536            2s 3ms/step -
accuracy: 0.8168 - loss: 0.4315 - val_accuracy: 0.8133 - val_loss: 0.4170
Epoch 54/100
536/536            2s 3ms/step -
accuracy: 0.8156 - loss: 0.4403 - val_accuracy: 0.8137 - val_loss: 0.4166
Epoch 55/100
536/536            2s 3ms/step -
accuracy: 0.8131 - loss: 0.4226 - val_accuracy: 0.8148 - val_loss: 0.4151
Epoch 56/100
536/536            2s 3ms/step -
accuracy: 0.8147 - loss: 0.4241 - val_accuracy: 0.8152 - val_loss: 0.4147
Epoch 57/100
536/536            2s 4ms/step -
accuracy: 0.8037 - loss: 0.4327 - val_accuracy: 0.8156 - val_loss: 0.4137
Epoch 58/100
536/536            2s 3ms/step -
accuracy: 0.8144 - loss: 0.4261 - val_accuracy: 0.8160 - val_loss: 0.4141
Epoch 59/100
536/536            2s 3ms/step -
accuracy: 0.8264 - loss: 0.4229 - val_accuracy: 0.8164 - val_loss: 0.4132
Epoch 60/100
536/536            2s 3ms/step -
accuracy: 0.8143 - loss: 0.4312 - val_accuracy: 0.8160 - val_loss: 0.4126
Epoch 61/100
536/536            2s 4ms/step -
accuracy: 0.8079 - loss: 0.4375 - val_accuracy: 0.8164 - val_loss: 0.4126
Epoch 62/100
536/536            2s 4ms/step -
accuracy: 0.8141 - loss: 0.4206 - val_accuracy: 0.8171 - val_loss: 0.4114
Epoch 63/100
536/536            2s 3ms/step -
accuracy: 0.8129 - loss: 0.4301 - val_accuracy: 0.8167 - val_loss: 0.4108
Epoch 64/100
536/536            2s 3ms/step -
accuracy: 0.8123 - loss: 0.4234 - val_accuracy: 0.8183 - val_loss: 0.4104
Epoch 65/100
536/536            2s 3ms/step -
accuracy: 0.8214 - loss: 0.4193 - val_accuracy: 0.8183 - val_loss: 0.4092
Epoch 66/100
536/536            2s 3ms/step -
```

```
accuracy: 0.8185 - loss: 0.4183 - val_accuracy: 0.8160 - val_loss: 0.4092
Epoch 67/100
536/536             2s 4ms/step -
accuracy: 0.8139 - loss: 0.4286 - val_accuracy: 0.8201 - val_loss: 0.4072
Epoch 68/100
536/536             2s 3ms/step -
accuracy: 0.8168 - loss: 0.4210 - val_accuracy: 0.8205 - val_loss: 0.4063
Epoch 69/100
536/536             2s 3ms/step -
accuracy: 0.8135 - loss: 0.4318 - val_accuracy: 0.8209 - val_loss: 0.4058
Epoch 70/100
536/536             2s 4ms/step -
accuracy: 0.8172 - loss: 0.4194 - val_accuracy: 0.8194 - val_loss: 0.4065
Epoch 71/100
536/536             2s 4ms/step -
accuracy: 0.8133 - loss: 0.4202 - val_accuracy: 0.8228 - val_loss: 0.4050
Epoch 72/100
536/536             2s 3ms/step -
accuracy: 0.8132 - loss: 0.4282 - val_accuracy: 0.8224 - val_loss: 0.4048
Epoch 73/100
536/536             2s 3ms/step -
accuracy: 0.8144 - loss: 0.4204 - val_accuracy: 0.8232 - val_loss: 0.4036
Epoch 74/100
536/536             3s 4ms/step -
accuracy: 0.8329 - loss: 0.3998 - val_accuracy: 0.8239 - val_loss: 0.4024
Epoch 75/100
536/536             3s 5ms/step -
accuracy: 0.8192 - loss: 0.4078 - val_accuracy: 0.8236 - val_loss: 0.4016
Epoch 76/100
536/536             2s 4ms/step -
accuracy: 0.8208 - loss: 0.4171 - val_accuracy: 0.8262 - val_loss: 0.4006
Epoch 77/100
536/536             2s 4ms/step -
accuracy: 0.8184 - loss: 0.4193 - val_accuracy: 0.8239 - val_loss: 0.4007
Epoch 78/100
536/536             2s 4ms/step -
accuracy: 0.8283 - loss: 0.4033 - val_accuracy: 0.8258 - val_loss: 0.3997
Epoch 79/100
536/536             3s 4ms/step -
accuracy: 0.8224 - loss: 0.4101 - val_accuracy: 0.8254 - val_loss: 0.3998
Epoch 80/100
536/536             2s 4ms/step -
accuracy: 0.8273 - loss: 0.4051 - val_accuracy: 0.8258 - val_loss: 0.3990
Epoch 81/100
536/536             2s 4ms/step -
accuracy: 0.8204 - loss: 0.4130 - val_accuracy: 0.8285 - val_loss: 0.3976
Epoch 82/100
536/536             2s 4ms/step -
```

```
accuracy: 0.8297 - loss: 0.3976 - val_accuracy: 0.8266 - val_loss: 0.3975
Epoch 83/100
536/536              2s 4ms/step -
accuracy: 0.8208 - loss: 0.4065 - val_accuracy: 0.8285 - val_loss: 0.3962
Epoch 84/100
536/536              2s 4ms/step -
accuracy: 0.8336 - loss: 0.3902 - val_accuracy: 0.8307 - val_loss: 0.3949
Epoch 85/100
536/536              3s 4ms/step -
accuracy: 0.8162 - loss: 0.4106 - val_accuracy: 0.8273 - val_loss: 0.3961
Epoch 86/100
536/536              2s 4ms/step -
accuracy: 0.8319 - loss: 0.4042 - val_accuracy: 0.8277 - val_loss: 0.3961
Epoch 87/100
536/536              2s 4ms/step -
accuracy: 0.8127 - loss: 0.4159 - val_accuracy: 0.8270 - val_loss: 0.3956
Epoch 88/100
536/536              2s 4ms/step -
accuracy: 0.8232 - loss: 0.4110 - val_accuracy: 0.8304 - val_loss: 0.3943
Epoch 89/100
536/536              3s 4ms/step -
accuracy: 0.8198 - loss: 0.4081 - val_accuracy: 0.8289 - val_loss: 0.3939
Epoch 90/100
536/536              2s 4ms/step -
accuracy: 0.8319 - loss: 0.4006 - val_accuracy: 0.8300 - val_loss: 0.3931
Epoch 91/100
536/536              2s 4ms/step -
accuracy: 0.8281 - loss: 0.3986 - val_accuracy: 0.8292 - val_loss: 0.3929
Epoch 92/100
536/536              3s 4ms/step -
accuracy: 0.8310 - loss: 0.3999 - val_accuracy: 0.8315 - val_loss: 0.3923
Epoch 93/100
536/536              2s 4ms/step -
accuracy: 0.8265 - loss: 0.4116 - val_accuracy: 0.8292 - val_loss: 0.3931
Epoch 94/100
536/536              2s 4ms/step -
accuracy: 0.8281 - loss: 0.4072 - val_accuracy: 0.8326 - val_loss: 0.3912
Epoch 95/100
536/536              2s 4ms/step -
accuracy: 0.8453 - loss: 0.3766 - val_accuracy: 0.8338 - val_loss: 0.3900
Epoch 96/100
536/536              3s 4ms/step -
accuracy: 0.8279 - loss: 0.4079 - val_accuracy: 0.8330 - val_loss: 0.3905
Epoch 97/100
536/536              2s 4ms/step -
accuracy: 0.8235 - loss: 0.3986 - val_accuracy: 0.8342 - val_loss: 0.3899
Epoch 98/100
536/536              3s 4ms/step -
```

```
accuracy: 0.8265 - loss: 0.4021 - val_accuracy: 0.8323 - val_loss: 0.3907
Epoch 99/100
536/536              2s 4ms/step -
accuracy: 0.8384 - loss: 0.3889 - val_accuracy: 0.8364 - val_loss: 0.3891
Epoch 100/100
536/536              3s 4ms/step -
accuracy: 0.8207 - loss: 0.4010 - val_accuracy: 0.8357 - val_loss: 0.3883
```

Model: "sequential_13"

Layer (type)	Output Shape	Param #
dense_34 (Dense)	(None, 10)	120
dropout_6 (Dropout)	(None, 10)	0
dense_35 (Dense)	(None, 20)	220
dropout_7 (Dropout)	(None, 20)	0
dense_36 (Dense)	(None, 15)	315
dropout_8 (Dropout)	(None, 15)	0
dense_37 (Dense)	(None, 1)	16

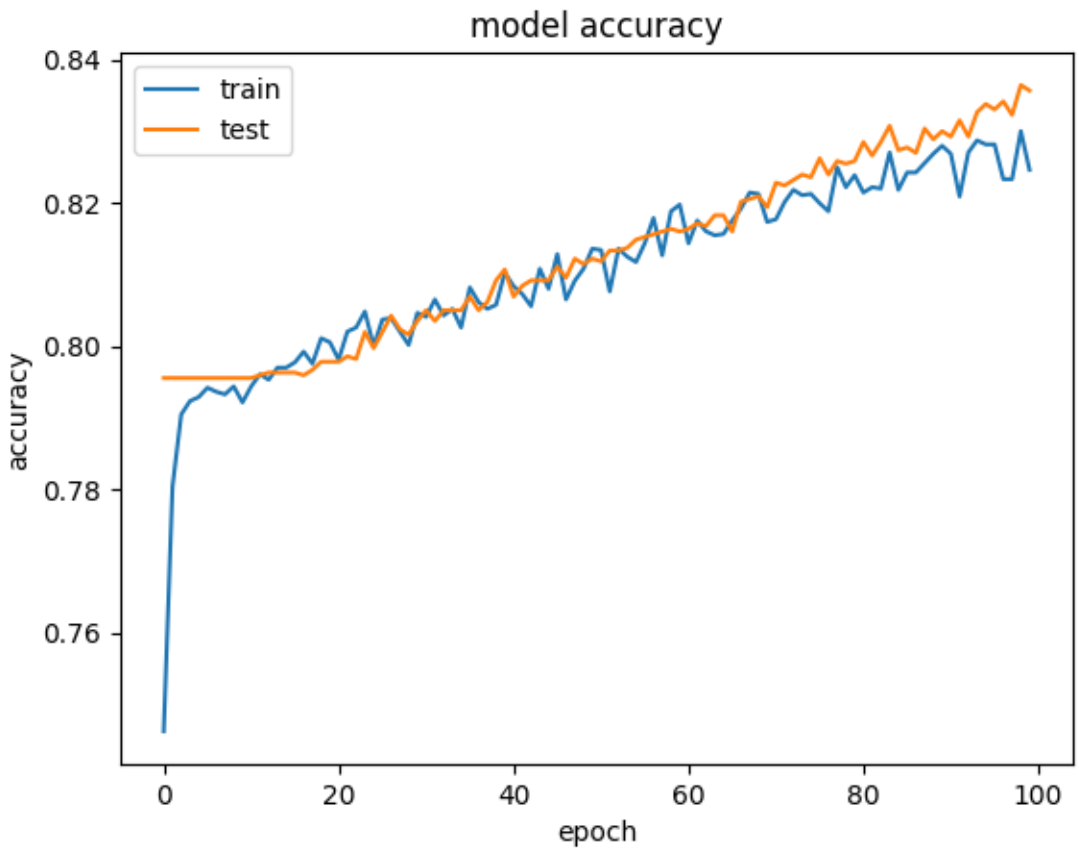
Total params: 2,015 (7.88 KB)

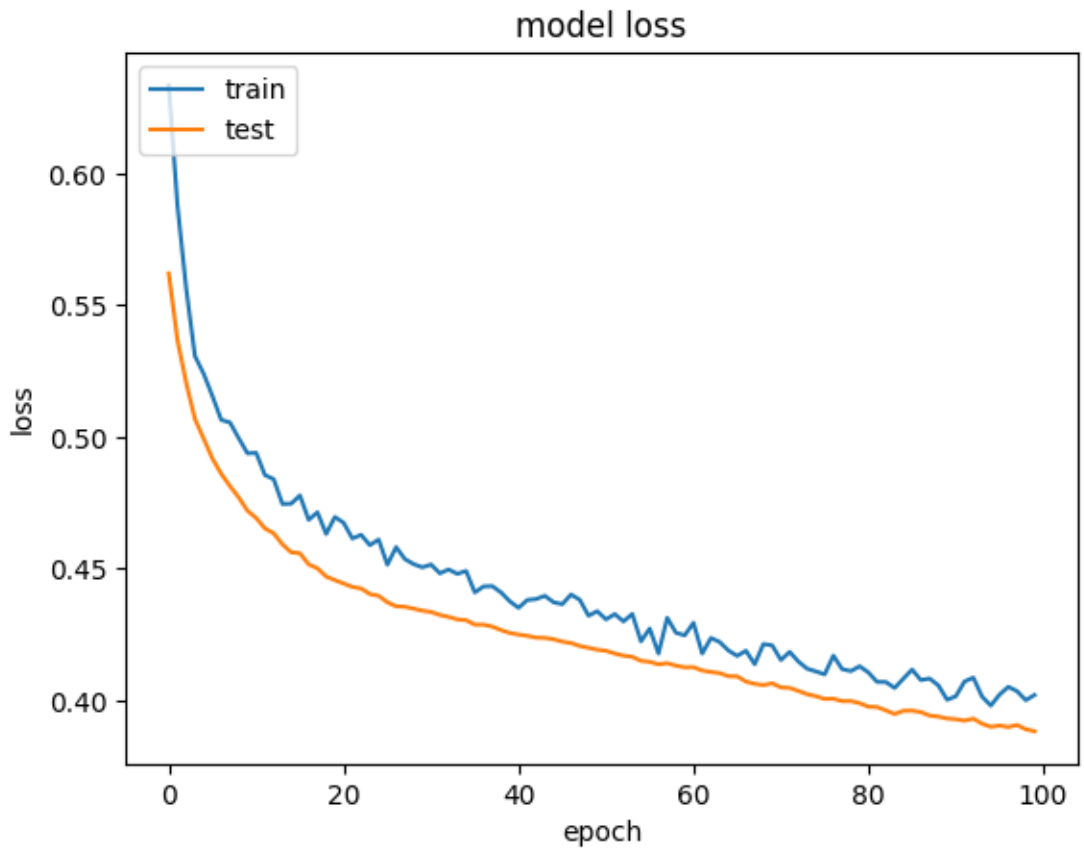
Trainable params: 671 (2.62 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 1,344 (5.25 KB)

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```


model accuracy
train
test
accuracy
epoch
0.84
0.82
0.80
0.78
0.76
0
20
40
60
80
100




```
63/63              0s 3ms/step
[[1586    9]
 [ 307   98]] 0.842
```

1.1 Hyper parameters

1.How many no.of hidden layers we should have? 2.How many no.of neurons we should have in hidden layers? 3.Learning rate

1.1.1 Hidden Layers and Hidden Neurons

[2]:
```
import pandas as pd
from tensorflow import keras
from tensorflow.keras import layers
from keras_tuner import RandomSearch

df=pd.read_csv("Real_Combine.csv")
df.head()

X=df.iloc[:,:-1] #independent features
```

```python
y=df.iloc[:,-1] #dependent features

def build_model(hp):
    model=keras.Sequential()
    for i in range(hp.Int('num_layers',2,20)):
        model.add(layers.Dense(units=hp.Int('units_'+str(i),
                                            min_value=32,
                                            max_value=512,
                                            step=32),
                               activation="relu"))
    model.add(layers.Dense(1,activation="linear"))
    model.compile(
        optimizer=keras.optimizers.Adam(
            hp.Choice('learning_rate',[1e-2,1e-3,1e-4])),
        loss='mean_absolute_error',
        metrics=['mean_absolute_error'])
    return model

tuner = RandomSearch(
    build_model,
    objective="val_mean_absolute_error",
    max_trials=3,
    executions_per_trial=3,
    directory='project',
    project_name='Air Quality Index')

tuner.search_space_summary()

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=0)

tuner.search(X_train,y_train,
             epochs=5,
             validation_data=(X_test,y_test))

tuner.results_summary()
```

```
Reloading Tuner from project/Air Quality Index/tuner0.json
Search space summary
Default search space size: 12
num_layers (Int)
{'default': None, 'conditions': [], 'min_value': 2, 'max_value': 20, 'step': 1,
'sampling': 'linear'}
units_0 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_1 (Int)
```

```
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
learning_rate (Choice)
{'default': 0.01, 'conditions': [], 'values': [0.01, 0.001, 0.0001], 'ordered':
True}
units_2 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_3 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_4 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_5 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_6 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_7 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_8 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}
units_9 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step':
32, 'sampling': 'linear'}

Search: Running Trial #3

Value             |Best Value So Far |Hyperparameter
16                |8                 |num_layers
96                |352               |units_0
320               |128               |units_1
0.0001            |0.0001            |learning_rate
32                |480               |units_2
320               |352               |units_3
448               |384               |units_4
256               |64                |units_5
128               |96                |units_6
384               |512               |units_7
64                |128               |units_8
224               |64                |units_9
32                |None              |units_10
32                |None              |units_11
32                |None              |units_12
```

```
32                |None              |units_13
32                |None              |units_14
32                |None              |units_15

Epoch 1/5
23/23            8s 40ms/step -
loss: 102.3348 - mean_absolute_error: 102.3348 - val_loss: 113.0514 -
val_mean_absolute_error: 113.0514
Epoch 2/5
23/23            1s 27ms/step -
loss: 106.4899 - mean_absolute_error: 106.4899 - val_loss: 111.4027 -
val_mean_absolute_error: 111.4027
Epoch 3/5
23/23            1s 19ms/step -
loss: 103.0735 - mean_absolute_error: 103.0735 - val_loss: 68.8962 -
val_mean_absolute_error: 68.8962
Epoch 4/5
23/23            0s 20ms/step -
loss: 70.3377 - mean_absolute_error: 70.3377 - val_loss: 66.2870 -
val_mean_absolute_error: 66.2870
Epoch 5/5
23/23            1s 19ms/step -
loss: 64.4202 - mean_absolute_error: 64.4202 - val_loss: 64.5094 -
val_mean_absolute_error: 64.5094
Epoch 1/5
23/23            7s 26ms/step -
loss: 104.7406 - mean_absolute_error: 104.7406 - val_loss: 107.0815 -
val_mean_absolute_error: 107.0815
Epoch 2/5
23/23            0s 20ms/step -
loss: 90.2805 - mean_absolute_error: 90.2805 - val_loss: 63.9265 -
val_mean_absolute_error: 63.9265
Epoch 3/5
23/23            0s 15ms/step -
loss: 66.1442 - mean_absolute_error: 66.1442 - val_loss: 65.1838 -
val_mean_absolute_error: 65.1838
Epoch 4/5
23/23            0s 14ms/step -
loss: 59.9438 - mean_absolute_error: 59.9438 - val_loss: 63.9676 -
val_mean_absolute_error: 63.9676
Epoch 5/5
23/23            1s 20ms/step -
loss: 64.5327 - mean_absolute_error: 64.5327 - val_loss: 64.6623 -
val_mean_absolute_error: 64.6623
Epoch 1/5
23/23            7s 28ms/step -
loss: 100.1747 - mean_absolute_error: 100.1747 - val_loss: 113.0354 -
val_mean_absolute_error: 113.0354
```

```
Epoch 2/5
23/23              0s 17ms/step -
loss: 105.8919 - mean_absolute_error: 105.8919 - val_loss: 110.7401 -
val_mean_absolute_error: 110.7401
Epoch 3/5
23/23              0s 15ms/step -
loss: 96.4021 - mean_absolute_error: 96.4021 - val_loss: 64.6812 -
val_mean_absolute_error: 64.6812
Epoch 4/5
23/23              0s 18ms/step -
loss: 63.7497 - mean_absolute_error: 63.7497 - val_loss: 64.0096 -
val_mean_absolute_error: 64.0096
Epoch 5/5
23/23              1s 15ms/step -
loss: 65.0907 - mean_absolute_error: 65.0907 - val_loss: 64.7055 -
val_mean_absolute_error: 64.7055
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras_tuner/src/engine/metrics_tracking.py:111: RuntimeWarning: All-NaN
axis encountered
  np.nanmin(values) if self.direction == "min" else np.nanmax(values)
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[2], line 41
     38 from sklearn.model_selection import train_test_split
     39 X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.
 ↪3,random_state=0)
---> 41 tuner.search(X_train,y_train,
     42              epochs=5,
     43              validation_data=(X_test,y_test))
     45 tuner.results_summary()

File ~/.local/lib/python3.10/site-packages/keras_tuner/src/engine/base_tuner.py
 ↪235, in BaseTuner.search(self, *fit_args, **fit_kwargs)
    233     self.on_trial_begin(trial)
    234     self._try_run_and_update_trial(trial, *fit_args, **fit_kwargs)
--> 235     self.on_trial_end(trial)
    236 self.on_search_end()

File ~/.local/lib/python3.10/site-packages/keras_tuner/src/engine/base_tuner.py
 ↪339, in BaseTuner.on_trial_end(self, trial)
    333 def on_trial_end(self, trial):
    334     """Called at the end of a trial.
    335
    336     Args:
    337         trial: A `Trial` instance.
    338     """
```

```
--> 339     self.oracle.end_trial(trial)
    340     self.save()

File ~/.local/lib/python3.10/site-packages/keras_tuner/src/engine/oracle.py:108
 ↪in synchronized.<locals>.wrapped_func(*args, **kwargs)
    106     LOCKS[oracle].acquire()
    107     THREADS[oracle] = thread_name
--> 108 ret_val = func(*args, **kwargs)
    109 if need_acquire:
    110     THREADS[oracle] = None

File ~/.local/lib/python3.10/site-packages/keras_tuner/src/engine/oracle.py:588
 ↪in Oracle.end_trial(self, trial)
    586 if not self._retry(trial):
    587     self.end_order.append(trial.trial_id)
--> 588     self._check_consecutive_failures()
    590 self._save_trial(trial)
    591 self.save()

File ~/.local/lib/python3.10/site-packages/keras_tuner/src/engine/oracle.py:545
 ↪in Oracle._check_consecutive_failures(self)
    543     consecutive_failures = 0
    544 if consecutive_failures == self.max_consecutive_failed_trials:
--> 545     raise RuntimeError(
    546         "Number of consecutive failures exceeded the limit "
    547         f"of {self.max_consecutive_failed_trials}.\n"
    548         + (trial.message or "")
    549     )

RuntimeError: Number of consecutive failures exceeded the limit of 3.
```

1.1.2 Hyper parameter.py

[25]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("Churn_Modelling.csv")
X=dataset.iloc[:,3:13]
y=dataset.iloc[:,13]

#create dummy variables
geography=pd.get_dummies(X["Geography"],drop_first=True)
gender=pd.get_dummies(X["Gender"],drop_first=True)
```

```python
#concat dataframes
X=pd.concat([X,geography,gender],axis=1)
#drop unecessary cols
X=X.drop(['Geography','Gender'],axis=1)

#split train test
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)

#feature scaling
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)

#perform hyperparameter optimization
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Activation,Dropout
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import GridSearchCV

def create_model(layers,activation):
    model=Sequential()
    for i,nodes in enumerate(layers):
        if i==0:
            model.add(Dense(nodes,input_dim=X_train.shape[1]))
            model.add(Activation(activation))
            model.add(Dropout(0.3))
        else:
            model.add(Dense(nodes))
            model.add(Activation(activation))
            model.add(Dropout(0.3))
    model.
 ↪add(Dense(units=1,kernel_initializer='glorot_uniform',activation='sigmoid'))
    model.
 ↪compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
    return model

# Wrap model using KerasClassifier
model=KerasClassifier(model=create_model,verbose=0)

# Hyperparameter grid
param_grid = {
    'model__layers': [[20], [40, 20], [45, 30, 15]],
    'model__activation': ['sigmoid', 'relu'],
    'batch_size': [128, 256],
    'epochs': [30]
```

```
}

# Grid search
grid=GridSearchCV(estimator=model,param_grid=param_grid,cv=5,verbose=1)
grid_result=grid.fit(X_train,y_train)

#Model best results
print(grid_result.best_score_,grid_result.best_params_)
y_pred=grid.predict(X_test)
y_pred=(y_pred>0.5)

from sklearn.metrics import confusion_matrix,accuracy_score
cm=confusion_matrix(y_pred,y_test)
score=accuracy_score(y_pred,y_test)

print(cm)
score
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits

/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
```

```
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
```

```
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

WARNING:tensorflow:5 out of the last 21 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x7f8ae5beec20> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.

/home/rgukt/.local/lib/python3.10/site-
```

```
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

WARNING:tensorflow:5 out of the last 15 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x7f8ae5a125f0> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.

/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
```

```
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
```

```
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

0.85625 {'batch_size': 128, 'epochs': 30, 'model__activation': 'relu',
'model__layers': [45, 30, 15]}
[[1560  239]
 [  35  166]]
```

[25]: 0.863

[18]:
```python
print(model.get_params().keys())
```

```
dict_keys(['model', 'build_fn', 'warm_start', 'random_state', 'optimizer',
'loss', 'metrics', 'batch_size', 'validation_batch_size', 'verbose',
'callbacks', 'validation_split', 'shuffle', 'run_eagerly', 'epochs',
'class_weight'])
```

1.2 ANN Google colab see there

2 CNN

2.1 CNN-Cat or Dog

[1]:
```python
#Data Augmentation using python and keras
from tensorflow.keras.preprocessing.image import␣
 ↪ImageDataGenerator,array_to_img,img_to_array,load_img

datagen=ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')

img_path='cat_or_dog3.png' #this is PIL image    #dog image
img=load_img(img_path,target_size=(150,150)) #resize
x=img_to_array(img) #numpy array eith shape(150,150)
```

```
x=x.reshape((1,)+x.shape) #numpy array with shape(1,3,150,150)

#.flow() ->batches of randomly transformed images
i=0
for batch in datagen.flow(x,batch_size=1,
                          ␣
 ↪save_to_dir="preview",save_prefix="cat",save_format="png"):
    i+=1
    if i>20:
        break;

from keras.models import Sequential
from keras.layers import Conv2D,MaxPooling2D
from keras.layers import Activation,Dropout,Flatten,Dense

model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=( 150, 150,3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(32, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(64, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))


model.add(Flatten())  # this converts our 3D feature maps to 1D feature vectors
model.add(Dense(64))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(1))
model.add(Activation('sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer='rmsprop',
              metrics=['accuracy'])

batch_size = 16

# this is the augmentation configuration we will use for training
train_datagen = ImageDataGenerator(
        rescale=1./255,
        shear_range=0.2,
        zoom_range=0.2,
```

```
        horizontal_flip=True)

# this is the augmentation configuration we will use for testing:
# only rescaling
test_datagen = ImageDataGenerator(rescale=1./255)

# this is a generator that will read pictures found in
# subfolers of 'data/train', and indefinitely generate
# batches of augmented image data
train_generator = train_datagen.flow_from_directory(
        'data/train',  # this is the target directory
        target_size=(150, 150),  # all images will be resized to 150x150
        batch_size=batch_size,
        class_mode='binary')  # since we use binary_crossentropy loss, we need␣
 ↪binary labels

# this is a similar generator, for validation data
validation_generator = test_datagen.flow_from_directory(
        'data/validation',
        target_size=(150, 150),
        batch_size=batch_size,
        class_mode='binary')

#train the model
model.fit(
        train_generator,
        steps_per_epoch=2000 // batch_size,
        epochs=50,
        validation_data=validation_generator,
        validation_steps=800 // batch_size)
#model.save_weights('first_try.h5')  # always save your weights after training␣
 ↪or during training
model.save_weights('first_try.weights.h5')
model.save('CatsDog.h5')

prediction = model.predict(x)
if prediction[0] > 0.5:
    print("The image is classified as a Dog!")
else:
    print("The image is classified as a Cat!")
```

```
2024-12-19 13:30:23.258565: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
```

```
2024-12-19 13:30:23.291253: E
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8454] Unable to register
cuDNN factory: Attempting to register factory for plugin cuDNN when one has
already been registered
2024-12-19 13:30:23.300757: E
external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1452] Unable to
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
2024-12-19 13:30:23.324400: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2024-12-19 13:30:28.273564: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not
find TensorRT
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Found 20 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
Epoch 1/50

/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122:
UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in
its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be
ignored.
  self._warn_if_super_not_called()
2024-12-19 13:30:35.000856: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
44859392 exceeds 10% of free system memory.

  1/125            5:13 3s/step - accuracy:
0.4375 - loss: 0.6974

2024-12-19 13:30:35.328499: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
44859392 exceeds 10% of free system memory.

  2/125            28s 229ms/step - accuracy:
0.4187 - loss: 0.7188

2024-12-19 13:30:35.737651: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]
```

```
/usr/lib/python3.10/contextlib.py:153: UserWarning: Your input ran out of data;
interrupting training. Make sure that your dataset or generator can generate at
least `steps_per_epoch * epochs` batches. You may need to use the `.repeat()`
function when building your dataset.
  self.gen.throw(typ, value, traceback)

125/125            4s 8ms/step -
accuracy: 0.4003 - loss: 0.7398 - val_accuracy: 0.5500 - val_loss: 0.6780
Epoch 2/50

2024-12-19 13:30:36.332359: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
44859392 exceeds 10% of free system memory.
2024-12-19 13:30:36.490939: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

  1/125            2:32 1s/step - accuracy:
0.7500 - loss: 0.6692

2024-12-19 13:30:37.759999: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
44859392 exceeds 10% of free system memory.

  2/125            1:05 533ms/step -
accuracy: 0.7000 - loss: 0.6868

2024-12-19 13:30:38.124737: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
44859392 exceeds 10% of free system memory.

125/125            2s 7ms/step -
accuracy: 0.6508 - loss: 0.7042 - val_accuracy: 0.5000 - val_loss: 0.7068
Epoch 3/50

2024-12-19 13:30:38.562611: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

125/125            1s 4ms/step -
accuracy: 0.3502 - loss: 0.7215 - val_accuracy: 0.5500 - val_loss: 0.6697
Epoch 4/50
125/125            1s 6ms/step -
accuracy: 0.6508 - loss: 0.6966 - val_accuracy: 0.7500 - val_loss: 0.6694
Epoch 5/50

2024-12-19 13:30:40.813000: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

125/125            1s 3ms/step -
accuracy: 0.6503 - loss: 0.7185 - val_accuracy: 0.5500 - val_loss: 0.6661
Epoch 6/50
```

```
125/125              1s 5ms/step -
accuracy: 0.4464 - loss: 0.6969 - val_accuracy: 0.5000 - val_loss: 0.6809
Epoch 7/50
125/125              1s 6ms/step -
accuracy: 0.4980 - loss: 0.6960 - val_accuracy: 0.7000 - val_loss: 0.6824
Epoch 8/50
125/125              1s 7ms/step -
accuracy: 0.6488 - loss: 0.6804 - val_accuracy: 0.8500 - val_loss: 0.6678
Epoch 9/50

2024-12-19 13:30:45.231706: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

125/125              1s 6ms/step -
accuracy: 0.5000 - loss: 0.6983 - val_accuracy: 0.7000 - val_loss: 0.6708
Epoch 10/50
125/125              1s 7ms/step -
accuracy: 0.6012 - loss: 0.6742 - val_accuracy: 0.8500 - val_loss: 0.6459
Epoch 11/50
125/125              1s 7ms/step -
accuracy: 0.4504 - loss: 0.7119 - val_accuracy: 0.6500 - val_loss: 0.6691
Epoch 12/50
125/125              1s 7ms/step -
accuracy: 0.6528 - loss: 0.6597 - val_accuracy: 0.6500 - val_loss: 0.6470
Epoch 13/50
125/125              1s 3ms/step -
accuracy: 0.5506 - loss: 0.6706 - val_accuracy: 0.5000 - val_loss: 0.6611
Epoch 14/50
125/125              1s 3ms/step -
accuracy: 0.5005 - loss: 0.6588 - val_accuracy: 0.8000 - val_loss: 0.6604
Epoch 15/50
125/125              1s 4ms/step -
accuracy: 0.7500 - loss: 0.6604 - val_accuracy: 0.5000 - val_loss: 0.6703
Epoch 16/50
125/125              1s 7ms/step -
accuracy: 0.5992 - loss: 0.6710 - val_accuracy: 0.7500 - val_loss: 0.6241
Epoch 17/50

2024-12-19 13:30:55.314753: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

125/125              1s 3ms/step -
accuracy: 0.6994 - loss: 0.6225 - val_accuracy: 0.7000 - val_loss: 0.5874
Epoch 18/50
125/125              1s 6ms/step -
accuracy: 0.6012 - loss: 0.7010 - val_accuracy: 0.7000 - val_loss: 0.6251
Epoch 19/50
125/125              1s 3ms/step -
```

```
accuracy: 0.8001 - loss: 0.6058 - val_accuracy: 0.8500 - val_loss: 0.5730
Epoch 20/50
125/125            1s 3ms/step -
accuracy: 0.7510 - loss: 0.6325 - val_accuracy: 0.5500 - val_loss: 0.6767
Epoch 21/50
125/125            1s 6ms/step -
accuracy: 0.5992 - loss: 0.6119 - val_accuracy: 0.9000 - val_loss: 0.5908
Epoch 22/50
125/125            2s 7ms/step -
accuracy: 0.6508 - loss: 0.6304 - val_accuracy: 0.7500 - val_loss: 0.5971
Epoch 23/50
125/125            1s 7ms/step -
accuracy: 0.8492 - loss: 0.5509 - val_accuracy: 0.9000 - val_loss: 0.5543
Epoch 24/50
125/125            1s 6ms/step -
accuracy: 0.5536 - loss: 1.1752 - val_accuracy: 0.8500 - val_loss: 0.5763
Epoch 25/50
125/125            1s 4ms/step -
accuracy: 0.8502 - loss: 0.6003 - val_accuracy: 0.8000 - val_loss: 0.5491
Epoch 26/50
125/125            1s 4ms/step -
accuracy: 0.8507 - loss: 0.5385 - val_accuracy: 0.7500 - val_loss: 0.5219
Epoch 27/50
125/125            1s 3ms/step -
accuracy: 0.8497 - loss: 0.5329 - val_accuracy: 0.7000 - val_loss: 0.4744
Epoch 28/50
125/125            1s 6ms/step -
accuracy: 0.5536 - loss: 0.8898 - val_accuracy: 0.7000 - val_loss: 0.5794
Epoch 29/50
125/125            1s 5ms/step -
accuracy: 0.7460 - loss: 0.5582 - val_accuracy: 0.9500 - val_loss: 0.4661
Epoch 30/50
125/125            1s 4ms/step -
accuracy: 0.9499 - loss: 0.4175 - val_accuracy: 0.9500 - val_loss: 0.4118
Epoch 31/50
125/125            1s 6ms/step -
accuracy: 0.8016 - loss: 0.4840 - val_accuracy: 0.9500 - val_loss: 0.3767
Epoch 32/50
125/125            1s 5ms/step -
accuracy: 0.9008 - loss: 0.3850 - val_accuracy: 0.9500 - val_loss: 0.3156
Epoch 33/50

2024-12-19 13:31:13.919359: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

125/125            1s 3ms/step -
accuracy: 0.9003 - loss: 0.4536 - val_accuracy: 0.5000 - val_loss: 0.9032
Epoch 34/50
```

```
125/125              1s 6ms/step -
accuracy: 0.8988 - loss: 0.3565 - val_accuracy: 0.9500 - val_loss: 0.3446
Epoch 35/50
125/125              1s 3ms/step -
accuracy: 0.9003 - loss: 0.3415 - val_accuracy: 0.9000 - val_loss: 0.4103
Epoch 36/50
125/125              1s 5ms/step -
accuracy: 0.8492 - loss: 0.4315 - val_accuracy: 0.9500 - val_loss: 0.3105
Epoch 37/50
125/125              1s 3ms/step -
accuracy: 0.9003 - loss: 0.3880 - val_accuracy: 0.9500 - val_loss: 0.2689
Epoch 38/50
125/125              1s 6ms/step -
accuracy: 0.9504 - loss: 0.2173 - val_accuracy: 0.9500 - val_loss: 0.2312
Epoch 39/50
125/125              1s 4ms/step -
accuracy: 0.8998 - loss: 0.3294 - val_accuracy: 0.9500 - val_loss: 0.2141
Epoch 40/50
125/125              1s 4ms/step -
accuracy: 0.9499 - loss: 0.3117 - val_accuracy: 0.9500 - val_loss: 0.1696
Epoch 41/50
125/125              1s 4ms/step -
accuracy: 0.9499 - loss: 0.2324 - val_accuracy: 0.9500 - val_loss: 0.2154
Epoch 42/50
125/125              1s 7ms/step -
accuracy: 1.0000 - loss: 0.2006 - val_accuracy: 0.9500 - val_loss: 0.1186
Epoch 43/50
125/125              1s 4ms/step -
accuracy: 1.0000 - loss: 0.0968 - val_accuracy: 0.9500 - val_loss: 0.1322
Epoch 44/50
125/125              1s 6ms/step -
accuracy: 0.9504 - loss: 0.2326 - val_accuracy: 0.6500 - val_loss: 1.1530
Epoch 45/50
125/125              1s 6ms/step -
accuracy: 0.6032 - loss: 1.4166 - val_accuracy: 0.9500 - val_loss: 0.2064
Epoch 46/50
125/125              1s 6ms/step -
accuracy: 0.8988 - loss: 0.2370 - val_accuracy: 0.9500 - val_loss: 0.1355
Epoch 47/50
125/125              1s 3ms/step -
accuracy: 0.8998 - loss: 0.2463 - val_accuracy: 0.9500 - val_loss: 0.1348
Epoch 48/50
125/125              1s 7ms/step -
accuracy: 1.0000 - loss: 0.1831 - val_accuracy: 0.9500 - val_loss: 0.1111
Epoch 49/50
125/125              1s 6ms/step -
accuracy: 0.9504 - loss: 0.1305 - val_accuracy: 0.9500 - val_loss: 0.1286
Epoch 50/50
```

```
125/125              1s 6ms/step -
accuracy: 0.9504 - loss: 0.1114 - val_accuracy: 0.9500 - val_loss: 0.1477

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

1/1              0s 117ms/step
The image is classified as a Dog!
```

[4]:

```
#Data Augmentation using python and keras
from tensorflow.keras.preprocessing.image import␣
 ↪ImageDataGenerator,array_to_img,img_to_array,load_img

datagen=ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')

img_path='cat_or_dog4.png' #this is PIL image   #cat
img=load_img(img_path,target_size=(150,150)) #resize
x=img_to_array(img) #numpy array eith shape(3,150,150)
x=x.reshape((1,)+x.shape) #numpy array with shape(1,3,150,150)

#.flow() ->batches of randomly transformed images
i=0
for batch in datagen.flow(x,batch_size=1,
                          ␣
 ↪save_to_dir="preview",save_prefix="cat",save_format="png"):
    i+=1
    if i>20:
        break;

from keras.models import Sequential
from keras.layers import Conv2D,MaxPooling2D
from keras.layers import Activation,Dropout,Flatten,Dense

model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=( 150, 150,3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
model.add(Conv2D(32, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(64, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))


model.add(Flatten())  # this converts our 3D feature maps to 1D feature vectors
model.add(Dense(64))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(1))
model.add(Activation('sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer='rmsprop',
              metrics=['accuracy'])

batch_size = 16

# this is the augmentation configuration we will use for training
train_datagen = ImageDataGenerator(
        rescale=1./255,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True)

# this is the augmentation configuration we will use for testing:
# only rescaling
test_datagen = ImageDataGenerator(rescale=1./255)

# this is a generator that will read pictures found in
# subfolers of 'data/train', and indefinitely generate
# batches of augmented image data
train_generator = train_datagen.flow_from_directory(
        'data/train',  # this is the target directory
        target_size=(150, 150),  # all images will be resized to 150x150
        batch_size=batch_size,
        class_mode='binary')  # since we use binary_crossentropy loss, we need␣
 ↪binary labels

# this is a similar generator, for validation data
validation_generator = test_datagen.flow_from_directory(
        'data/validation',
        target_size=(150, 150),
```

```
        batch_size=batch_size,
        class_mode='binary')

#train the model
model.fit(
        train_generator,
        steps_per_epoch=2000 // batch_size,
        epochs=50,
        validation_data=validation_generator,
        validation_steps=800 // batch_size)
#model.save_weights('first_try.h5')  # always save your weights after training␣
 ↪or during training
model.save_weights('first_try.weights.h5')
model.save('CatsDog.h5')

prediction = model.predict(x)


# The model outputs a probability (since the final activation is sigmoid)
    # If prediction > 0.5, it is a dog (class 1), otherwise it's a cat (class 0)
if prediction[0] > 0.5:
    print("The image is classified as a Dog!")
else:
    print("The image is classified as a Cat!")
```

```
Found 20 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
Epoch 1/50
125/125              3s 8ms/step -
accuracy: 0.4008 - loss: 1.1426 - val_accuracy: 0.5000 - val_loss: 0.7501
Epoch 2/50
125/125              2s 3ms/step -
accuracy: 0.4995 - loss: 0.7435 - val_accuracy: 0.5000 - val_loss: 0.7002
Epoch 3/50
125/125              1s 4ms/step -
accuracy: 0.4995 - loss: 0.7351 - val_accuracy: 0.5000 - val_loss: 0.6794
Epoch 4/50
125/125              1s 3ms/step -
accuracy: 0.5501 - loss: 0.7038 - val_accuracy: 0.8000 - val_loss: 0.6656
Epoch 5/50
125/125              1s 5ms/step -
accuracy: 0.5040 - loss: 1.1887 - val_accuracy: 0.5000 - val_loss: 0.7008
Epoch 6/50
125/125              1s 5ms/step -
accuracy: 0.3512 - loss: 0.7764 - val_accuracy: 0.5000 - val_loss: 0.6792
```

```
Epoch 7/50
125/125              1s 3ms/step -
accuracy: 0.7004 - loss: 0.6685 - val_accuracy: 0.7000 - val_loss: 0.6617
Epoch 8/50
125/125              1s 3ms/step -
accuracy: 0.6999 - loss: 0.6661 - val_accuracy: 0.6500 - val_loss: 0.6355
Epoch 9/50
125/125              1s 3ms/step -
accuracy: 0.7004 - loss: 0.6065 - val_accuracy: 0.5000 - val_loss: 0.7329
Epoch 10/50
125/125              1s 6ms/step -
accuracy: 0.6984 - loss: 0.6758 - val_accuracy: 0.8500 - val_loss: 0.6006
Epoch 11/50
125/125              1s 6ms/step -
accuracy: 0.5516 - loss: 0.7322 - val_accuracy: 0.7000 - val_loss: 0.6378
Epoch 12/50
125/125              1s 3ms/step -
accuracy: 0.6002 - loss: 0.6432 - val_accuracy: 0.6000 - val_loss: 0.6315
Epoch 13/50
125/125              1s 4ms/step -
accuracy: 0.6498 - loss: 0.6382 - val_accuracy: 0.9000 - val_loss: 0.5442
Epoch 14/50
125/125              1s 4ms/step -
accuracy: 0.6002 - loss: 0.6437 - val_accuracy: 0.5000 - val_loss: 0.6796
Epoch 15/50
125/125              1s 5ms/step -
accuracy: 0.5992 - loss: 0.6253 - val_accuracy: 0.7000 - val_loss: 0.5783
Epoch 16/50
125/125              1s 3ms/step -
accuracy: 0.6498 - loss: 0.6148 - val_accuracy: 0.7000 - val_loss: 0.5370
Epoch 17/50
125/125              1s 4ms/step -
accuracy: 0.6493 - loss: 0.6056 - val_accuracy: 0.8000 - val_loss: 0.4784
Epoch 18/50
125/125              1s 5ms/step -
accuracy: 0.6488 - loss: 0.5062 - val_accuracy: 0.9000 - val_loss: 0.4211
Epoch 19/50
125/125              1s 6ms/step -
accuracy: 0.6012 - loss: 0.6997 - val_accuracy: 0.8500 - val_loss: 0.4912
Epoch 20/50
125/125              1s 3ms/step -
accuracy: 0.6012 - loss: 0.5929 - val_accuracy: 0.9000 - val_loss: 0.5759
Epoch 21/50
125/125              1s 3ms/step -
accuracy: 0.6999 - loss: 0.5807 - val_accuracy: 0.9000 - val_loss: 0.4731
Epoch 22/50
125/125              1s 7ms/step -
accuracy: 0.8016 - loss: 0.5276 - val_accuracy: 0.9500 - val_loss: 0.3827
```

```
Epoch 23/50
125/125            1s 4ms/step -
accuracy: 0.8497 - loss: 0.4167 - val_accuracy: 0.8500 - val_loss: 0.3662
Epoch 24/50
125/125            1s 6ms/step -
accuracy: 0.7480 - loss: 0.4579 - val_accuracy: 0.8000 - val_loss: 0.3186
Epoch 25/50
125/125            1s 3ms/step -
accuracy: 0.7009 - loss: 0.6256 - val_accuracy: 0.7000 - val_loss: 0.4947
Epoch 26/50
125/125            1s 5ms/step -
accuracy: 0.8492 - loss: 0.5305 - val_accuracy: 1.0000 - val_loss: 0.3981
Epoch 27/50
125/125            1s 3ms/step -
accuracy: 0.8001 - loss: 0.4554 - val_accuracy: 0.9000 - val_loss: 0.3709
Epoch 28/50
125/125            1s 3ms/step -
accuracy: 0.9499 - loss: 0.4308 - val_accuracy: 0.9500 - val_loss: 0.3203
Epoch 29/50

2024-12-19 13:33:54.149686: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

125/125            1s 6ms/step -
accuracy: 0.7996 - loss: 0.3936 - val_accuracy: 0.8500 - val_loss: 0.2975
Epoch 30/50
125/125            1s 3ms/step -
accuracy: 0.8497 - loss: 0.3211 - val_accuracy: 0.9000 - val_loss: 0.2530
Epoch 31/50
125/125            1s 6ms/step -
accuracy: 0.9504 - loss: 0.3231 - val_accuracy: 0.8500 - val_loss: 0.2452
Epoch 32/50
125/125            1s 5ms/step -
accuracy: 0.7976 - loss: 0.4646 - val_accuracy: 0.9500 - val_loss: 0.2383
Epoch 33/50
125/125            1s 5ms/step -
accuracy: 0.7996 - loss: 0.3477 - val_accuracy: 0.9000 - val_loss: 0.2031
Epoch 34/50
125/125            1s 5ms/step -
accuracy: 0.8492 - loss: 0.2673 - val_accuracy: 0.9000 - val_loss: 0.2271
Epoch 35/50
125/125            1s 3ms/step -
accuracy: 0.9499 - loss: 0.1893 - val_accuracy: 0.9000 - val_loss: 0.1571
Epoch 36/50
125/125            1s 3ms/step -
accuracy: 0.9499 - loss: 0.1866 - val_accuracy: 0.9500 - val_loss: 0.1160
Epoch 37/50
125/125            2s 4ms/step -
```

```
accuracy: 0.8497 - loss: 0.2223 - val_accuracy: 0.9500 - val_loss: 0.1184
Epoch 38/50
125/125             1s 3ms/step -
accuracy: 0.9008 - loss: 0.2026 - val_accuracy: 0.8000 - val_loss: 0.3827
Epoch 39/50
125/125             1s 4ms/step -
accuracy: 0.8001 - loss: 0.4149 - val_accuracy: 1.0000 - val_loss: 0.1866
Epoch 40/50
125/125             1s 3ms/step -
accuracy: 1.0000 - loss: 0.2187 - val_accuracy: 1.0000 - val_loss: 0.1124
Epoch 41/50
125/125             1s 3ms/step -
accuracy: 0.7996 - loss: 0.3058 - val_accuracy: 1.0000 - val_loss: 0.0932
Epoch 42/50
125/125             1s 3ms/step -
accuracy: 0.9499 - loss: 0.1500 - val_accuracy: 1.0000 - val_loss: 0.0679
Epoch 43/50
125/125             1s 4ms/step -
accuracy: 0.9499 - loss: 0.1715 - val_accuracy: 0.9500 - val_loss: 0.1289
Epoch 44/50
125/125             1s 3ms/step -
accuracy: 0.9003 - loss: 0.2044 - val_accuracy: 0.6000 - val_loss: 0.6830
Epoch 45/50
125/125             1s 5ms/step -
accuracy: 0.8472 - loss: 0.4539 - val_accuracy: 1.0000 - val_loss: 0.0963
Epoch 46/50
125/125             1s 4ms/step -
accuracy: 0.9499 - loss: 0.2034 - val_accuracy: 1.0000 - val_loss: 0.0835
Epoch 47/50
125/125             1s 5ms/step -
accuracy: 0.9504 - loss: 0.1146 - val_accuracy: 1.0000 - val_loss: 0.0557
Epoch 48/50
125/125             1s 4ms/step -
accuracy: 1.0000 - loss: 0.1311 - val_accuracy: 0.9500 - val_loss: 0.0815
Epoch 49/50
125/125             1s 6ms/step -
accuracy: 0.7480 - loss: 0.4253 - val_accuracy: 1.0000 - val_loss: 0.1164
Epoch 50/50
125/125             1s 6ms/step -
accuracy: 1.0000 - loss: 0.1712 - val_accuracy: 1.0000 - val_loss: 0.0649
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

```
1/1             0s 161ms/step
The image is classified as a Cat!
```

2.2 CNN-optimize using Keras Tuner ct=2hrs

[32]:
```
import tensorflow as tf
from tensorflow import keras
import numpy as np

print(tf.__version__)

fashion_mnist=keras.datasets.fashion_mnist

(train_images,train_labels),(test_images,test_labels)=fashion_mnist.load_data()

train_images=train_images/255.0
test_images=test_images/255.0

print("train images shape:",train_images[0].shape)

train_images=train_images.reshape(len(train_images),28,28,1)
test_images=test_images.reshape(len(test_images),28,28,1)

def build_model(hp):
    model=keras.Sequential([
        keras.layers.Conv2D(
            filters=hp.Int('conv_1_filter',min_value=64,max_value=128,step=16),
            kernel_size=hp.Choice('conv_1_kernel',values=[3,5]),
            activation='relu',
            input_shape=(28,28,1)
        ),
        keras.layers.Conv2D(
            filters=hp.Int('conv_2_filter',min_value=32,max_value=64,step=16),
            kernel_size=hp.Choice('conv_2_kernel',values=[3,5]),
            activation='relu',
        ),
        keras.layers.Flatten(),
        keras.layers.Dense(
            units=hp.Int('dense_1_units',min_value=32,max_value=128,step=16),
            activation='relu'
        ),
        keras.layers.Dense(10,activation='softmax')
    ])

    model.compile(optimizer=keras.optimizers.Adam(hp.
 ↪Choice('learning_rate',values=[1e-2,1e-3])),
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model
```

```python
from kerastuner import RandomSearch
from kerastuner.engine.hyperparameters import HyperParameters

tuner_search=RandomSearch(build_model,objective='val_accuracy',max_trials=5,directory='output'
 ↪Fashion")

tuner_search.search(train_images,train_labels,epochs=3,validation_split=0.1)

model=tuner_search.get_best_models(num_models=1)[0]

print("Model summary:")
print(model.summary())

tuner_search.search(train_images,train_labels,epochs=3,validation_split=0.1)
```

```
Trial 5 Complete [00h 11m 08s]
val_accuracy: 0.9114999771118164

Best val_accuracy So Far: 0.9120000004768372
Total elapsed time: 01h 16m 27s
Model summary:
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/saving/saving_lib.py:713: UserWarning: Skipping variable
loading for optimizer 'adam', because it has 2 variables whereas the saved
optimizer has 18 variables.
  saveable.load_own_variables(weights_store.get(inner_path))
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 80)	800
conv2d_1 (Conv2D)	(None, 24, 24, 32)	23,072
flatten (Flatten)	(None, 18432)	0
dense (Dense)	(None, 96)	1,769,568
dense_1 (Dense)	(None, 10)	970

Total params: 1,794,410 (6.85 MB)

Trainable params: 1,794,410 (6.85 MB)

```
Non-trainable params: 0 (0.00 B)
```

```
None
```

2.3 CNN -Face recongnition (VGG16)

2.3.1 Image collection – no need to learn

[5]:
```
from PIL import Image
import os
import random

# Function to generate random images
def generate_images(output_dir, num_images=100, image_size=(224, 224)):
    os.makedirs(output_dir, exist_ok=True)
    for i in range(num_images):
        img = Image.new('RGB', image_size, (random.randint(0, 255), random.
 ↪randint(0, 255), random.randint(0, 255)))
        img.save(os.path.join(output_dir, f"img_{i}.jpg"))

# Generate Training Dataset
generate_images("Datasets/Train/Class1", num_images=50)
generate_images("Datasets/Train/Class2", num_images=50)

# Generate Testing Dataset
generate_images("Datasets/Test/Class1", num_images=10)
generate_images("Datasets/Test/Class2", num_images=10)

print("Synthetic dataset generated successfully!")
```

```
Synthetic dataset generated successfully!
```

[15]:
```
from keras.layers import Input,Lambda,Dense,Flatten
from keras.models import Model,Sequential
from keras.applications.vgg16 import VGG16,preprocess_input
from keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
from glob import glob
import matplotlib.pyplot as plt

#resize all images
image_size=[224,224]

train_path="Datasets/Train"
valid_path="Datasets/Test"
```

```
#add preprocessing layer to front of VGG
vgg=VGG16(input_shape=image_size +[3],weights="imagenet",include_top=False) #3␣
 ↪for rgb ,1 for grey

#dont  train existed weights
for layer in vgg.layers:
    layer.trainable=False

#useful for no.of classes
folders=glob("Datasets/Train/*")

#our layers -add more if u want
x=Flatten()(vgg.output)
#x-dense 1000 activation-relu
prediction=Dense(len(folders),activation='softmax')(x)

#model object
model=Model(inputs=vgg.input,outputs=prediction)

print("Model summary")
print(model.summary())

#cost & optimization
model.
 ↪compile(loss="categorical_crossentropy",optimizer="adam",metrics=['accuracy'])

from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen=ImageDataGenerator(rescale=1./255,
                                 shear_range=0.2,
                                 zoom_range=0.2,
                                 horizontal_flip=True)

test_datagen=ImageDataGenerator(rescale=1./255)

training_set=train_datagen.flow_from_directory('Datasets/Train',
                                               target_size=(224,224),
                                               batch_size=32,
                                               class_mode='categorical')

test_set=test_datagen.flow_from_directory('Datasets/Test',
                                               target_size=(224,224),
                                               batch_size=32,
                                               class_mode='categorical')

# r=model.fit_generator(training_set,
```

```
#                        samples_per_epoch=8000,
#                        epochs=5,
#                        validation_data=test_set,
#                        nb_val_samples=2000)

#fit the model
r = model.fit(
    training_set,
    validation_data=test_set,
    epochs=5,
    steps_per_epoch=len(training_set),
    validation_steps=len(test_set)
)


#loss
plt.plot(r.history['loss'],label='train_loss')
plt.plot(r.history['val_loss'],label='val_loss')
plt.legend()
plt.show()
plt.savefig('AccVal_acc')


import tensorflow as tf
from keras.models import load_model

model.save('facefeatures_new_model.h5')
```

Model summary

Model: "functional_6"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584

block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten_3 (Flatten)	(None, 25088)	0
dense_6 (Dense)	(None, 2)	50,178

Total params: 14,764,866 (56.32 MB)

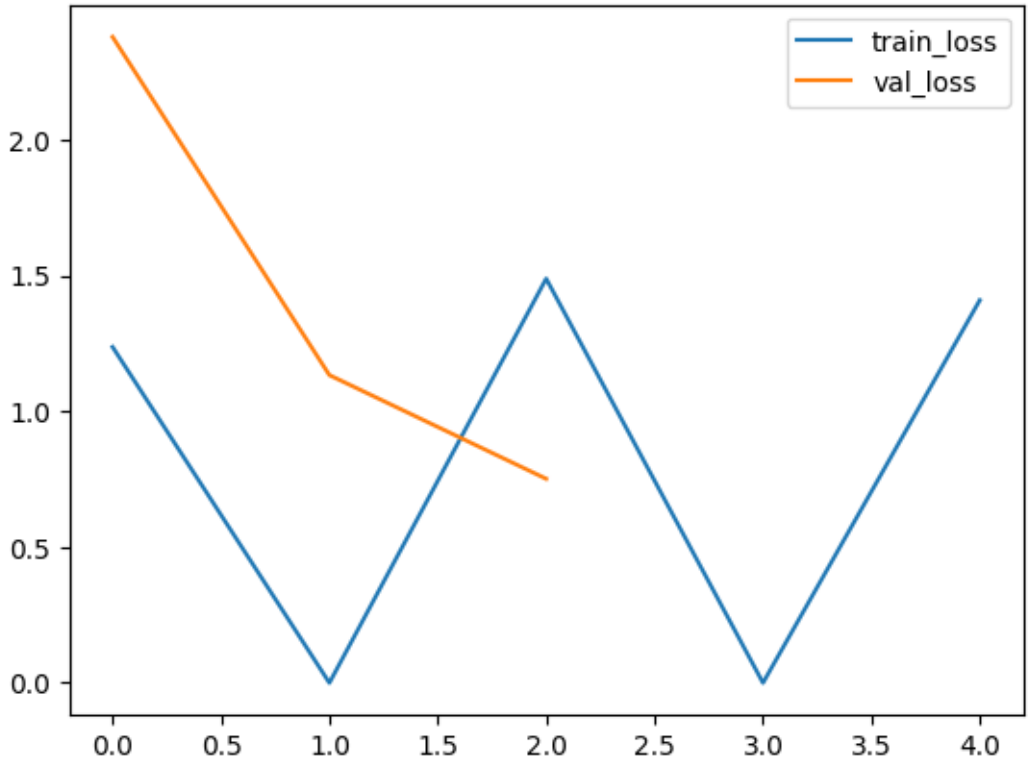
Trainable params: 50,178 (196.01 KB)

Non-trainable params: 14,714,688 (56.13 MB)

```
None
Found 100 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
Epoch 1/5
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122:
UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in
```

```
its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be
ignored.
  self._warn_if_super_not_called()
2024-12-14 18:17:29.878551: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
411041792 exceeds 10% of free system memory.
2024-12-14 18:17:30.475192: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
411041792 exceeds 10% of free system memory.
2024-12-14 18:17:33.103695: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
102760448 exceeds 10% of free system memory.
2024-12-14 18:17:33.311218: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
205520896 exceeds 10% of free system memory.
2024-12-14 18:17:34.524159: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
205520896 exceeds 10% of free system memory.

4/4             67s 15s/step -
accuracy: 0.5101 - loss: 1.1845 - val_accuracy: 0.5000 - val_loss: 2.3793
Epoch 2/5
4/4             0s 49ms/step -
accuracy: 0.0000e+00 - loss: 0.0000e+00
Epoch 3/5

2024-12-14 18:18:33.451222: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]
/usr/lib/python3.10/contextlib.py:153: UserWarning: Your input ran out of data;
interrupting training. Make sure that your dataset or generator can generate at
least `steps_per_epoch * epochs` batches. You may need to use the `.repeat()`
function when building your dataset.
  self.gen.throw(typ, value, traceback)
2024-12-14 18:18:33.600095: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

4/4             62s 15s/step -
accuracy: 0.5571 - loss: 1.6666 - val_accuracy: 0.5000 - val_loss: 1.1336
Epoch 4/5
4/4             0s 5ms/step -
accuracy: 0.0000e+00 - loss: 0.0000e+00
Epoch 5/5

2024-12-14 18:19:35.725431: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]
```

```
4/4              63s 15s/step -
accuracy: 0.4881 - loss: 1.4125 - val_accuracy: 0.4500 - val_loss: 0.7516
```




```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

```
<Figure size 640x480 with 0 Axes>
```

2.4 CNN -Face recongnition (ResNet50)

[18]:
```
from keras.layers import Input,Lambda,Dense,Flatten
from keras.models import Model,Sequential
from keras.applications.resnet50 import ResNet50,preprocess_input
from keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
from glob import glob
import matplotlib.pyplot as plt

#resize all images
```

```
image_size=[224,224]

train_path="Datasets/Train"
valid_path="Datasets/Test"

#add preprocessing layer to front of VGG
resnet=ResNet50(input_shape=image_size␣
 ↪+[3],weights="imagenet",include_top=False) #3 for rgb ,1 for grey

#dont  train existed weights
for layer in resnet.layers:
    layer.trainable=False

#useful for no.of classes
folders=glob("Datasets/Train/*")

#our layers -add more if u want
x=Flatten()(resnet.output)
#x-dense 1000 activation-relu
prediction=Dense(len(folders),activation='softmax')(x)

#model object
model=Model(inputs=resnet.input,outputs=prediction)

print("Model summary")
print(model.summary())

#cost & optimization
model.
 ↪compile(loss="categorical_crossentropy",optimizer="adam",metrics=['accuracy'])

from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen=ImageDataGenerator(rescale=1./255,
                                 shear_range=0.2,
                                 zoom_range=0.2,
                                 horizontal_flip=True)

test_datagen=ImageDataGenerator(rescale=1./255)

training_set=train_datagen.flow_from_directory('Datasets/Train',
                                               target_size=(224,224),
                                               batch_size=32,
                                               class_mode='categorical')

test_set=test_datagen.flow_from_directory('Datasets/Test',
                                          target_size=(224,224),
```

```
                                                    batch_size=32,
                                                    class_mode='categorical')

# r=model.fit_generator(training_set,
#                        samples_per_epoch=8000,
#                        epochs=5,
#                        validation_data=test_set,
#                        nb_val_samples=2000)

#fit the model
r = model.fit(
    training_set,
    validation_data=test_set,
    epochs=5,
    steps_per_epoch=len(training_set),
    validation_steps=len(test_set)
)


#loss
plt.plot(r.history['loss'],label='train_loss')
plt.plot(r.history['val_loss'],label='val_loss')
plt.legend()
plt.show()
plt.savefig('AccVal_acc')


import tensorflow as tf
from keras.models import load_model

model.save('facefeatures_new_model_resnet50.h5')
```

Model summary

Model: "functional_8"

Layer (type)	Output Shape	Param #	Connected to
input_layer_6 (InputLayer)	(None, 224, 224, 3)	0	-
conv1_pad (ZeroPadding2D)	(None, 230, 230, 3)	0	input_layer_6[0]…
conv1_conv (Conv2D)	(None, 112, 112, 64)	9,472	conv1_pad[0][0]

```
conv1_bn                (None, 112, 112,          256   conv1_conv[0][0]
(BatchNormalizatio…     64)

conv1_relu              (None, 112, 112,            0   conv1_bn[0][0]
(Activation)            64)

pool1_pad               (None, 114, 114,            0   conv1_relu[0][0]
(ZeroPadding2D)         64)

pool1_pool              (None, 56, 56,              0   pool1_pad[0][0]
(MaxPooling2D)          64)

conv2_block1_1_conv     (None, 56, 56,          4,160   pool1_pool[0][0]
(Conv2D)                64)

conv2_block1_1_bn       (None, 56, 56,            256   conv2_block1_1_c…
(BatchNormalizatio…     64)

conv2_block1_1_relu     (None, 56, 56,              0   conv2_block1_1_b…
(Activation)            64)

conv2_block1_2_conv     (None, 56, 56,         36,928   conv2_block1_1_r…
(Conv2D)                64)

conv2_block1_2_bn       (None, 56, 56,            256   conv2_block1_2_c…
(BatchNormalizatio…     64)

conv2_block1_2_relu     (None, 56, 56,              0   conv2_block1_2_b…
(Activation)            64)

conv2_block1_0_conv     (None, 56, 56,         16,640   pool1_pool[0][0]
(Conv2D)                256)

conv2_block1_3_conv     (None, 56, 56,         16,640   conv2_block1_2_r…
(Conv2D)                256)

conv2_block1_0_bn       (None, 56, 56,          1,024   conv2_block1_0_c…
(BatchNormalizatio…     256)

conv2_block1_3_bn       (None, 56, 56,          1,024   conv2_block1_3_c…
(BatchNormalizatio…     256)

conv2_block1_add        (None, 56, 56,              0   conv2_block1_0_b…
(Add)                   256)                            conv2_block1_3_b…

conv2_block1_out        (None, 56, 56,              0   conv2_block1_add…
(Activation)            256)
```

conv2_block2_1_conv (Conv2D)	(None, 56, 56, 64)	16,448	conv2_block1_out…
conv2_block2_1_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block2_1_c…
conv2_block2_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_1_b…
conv2_block2_2_conv (Conv2D)	(None, 56, 56, 64)	36,928	conv2_block2_1_r…
conv2_block2_2_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block2_2_c…
conv2_block2_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_2_b…
conv2_block2_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block2_2_r…
conv2_block2_3_bn (BatchNormalizatio…	(None, 56, 56, 256)	1,024	conv2_block2_3_c…
conv2_block2_add (Add)	(None, 56, 56, 256)	0	conv2_block1_out… conv2_block2_3_b…
conv2_block2_out (Activation)	(None, 56, 56, 256)	0	conv2_block2_add…
conv2_block3_1_conv (Conv2D)	(None, 56, 56, 64)	16,448	conv2_block2_out…
conv2_block3_1_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block3_1_c…
conv2_block3_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block3_1_b…
conv2_block3_2_conv (Conv2D)	(None, 56, 56, 64)	36,928	conv2_block3_1_r…
conv2_block3_2_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block3_2_c…
conv2_block3_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block3_2_b…

conv2_block3_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block3_2_r…
conv2_block3_3_bn (BatchNormalizatio…	(None, 56, 56, 256)	1,024	conv2_block3_3_c…
conv2_block3_add (Add)	(None, 56, 56, 256)	0	conv2_block2_out… conv2_block3_3_b…
conv2_block3_out (Activation)	(None, 56, 56, 256)	0	conv2_block3_add…
conv3_block1_1_conv (Conv2D)	(None, 28, 28, 128)	32,896	conv2_block3_out…
conv3_block1_1_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block1_1_c…
conv3_block1_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block1_1_b…
conv3_block1_2_conv (Conv2D)	(None, 28, 28, 128)	147,584	conv3_block1_1_r…
conv3_block1_2_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block1_2_c…
conv3_block1_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block1_2_b…
conv3_block1_0_conv (Conv2D)	(None, 28, 28, 512)	131,584	conv2_block3_out…
conv3_block1_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block1_2_r…
conv3_block1_0_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block1_0_c…
conv3_block1_3_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block1_3_c…
conv3_block1_add (Add)	(None, 28, 28, 512)	0	conv3_block1_0_b… conv3_block1_3_b…
conv3_block1_out (Activation)	(None, 28, 28, 512)	0	conv3_block1_add…

conv3_block2_1_conv (Conv2D)	(None, 28, 28, 128)	65,664	conv3_block1_out…
conv3_block2_1_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block2_1_c…
conv3_block2_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block2_1_b…
conv3_block2_2_conv (Conv2D)	(None, 28, 28, 128)	147,584	conv3_block2_1_r…
conv3_block2_2_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block2_2_c…
conv3_block2_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block2_2_b…
conv3_block2_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block2_2_r…
conv3_block2_3_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block2_3_c…
conv3_block2_add (Add)	(None, 28, 28, 512)	0	conv3_block1_out… conv3_block2_3_b…
conv3_block2_out (Activation)	(None, 28, 28, 512)	0	conv3_block2_add…
conv3_block3_1_conv (Conv2D)	(None, 28, 28, 128)	65,664	conv3_block2_out…
conv3_block3_1_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block3_1_c…
conv3_block3_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block3_1_b…
conv3_block3_2_conv (Conv2D)	(None, 28, 28, 128)	147,584	conv3_block3_1_r…
conv3_block3_2_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block3_2_c…
conv3_block3_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block3_2_b…

conv3_block3_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block3_2_r…
conv3_block3_3_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block3_3_c…
conv3_block3_add (Add)	(None, 28, 28, 512)	0	conv3_block2_out… conv3_block3_3_b…
conv3_block3_out (Activation)	(None, 28, 28, 512)	0	conv3_block3_add…
conv3_block4_1_conv (Conv2D)	(None, 28, 28, 128)	65,664	conv3_block3_out…
conv3_block4_1_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block4_1_c…
conv3_block4_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block4_1_b…
conv3_block4_2_conv (Conv2D)	(None, 28, 28, 128)	147,584	conv3_block4_1_r…
conv3_block4_2_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block4_2_c…
conv3_block4_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block4_2_b…
conv3_block4_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block4_2_r…
conv3_block4_3_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block4_3_c…
conv3_block4_add (Add)	(None, 28, 28, 512)	0	conv3_block3_out… conv3_block4_3_b…
conv3_block4_out (Activation)	(None, 28, 28, 512)	0	conv3_block4_add…
conv4_block1_1_conv (Conv2D)	(None, 14, 14, 256)	131,328	conv3_block4_out…
conv4_block1_1_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block1_1_c…

conv4_block1_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block1_1_b…
conv4_block1_2_conv (Conv2D)	(None, 14, 14, 256)	590,080	conv4_block1_1_r…
conv4_block1_2_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block1_2_c…
conv4_block1_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block1_2_b…
conv4_block1_0_conv (Conv2D)	(None, 14, 14, 1024)	525,312	conv3_block4_out…
conv4_block1_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block1_2_r…
conv4_block1_0_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block1_0_c…
conv4_block1_3_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block1_3_c…
conv4_block1_add (Add)	(None, 14, 14, 1024)	0	conv4_block1_0_b… conv4_block1_3_b…
conv4_block1_out (Activation)	(None, 14, 14, 1024)	0	conv4_block1_add…
conv4_block2_1_conv (Conv2D)	(None, 14, 14, 256)	262,400	conv4_block1_out…
conv4_block2_1_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block2_1_c…
conv4_block2_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block2_1_b…
conv4_block2_2_conv (Conv2D)	(None, 14, 14, 256)	590,080	conv4_block2_1_r…
conv4_block2_2_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block2_2_c…
conv4_block2_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block2_2_b…

conv4_block2_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block2_2_r…
conv4_block2_3_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block2_3_c…
conv4_block2_add (Add)	(None, 14, 14, 1024)	0	conv4_block1_out… conv4_block2_3_b…
conv4_block2_out (Activation)	(None, 14, 14, 1024)	0	conv4_block2_add…
conv4_block3_1_conv (Conv2D)	(None, 14, 14, 256)	262,400	conv4_block2_out…
conv4_block3_1_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block3_1_c…
conv4_block3_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block3_1_b…
conv4_block3_2_conv (Conv2D)	(None, 14, 14, 256)	590,080	conv4_block3_1_r…
conv4_block3_2_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block3_2_c…
conv4_block3_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block3_2_b…
conv4_block3_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block3_2_r…
conv4_block3_3_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block3_3_c…
conv4_block3_add (Add)	(None, 14, 14, 1024)	0	conv4_block2_out… conv4_block3_3_b…
conv4_block3_out (Activation)	(None, 14, 14, 1024)	0	conv4_block3_add…
conv4_block4_1_conv (Conv2D)	(None, 14, 14, 256)	262,400	conv4_block3_out…
conv4_block4_1_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block4_1_c…

conv4_block4_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block4_1_b…
conv4_block4_2_conv (Conv2D)	(None, 14, 14, 256)	590,080	conv4_block4_1_r…
conv4_block4_2_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block4_2_c…
conv4_block4_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block4_2_b…
conv4_block4_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block4_2_r…
conv4_block4_3_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block4_3_c…
conv4_block4_add (Add)	(None, 14, 14, 1024)	0	conv4_block3_out… conv4_block4_3_b…
conv4_block4_out (Activation)	(None, 14, 14, 1024)	0	conv4_block4_add…
conv4_block5_1_conv (Conv2D)	(None, 14, 14, 256)	262,400	conv4_block4_out…
conv4_block5_1_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block5_1_c…
conv4_block5_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block5_1_b…
conv4_block5_2_conv (Conv2D)	(None, 14, 14, 256)	590,080	conv4_block5_1_r…
conv4_block5_2_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block5_2_c…
conv4_block5_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block5_2_b…
conv4_block5_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block5_2_r…
conv4_block5_3_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block5_3_c…

conv4_block5_add (Add)	(None, 14, 14, 1024)	0	conv4_block4_out… conv4_block5_3_b…
conv4_block5_out (Activation)	(None, 14, 14, 1024)	0	conv4_block5_add…
conv4_block6_1_conv (Conv2D)	(None, 14, 14, 256)	262,400	conv4_block5_out…
conv4_block6_1_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block6_1_c…
conv4_block6_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block6_1_b…
conv4_block6_2_conv (Conv2D)	(None, 14, 14, 256)	590,080	conv4_block6_1_r…
conv4_block6_2_bn (BatchNormalizatio…	(None, 14, 14, 256)	1,024	conv4_block6_2_c…
conv4_block6_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block6_2_b…
conv4_block6_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block6_2_r…
conv4_block6_3_bn (BatchNormalizatio…	(None, 14, 14, 1024)	4,096	conv4_block6_3_c…
conv4_block6_add (Add)	(None, 14, 14, 1024)	0	conv4_block5_out… conv4_block6_3_b…
conv4_block6_out (Activation)	(None, 14, 14, 1024)	0	conv4_block6_add…
conv5_block1_1_conv (Conv2D)	(None, 7, 7, 512)	524,800	conv4_block6_out…
conv5_block1_1_bn (BatchNormalizatio…	(None, 7, 7, 512)	2,048	conv5_block1_1_c…
conv5_block1_1_relu (Activation)	(None, 7, 7, 512)	0	conv5_block1_1_b…
conv5_block1_2_conv (Conv2D)	(None, 7, 7, 512)	2,359,808	conv5_block1_1_r…

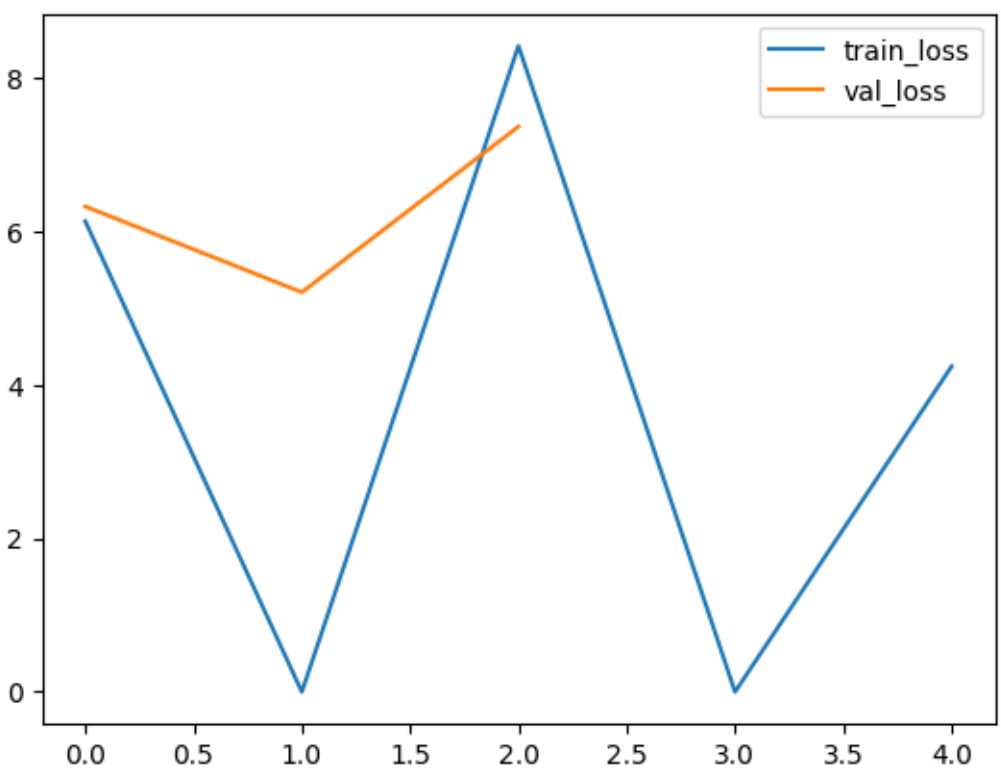
```
conv5_block1_2_bn     (None, 7, 7, 512)        2,048   conv5_block1_2_c…
(BatchNormalizatio…

conv5_block1_2_relu   (None, 7, 7, 512)            0   conv5_block1_2_b…
(Activation)

conv5_block1_0_conv   (None, 7, 7,         2,099,200   conv4_block6_out…
(Conv2D)              2048)

conv5_block1_3_conv   (None, 7, 7,         1,050,624   conv5_block1_2_r…
(Conv2D)              2048)

conv5_block1_0_bn     (None, 7, 7,             8,192   conv5_block1_0_c…
(BatchNormalizatio…   2048)

conv5_block1_3_bn     (None, 7, 7,             8,192   conv5_block1_3_c…
(BatchNormalizatio…   2048)

conv5_block1_add      (None, 7, 7,                 0   conv5_block1_0_b…
(Add)                 2048)                            conv5_block1_3_b…

conv5_block1_out      (None, 7, 7,                 0   conv5_block1_add…
(Activation)          2048)

conv5_block2_1_conv   (None, 7, 7, 512)    1,049,088   conv5_block1_out…
(Conv2D)

conv5_block2_1_bn     (None, 7, 7, 512)        2,048   conv5_block2_1_c…
(BatchNormalizatio…

conv5_block2_1_relu   (None, 7, 7, 512)            0   conv5_block2_1_b…
(Activation)

conv5_block2_2_conv   (None, 7, 7, 512)    2,359,808   conv5_block2_1_r…
(Conv2D)

conv5_block2_2_bn     (None, 7, 7, 512)        2,048   conv5_block2_2_c…
(BatchNormalizatio…

conv5_block2_2_relu   (None, 7, 7, 512)            0   conv5_block2_2_b…
(Activation)

conv5_block2_3_conv   (None, 7, 7,         1,050,624   conv5_block2_2_r…
(Conv2D)              2048)

conv5_block2_3_bn     (None, 7, 7,             8,192   conv5_block2_3_c…
(BatchNormalizatio…   2048)
```

conv5_block2_add (Add)	(None, 7, 7, 2048)	0	conv5_block1_out… conv5_block2_3_b…
conv5_block2_out (Activation)	(None, 7, 7, 2048)	0	conv5_block2_add…
conv5_block3_1_conv (Conv2D)	(None, 7, 7, 512)	1,049,088	conv5_block2_out…
conv5_block3_1_bn (BatchNormalizatio…	(None, 7, 7, 512)	2,048	conv5_block3_1_c…
conv5_block3_1_relu (Activation)	(None, 7, 7, 512)	0	conv5_block3_1_b…
conv5_block3_2_conv (Conv2D)	(None, 7, 7, 512)	2,359,808	conv5_block3_1_r…
conv5_block3_2_bn (BatchNormalizatio…	(None, 7, 7, 512)	2,048	conv5_block3_2_c…
conv5_block3_2_relu (Activation)	(None, 7, 7, 512)	0	conv5_block3_2_b…
conv5_block3_3_conv (Conv2D)	(None, 7, 7, 2048)	1,050,624	conv5_block3_2_r…
conv5_block3_3_bn (BatchNormalizatio…	(None, 7, 7, 2048)	8,192	conv5_block3_3_c…
conv5_block3_add (Add)	(None, 7, 7, 2048)	0	conv5_block2_out… conv5_block3_3_b…
conv5_block3_out (Activation)	(None, 7, 7, 2048)	0	conv5_block3_add…
flatten_5 (Flatten)	(None, 100352)	0	conv5_block3_out…
dense_8 (Dense)	(None, 2)	200,706	flatten_5[0][0]

Total params: 23,788,418 (90.75 MB)

Trainable params: 200,706 (784.01 KB)

```
 Non-trainable params: 23,587,712 (89.98 MB)


None
Found 100 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
Epoch 1/5
4/4              35s 8s/step -
accuracy: 0.4361 - loss: 6.0319 - val_accuracy: 0.5000 - val_loss: 6.3233
Epoch 2/5
4/4              0s 56ms/step -
accuracy: 0.0000e+00 - loss: 0.0000e+00
Epoch 3/5
4/4              39s 5s/step -
accuracy: 0.5522 - loss: 7.1537 - val_accuracy: 0.5000 - val_loss: 5.2061
Epoch 4/5
4/4              0s 16ms/step -
accuracy: 0.0000e+00 - loss: 0.0000e+00
Epoch 5/5

2024-12-14 18:31:46.101096: I tensorflow/core/framework/local_rendezvous.cc:404]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
         [[{{node IteratorGetNext}}]]

4/4              41s 5s/step -
accuracy: 0.4270 - loss: 4.5768 - val_accuracy: 0.5000 - val_loss: 7.3644
```



```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

```
<Figure size 640x480 with 0 Axes>
```

3 RNN

3.1 LSTM - Word Embedding using Embedded layers in layers

[14]:
```
from tensorflow.keras.preprocessing.text import one_hot

#sentences
sent=['the glass of milk',
      'the glass of juice',
      'the cup of tea',
      'I am a good boy',
      'I am a good developer',
      'understand the meaning of words',
      'your videos are good']

#vocabulary size
voc_size=10000

#one hot representation
onehot_repr=[one_hot(words,voc_size)for words in sent]
print("One hot:",onehot_repr)

#word embedding representation
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
import numpy as np

sent_len=8
embed_docs=pad_sequences(onehot_repr,padding='pre',maxlen=sent_len)
print("Embedded docs:")
print(embed_docs)

dim=10
model=Sequential()
model.add(Embedding(voc_size,10,input_length=sent_len))
model.compile('adam','mse')
```

```
print("Model summary")
print(model.summary())

print("Model prediction of Embedded docs:")
print(model.predict(embed_docs))

print(embed_docs[0])
print(model.predict(embed_docs)[0])
```

```
One hot: [[540, 4444, 5162, 120], [540, 4444, 5162, 7688], [540, 8618, 5162,
8234], [4317, 5657, 2478, 5867, 5], [4317, 5657, 2478, 5867, 9510], [9536, 540,
7206, 5162, 3425], [7334, 6455, 7780, 5867]]
Embedded docs:
[[   0    0    0    0  540 4444 5162  120]
 [   0    0    0    0  540 4444 5162 7688]
 [   0    0    0    0  540 8618 5162 8234]
 [   0    0    0 4317 5657 2478 5867    5]
 [   0    0    0 4317 5657 2478 5867 9510]
 [   0    0    0 9536  540 7206 5162 3425]
 [   0    0    0    0 7334 6455 7780 5867]]
Model summary
```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
None
Model prediction of Embedded docs:
1/1              0s 307ms/step
[[[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
   -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
    1.96115039e-02 -3.14387567e-02]
  [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
   -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
```

```
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 3.01471837e-02 -1.33074149e-02  3.86753194e-02  4.01095636e-02
   1.54487155e-02  1.68926381e-02 -5.51640987e-03 -2.86046155e-02
   2.00294368e-02  2.71058194e-02]
 [ 5.82434982e-03 -4.79007363e-02  1.28389932e-02 -4.11167853e-02
   3.67891081e-02  1.82632320e-02  8.13936070e-03  4.02706750e-02
  -5.94891235e-03  7.84546137e-03]
 [ 1.66882761e-02  3.35900672e-02  4.07370217e-02  4.93160971e-02
  -2.02896353e-02 -3.56819518e-02 -2.44956017e-02  1.72162913e-02
  -3.76263037e-02 -9.69011709e-03]
 [-4.70095649e-02  3.53807323e-02  2.83009671e-02 -2.75694616e-02
  -1.59739368e-02  9.89709049e-03  5.31781837e-03  4.63197380e-03
  -1.21736415e-02 -6.89471886e-03]]

[[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 3.01471837e-02 -1.33074149e-02  3.86753194e-02  4.01095636e-02
   1.54487155e-02  1.68926381e-02 -5.51640987e-03 -2.86046155e-02
   2.00294368e-02  2.71058194e-02]
 [ 5.82434982e-03 -4.79007363e-02  1.28389932e-02 -4.11167853e-02
   3.67891081e-02  1.82632320e-02  8.13936070e-03  4.02706750e-02
  -5.94891235e-03  7.84546137e-03]
 [ 1.66882761e-02  3.35900672e-02  4.07370217e-02  4.93160971e-02
  -2.02896353e-02 -3.56819518e-02 -2.44956017e-02  1.72162913e-02
  -3.76263037e-02 -9.69011709e-03]
 [ 4.74037640e-02  3.50226872e-02  2.66630910e-02  2.88674496e-02
  -2.17971094e-02 -1.29960664e-02  3.08775567e-02  1.47091411e-02
   7.26909563e-03  1.89499967e-02]]

[[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
```

```
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 3.01471837e-02 -1.33074149e-02  3.86753194e-02  4.01095636e-02
   1.54487155e-02  1.68926381e-02 -5.51640987e-03 -2.86046155e-02
   2.00294368e-02  2.71058194e-02]
 [ 4.52830307e-02 -3.92415635e-02  3.76495160e-02 -3.48328128e-02
   1.16995573e-02  4.23094667e-02 -4.85279225e-02  2.93503292e-02
   1.41050704e-02  2.88456716e-02]
 [ 1.66882761e-02  3.35900672e-02  4.07370217e-02  4.93160971e-02
  -2.02896353e-02 -3.56819518e-02 -2.44956017e-02  1.72162913e-02
  -3.76263037e-02 -9.69011709e-03]
 [-3.38809490e-02  6.32993132e-03 -1.38755552e-02  2.04334408e-03
  -2.71243975e-03 -1.54561996e-02 -4.69651707e-02  4.86999191e-02
   7.37918541e-03 -4.49054018e-02]]

[[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
  -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
   1.96115039e-02 -3.14387567e-02]
 [ 2.58458592e-02  3.20011266e-02  1.12991408e-03 -3.87501344e-02
  -3.78918760e-02  4.23658527e-02  1.33154541e-03  2.68331431e-02
  -4.39875200e-03 -2.48774886e-05]
 [-2.25200895e-02  4.70875576e-03 -1.25342123e-02 -7.45487213e-03
   1.93961710e-03  4.08801921e-02  3.19060199e-02 -3.89952585e-03
  -2.21005678e-02 -4.10043485e-02]
 [ 4.22158130e-02 -5.36806509e-03  1.08857751e-02 -2.15619802e-03
  -2.79197581e-02 -6.16036728e-03 -9.69155878e-03  3.79027165e-02
   1.90159939e-02  3.44326757e-02]
 [-3.67483720e-02 -1.58362016e-02  4.23095562e-02 -2.21796632e-02
  -1.94715261e-02 -1.44128874e-03  4.64588515e-02 -3.15148234e-02
  -1.41386986e-02  3.03552411e-02]
 [ 8.39642435e-03 -2.55642086e-03 -1.98414326e-02  1.55228861e-02
   2.72683538e-02 -3.13546285e-02 -3.04762609e-02 -1.72704346e-02
   2.77050994e-02 -2.73955949e-02]]

[[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
```

```
 -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
  1.96115039e-02 -3.14387567e-02]
[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
 -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
  1.96115039e-02 -3.14387567e-02]
[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
 -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
  1.96115039e-02 -3.14387567e-02]
[ 2.58458592e-02  3.20011266e-02  1.12991408e-03 -3.87501344e-02
 -3.78918760e-02  4.23658527e-02  1.33154541e-03  2.68331431e-02
 -4.39875200e-03 -2.48774886e-05]
[-2.25200895e-02  4.70875576e-03 -1.25342123e-02 -7.45487213e-03
  1.93961710e-03  4.08801921e-02  3.19060199e-02 -3.89952585e-03
 -2.21005678e-02 -4.10043485e-02]
[ 4.22158130e-02 -5.36806509e-03  1.08857751e-02 -2.15619802e-03
 -2.79197581e-02 -6.16036728e-03 -9.69155878e-03  3.79027165e-02
  1.90159939e-02  3.44326757e-02]
[-3.67483720e-02 -1.58362016e-02  4.23095562e-02 -2.21796632e-02
 -1.94715261e-02 -1.44128874e-03  4.64588515e-02 -3.15148234e-02
 -1.41386986e-02  3.03552411e-02]
[ 2.75449194e-02  3.92622463e-02 -4.88924049e-02 -2.31242534e-02
  2.13169120e-02  3.40432189e-02  3.92330028e-02  1.85859911e-02
 -2.22419500e-02 -5.56439161e-03]]

[[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
 -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
  1.96115039e-02 -3.14387567e-02]
[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
 -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
  1.96115039e-02 -3.14387567e-02]
[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
 -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
  1.96115039e-02 -3.14387567e-02]
[ 3.39737646e-02 -1.89258456e-02 -3.68054397e-02  1.37986802e-02
  5.42365387e-03 -3.65566723e-02  4.15570624e-02  3.37009318e-02
 -3.25167999e-02  3.38438489e-02]
[ 3.01471837e-02 -1.33074149e-02  3.86753194e-02  4.01095636e-02
  1.54487155e-02  1.68926381e-02 -5.51640987e-03 -2.86046155e-02
  2.00294368e-02  2.71058194e-02]
[-2.86069401e-02  4.07935120e-02  3.09295319e-02  4.43474539e-02
  2.42772438e-02 -1.62014738e-02 -5.99372387e-03 -2.36364845e-02
  4.96536978e-02 -3.58667858e-02]
[ 1.66882761e-02  3.35900672e-02  4.07370217e-02  4.93160971e-02
 -2.02896353e-02 -3.56819518e-02 -2.44956017e-02  1.72162913e-02
 -3.76263037e-02 -9.69011709e-03]
[-1.04588866e-02  3.36807035e-02 -2.76969206e-02 -3.76005061e-02
  1.82903446e-02 -3.70061882e-02  1.95229538e-02 -2.66347285e-02
 -3.34459543e-02 -4.63164821e-02]]
```

```
 [[ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
   -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
    1.96115039e-02 -3.14387567e-02]
  [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
   -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
    1.96115039e-02 -3.14387567e-02]
  [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
   -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
    1.96115039e-02 -3.14387567e-02]
  [ 2.04642750e-02 -2.73093935e-02  4.19315957e-02 -1.27298757e-03
   -3.43557820e-02  4.50883619e-02 -4.47378270e-02  1.61950700e-02
    1.96115039e-02 -3.14387567e-02]
  [-2.38909852e-02 -3.57549675e-02 -1.96445119e-02  3.69338430e-02
    2.71166079e-02  1.77083872e-02  1.85069181e-02  4.69415262e-03
    4.24463637e-02 -1.54587142e-02]
  [ 1.45335309e-02  2.70994045e-02  8.96598026e-03 -1.25978477e-02
    2.10109241e-02  3.77000496e-03  3.09815146e-02  4.95004654e-03
    3.54132093e-02  2.56300084e-02]
  [ 5.21495193e-03 -4.67697158e-02  3.57426144e-02  3.36957835e-02
    3.24288942e-02  3.05952877e-03 -2.66081449e-02  3.52200381e-02
   -4.11527865e-02 -2.99052354e-02]
  [-3.67483720e-02 -1.58362016e-02  4.23095562e-02 -2.21796632e-02
   -1.94715261e-02 -1.44128874e-03  4.64588515e-02 -3.15148234e-02
   -1.41386986e-02  3.03552411e-02]]]
[   0    0    0    0  540 4444 5162  120]
1/1             0s 24ms/step
[[ 0.02046428 -0.02730939  0.0419316  -0.00127299 -0.03435578  0.04508836
  -0.04473783  0.01619507  0.0196115  -0.03143876]
 [ 0.02046428 -0.02730939  0.0419316  -0.00127299 -0.03435578  0.04508836
  -0.04473783  0.01619507  0.0196115  -0.03143876]
 [ 0.02046428 -0.02730939  0.0419316  -0.00127299 -0.03435578  0.04508836
  -0.04473783  0.01619507  0.0196115  -0.03143876]
 [ 0.02046428 -0.02730939  0.0419316  -0.00127299 -0.03435578  0.04508836
  -0.04473783  0.01619507  0.0196115  -0.03143876]
 [ 0.03014718 -0.01330741  0.03867532  0.04010956  0.01544872  0.01689264
  -0.00551641 -0.02860462  0.02002944  0.02710582]
 [ 0.00582435 -0.04790074  0.01283899 -0.04111679  0.03678911  0.01826323
   0.00813936  0.04027067 -0.00594891  0.00784546]
 [ 0.01668828  0.03359007  0.04073702  0.0493161  -0.02028964 -0.03568195
  -0.0244956   0.01721629 -0.0376263  -0.00969012]
 [-0.04700956  0.03538073  0.02830097 -0.02756946 -0.01597394  0.00989709
   0.00531782  0.00463197 -0.01217364 -0.00689472]]
```

3.2 Fake News Classifier Using LSTM

[25]:
```
import pandas as pd

df=pd.read_csv('train.csv')

print("Head")
df.head()

df=df.dropna()

#get Independent Features
X=df.drop('label',axis=1)

#get dependent Features
y=df['label']

print('X Shape:',X.shape)
print('y Shape:',y.shape)

import tensorflow as tf
from tensorflow.keras.layers import Embedding
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing.text import one_hot
from tensorflow.keras.layers import LSTM,Dense

voc_size=5000

#one hot representation
messages=X.copy()
messages.reset_index(inplace=True)

print("Actual messages")
print(messages['title'])

import nltk
import re
from nltk.corpus import stopwords

nltk.download('stopwords')

    #dataset preparation
from nltk.stem.porter import PorterStemmer
ps=PorterStemmer()
corpus=[]
```

```
print(len(messages),"of messgaes ,takes conversion")
for i in range(0,len(messages)):
    review=re.sub('[^a-zA-Z]',' ',messages['title'][i])
    review=review.lower()
    review=review.split()

    review=[ps.stem(word) for word in review if not word in stopwords.
 ↪words('english')]
    review=' '.join(review)
    corpus.append(review)

print("After converting :",len(corpus))
print(corpus[:5])

onehot_repr=[one_hot(words,voc_size)for words in corpus]
print("One hot represention:")
print(onehot_repr[:5])

#Embedding Representation
sent_len=20
embed_docs=pad_sequences(onehot_repr,padding='pre',maxlen=sent_len)
print("Embedded docs:")
print(embed_docs[:5])

    #create model
embed_vec_features=40
model=Sequential()
model.add(Embedding(voc_size,embed_vec_features,input_length=sent_len))
model.add(LSTM(100))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
print("Model summary:")
print(model.summary())

print("Embed docs:",len(embed_docs),"y shape:",y.shape)

import numpy as np
X_final=np.array(embed_docs)
y_final=np.array(y)

print("X final shape:",X_final.shape,"y final shape:",y_final.shape)

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X_final,y_final,test_size=0.
 ↪33,random_state=42)

#model training
```

```
model.
 ↪fit(X_train,y_train,validation_data=(X_test,y_test),epochs=10,batch_size=64)

#adding dropout
from tensorflow.keras.layers import Dropout
    #create model
embed_vec_features=40
model=Sequential()
model.add(Embedding(voc_size,embed_vec_features,input_length=sent_len))
model.add(Dropout(0.3))
model.add(LSTM(100))
model.add(Dropout(0.3))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])

#performance metrics
y_pred=model.predict(X_test)>0.5

from sklearn.metrics import confusion_matrix,accuracy_score

cm=confusion_matrix(y_test,y_pred)
acc=accuracy_score(y_test,y_pred)
print("Confusion matrix:",cm)
print("Accuarcy:",acc)
```

```
Head
X Shape: (18285, 4)
y Shape: (18285,)
Actual messages
0        House Dem Aide: We Didn't Even See Comey's Let…
1        FLYNN: Hillary Clinton, Big Woman on Campus - …
2                        Why the Truth Might Get You Fired
3        15 Civilians Killed In Single US Airstrike Hav…
4        Iranian woman jailed for fictional unpublished…
                                 …
18280    Rapper T.I.: Trump a 'Poster Child For White S…
18281    N.F.L. Playoffs: Schedule, Matchups and Odds -…
18282    Macy's Is Said to Receive Takeover Approach by…
18283    NATO, Russia To Hold Parallel Exercises In Bal…
18284                            What Keeps the F-35 Alive
Name: title, Length: 18285, dtype: object
18285 of messgaes ,takes conversion

[nltk_data] Downloading package stopwords to /home/rgukt/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!

After converting : 18285
['hous dem aid even see comey letter jason chaffetz tweet', 'flynn hillari
```

```
clinton big woman campu breitbart', 'truth might get fire', 'civilian kill singl
us airstrik identifi', 'iranian woman jail fiction unpublish stori woman stone
death adulteri']
One hot represention:
[[3553, 4564, 1280, 429, 2514, 3029, 3369, 553, 4876, 2764], [4571, 3396, 1220,
2441, 2422, 2530, 1338], [3708, 4372, 2868, 2618], [4569, 916, 1308, 1455, 2304,
4626], [873, 2422, 1098, 4558, 3517, 2747, 2422, 4244, 3525, 3097]]
Embedded docs:
[[   0    0    0    0    0    0    0    0    0    0 3553 4564 1280  429
  2514 3029 3369  553 4876 2764]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0 4571
  3396 1220 2441 2422 2530 1338]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0 3708 4372 2868 2618]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
  4569  916 1308 1455 2304 4626]
 [   0    0    0    0    0    0    0    0    0    0  873 2422 1098 4558
  3517 2747 2422 4244 3525 3097]]
Model summary:
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential_20"

Layer (type)	Output Shape	Param #
embedding_20 (Embedding)	?	0 (unbuilt)
lstm_10 (LSTM)	?	0 (unbuilt)
dense_10 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
None
Embed docs: 18285 y shape: (18285,)
X final shape: (18285, 20) y final shape: (18285,)
```

```
Epoch 1/10
192/192              7s 27ms/step -
accuracy: 0.7866 - loss: 0.4270 - val_accuracy: 0.9162 - val_loss: 0.1900
Epoch 2/10
192/192              5s 25ms/step -
accuracy: 0.9474 - loss: 0.1353 - val_accuracy: 0.9183 - val_loss: 0.1885
Epoch 3/10
192/192              5s 26ms/step -
accuracy: 0.9671 - loss: 0.0921 - val_accuracy: 0.9193 - val_loss: 0.2014
Epoch 4/10
192/192              5s 25ms/step -
accuracy: 0.9791 - loss: 0.0647 - val_accuracy: 0.9228 - val_loss: 0.2734
Epoch 5/10
192/192              5s 25ms/step -
accuracy: 0.9883 - loss: 0.0370 - val_accuracy: 0.9122 - val_loss: 0.2628
Epoch 6/10
192/192              5s 25ms/step -
accuracy: 0.9950 - loss: 0.0207 - val_accuracy: 0.9168 - val_loss: 0.3040
Epoch 7/10
192/192              5s 26ms/step -
accuracy: 0.9975 - loss: 0.0095 - val_accuracy: 0.9077 - val_loss: 0.3335
Epoch 8/10
192/192              5s 25ms/step -
accuracy: 0.9979 - loss: 0.0085 - val_accuracy: 0.9163 - val_loss: 0.4661
Epoch 9/10
192/192              5s 25ms/step -
accuracy: 0.9987 - loss: 0.0045 - val_accuracy: 0.9062 - val_loss: 0.5546
Epoch 10/10
192/192              5s 26ms/step -
accuracy: 0.9994 - loss: 0.0037 - val_accuracy: 0.9176 - val_loss: 0.4683
189/189              1s 6ms/step
Confusion matrix: [[2334 1085]
 [1681  935]]
Accuarcy: 0.5416735708367855
```

3.3 Stock Market Prediction & Forecasting using stacked LSTM

[71]:
```
#data collection

import requests
import pandas as pd

api_key = '62394b1cf578f2c0be0a41a79be297ee3ff9194a'
url = f'https://api.tiingo.com/tiingo/daily/AAPL/prices?
 ↪startDate={start_date}&endDate={end_date}&token={api_key}'

response = requests.get(url)
```

```python
if response.status_code == 200:
    print("Data fetched successfully")
    data = response.json()
    dataf = pd.DataFrame(data)
    dataf['date'] = pd.to_datetime(dataf['date'], errors='coerce')
    print(dataf.head())
else:
    print("Error:", response.status_code, response.text)

import pandas_datareader as pdr
# df=pdr.get_data_tiingo('AAPL',api_key=key)
df.to_csv('AAPL.csv')

df=pd.read_csv('AAPL.csv')
df1=df.reset_index()['close']

print('df1 shape:',df1.shape)

import matplotlib.pyplot as plt
print("DF1 plot:")
plt.plot(df1)


#LSTM -sensitive to scale of data so use min-max scaler
import numpy as  np
from sklearn.preprocessing import MinMaxScaler

scaler=MinMaxScaler(feature_range=(0,1))
df1=scaler.fit_transform(np.array(df1).reshape(-1,1))

print('df1 shape after min max:',df1.shape)

#split train ,test
training_size=int(len(df1)*0.65)
test_size=len(df1)-training_size
train_data,test_data=df1[0:training_size,:],df1[training_size:len(df1):1]

print("Train size:",training_size,"Test size:",test_size)

#convert array of values into dataset matrix
def create_dataset(dataset,time_step=1):
    dataX,dataY=[],[]
    for i in range(len(dataset)-time_step-1):
        a=dataset[i:(i+time_step),0]
        dataX.append(a)
        dataY.append(dataset[i+time_step,0])
    return np.array(dataX),np.array(dataY)
```

```
#reshape into X=t,t+1,t+2,t+3 and Y=t+4
time_step=10
X_train,y_train=create_dataset(train_data,time_step)
X_test,y_test=create_dataset(test_data,time_step)

print("X_train shape:",X_train.shape,"y_train shape:",y_train.shape)
print("X_test shape:",X_test.shape,"y_test shape:",y_test.shape)

#reshape input to be [samples,time steps,features] need for LSTM
X_train=X_train.reshape(X_train.shape[0],X_train.shape[1],1)
X_test=X_test.reshape(X_test.shape[0],X_test.shape[1],1)

#create LSTM model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,LSTM

model=Sequential()
model.add(LSTM(50,return_sequences=True,input_shape=(100,1)))
model.add(LSTM(50,return_sequences=True))
model.add(LSTM(50))
model.add(Dense(1))
model.compile(loss='mean_squared_error',optimizer='adam')

print("Model summary:")
print(model.summary())

model.
 ↪fit(X_train,y_train,validation_data=(X_test,y_test),epochs=100,batch_size=64,verbose=1)

#prediction and check performance metrics
train_predict=model.predict(X_train)
test_predict=model.predict(X_test)

#transback to original form
train_predict=scaler.inverse_transform(train_predict)
test_predict=scaler.inverse_transform(test_predict)

#calculate RMSE performance metrics
import math
from sklearn.metrics import mean_squared_error
math.sqrt(mean_squared_error(y_train,train_predict))

#test data RMSE
math.sqrt(mean_squared_error(y_test,test_predict))

#plotting
```

```
#shift train predictions for plotting
look_back=10
trainPredictPlot=np.empty_like(df1)
trainPredictPlot[:,:]=np.nan
trainPredictPlot[look_back:len(train_predict)+look_back,:]=train_predict
#shift test predictions for plotting
testPredictPlot=np.empty_like(df1)
testPredictPlot[:,:]=np.nan
#testPredictPlot[len(train_predict)+(look_back*2)+1:len(df1)-1:]=test_predict
testPredictPlot[len(train_predict) + (look_back * 2) + 1:len(train_predict) +␣
↪(look_back * 2) + 1 + len(test_predict), :] = test_predict
#plot base line & predictions
plt.plot(scaler.inverse_transform(df1))
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()

print("Test data len:",len(test_data)) #441
x_input=test_data[31:].reshape(1,-1)

temp_input=list(x_input)
temp_input=temp_input[0].tolist()

#demonstrate prediction for next 10 days
lst_output=[]
n_steps=10
i=0
while(i<30):
    if len(temp_input)>10:
        #print(temp_input)
        x_input=np.array(temp_input[1:])
        print("{} day input {}".format(i,x_input))
        x_input=x_input.reshape(1,-1)
        x_input=x_input.reshape((1,n_steps,1))
        #print(x_input)
        yhat=model.predict(x_input,verbose=0)
        print("{} day output {}".format(i,yhat))
        temp_input.extend(yhat[0].tolist())
        temp_input=temp_input[1:]
        #print(temp_input)
        lst_output.extend(yhat.tolist())
        i=i+1
    else:
        x_input=x_input.reshape((1,n_steps,1))
        yhat=model.predict(x_input,verbose=0)
        print(yhat[0])
        temp_input.extend(yhat[0].tolist())
```

```
        print(len(temp_input))
        lst_output.extend(yhat.tolist())
        i=i+1


print(lst_output)

day_new=np.arange(1,11)
day_pred=np.arange(11,11 + len(lst_output_flat))

df3=df1.tolist()
df3.extend(lst_output)

plt.plot(day_new,scaler.inverse_transform(df1[10:20]),label="Actual Data")
plt.plot(day_pred,scaler.inverse_transform(lst_output),label="Predicted Data")
```

```
Data fetched successfully
                       date   close      high      low     open    volume  \
0 2024-01-02 00:00:00+00:00  185.64  188.4400  183.885  187.150  82488674
1 2024-01-03 00:00:00+00:00  184.25  185.8800  183.430  184.220  58414460
2 2024-01-04 00:00:00+00:00  181.91  183.0872  180.880  182.150  71983570
3 2024-01-05 00:00:00+00:00  181.18  182.7600  180.170  181.990  62196924
4 2024-01-08 00:00:00+00:00  185.56  185.6000  181.500  182.085  59144470

     adjClose     adjHigh      adjLow     adjOpen  adjVolume  divCash  \
0  184.735476  187.521833  182.989027  186.238118   82488674      0.0
1  183.352248  184.974306  182.536244  183.322394   58414460      0.0
2  181.023650  182.195114  179.998668  181.262480   71983570      0.0
3  180.297207  181.869508  179.292128  181.103260   62196924      0.0
4  184.655865  184.695670  180.615648  181.197797   59144470      0.0

   splitFactor
0          1.0
1          1.0
2          1.0
3          1.0
4          1.0
df1 shape: (115,)
DF1 plot:
df1 shape after min max: (115, 1)
Train size: 74 Test size: 41
X_train shape: (63, 10) y_train shape: (63,)
X_test shape: (30, 10) y_test shape: (30,)
Model summary:
```

```
/home/rgukt/.local/lib/python3.10/site-packages/keras/src/layers/rnn/rnn.py:204:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
```

```
layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
lstm_12 (LSTM)	(None, 100, 50)	10,400
lstm_13 (LSTM)	(None, 100, 50)	20,200
lstm_14 (LSTM)	(None, 50)	20,200
dense_4 (Dense)	(None, 1)	51

Total params: 50,851 (198.64 KB)

Trainable params: 50,851 (198.64 KB)

Non-trainable params: 0 (0.00 B)

```
None
Epoch 1/100
1/1              8s 8s/step - loss:
0.1143 - val_loss: 0.2797
Epoch 2/100
1/1              0s 122ms/step - loss:
0.0970 - val_loss: 0.2476
Epoch 3/100
1/1              0s 119ms/step - loss:
0.0811 - val_loss: 0.2162
Epoch 4/100
1/1              0s 119ms/step - loss:
0.0663 - val_loss: 0.1846
Epoch 5/100
1/1              0s 111ms/step - loss:
0.0523 - val_loss: 0.1526
Epoch 6/100
1/1              0s 111ms/step - loss:
0.0394 - val_loss: 0.1203
Epoch 7/100
1/1              0s 104ms/step - loss:
0.0282 - val_loss: 0.0885
Epoch 8/100
```

```
1/1            0s 144ms/step - loss:
0.0196 - val_loss: 0.0592
Epoch 9/100
1/1            0s 116ms/step - loss:
0.0156 - val_loss: 0.0361
Epoch 10/100
1/1            0s 111ms/step - loss:
0.0177 - val_loss: 0.0229
Epoch 11/100
1/1            0s 112ms/step - loss:
0.0238 - val_loss: 0.0187
Epoch 12/100
1/1            0s 112ms/step - loss:
0.0275 - val_loss: 0.0194
Epoch 13/100
1/1            0s 114ms/step - loss:
0.0264 - val_loss: 0.0236
Epoch 14/100
1/1            0s 214ms/step - loss:
0.0225 - val_loss: 0.0309
Epoch 15/100
1/1            0s 183ms/step - loss:
0.0183 - val_loss: 0.0406
Epoch 16/100
1/1            0s 246ms/step - loss:
0.0156 - val_loss: 0.0515
Epoch 17/100
1/1            0s 109ms/step - loss:
0.0145 - val_loss: 0.0621
Epoch 18/100
1/1            0s 117ms/step - loss:
0.0148 - val_loss: 0.0713
Epoch 19/100
1/1            0s 115ms/step - loss:
0.0158 - val_loss: 0.0783
Epoch 20/100
1/1            0s 149ms/step - loss:
0.0168 - val_loss: 0.0826
Epoch 21/100
1/1            0s 120ms/step - loss:
0.0176 - val_loss: 0.0841
Epoch 22/100
1/1            0s 109ms/step - loss:
0.0178 - val_loss: 0.0830
Epoch 23/100
1/1            0s 116ms/step - loss:
0.0175 - val_loss: 0.0797
Epoch 24/100
```

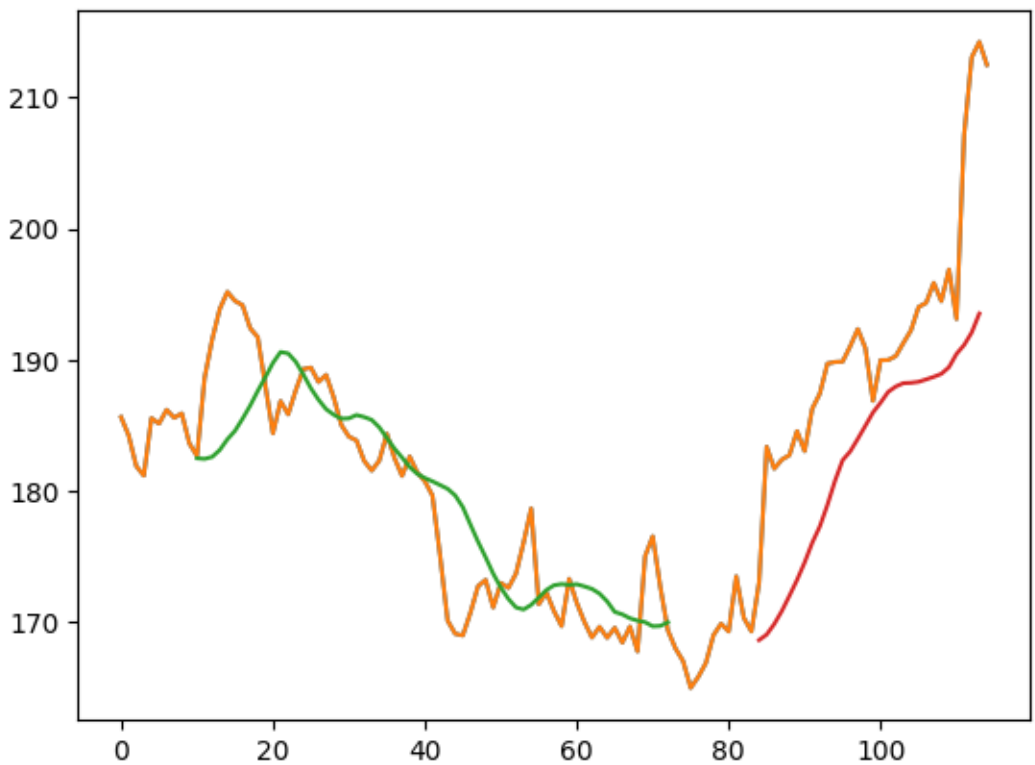
```
1/1              0s 116ms/step - loss:
0.0168 - val_loss: 0.0745
Epoch 25/100
1/1              0s 128ms/step - loss:
0.0157 - val_loss: 0.0680
Epoch 26/100
1/1              0s 129ms/step - loss:
0.0145 - val_loss: 0.0608
Epoch 27/100
1/1              0s 113ms/step - loss:
0.0135 - val_loss: 0.0534
Epoch 28/100
1/1              0s 109ms/step - loss:
0.0128 - val_loss: 0.0466
Epoch 29/100
1/1              0s 105ms/step - loss:
0.0125 - val_loss: 0.0408
Epoch 30/100
1/1              0s 111ms/step - loss:
0.0126 - val_loss: 0.0364
Epoch 31/100
1/1              0s 125ms/step - loss:
0.0130 - val_loss: 0.0337
Epoch 32/100
1/1              0s 86ms/step - loss:
0.0132 - val_loss: 0.0328
Epoch 33/100
1/1              0s 88ms/step - loss:
0.0132 - val_loss: 0.0334
Epoch 34/100
1/1              0s 96ms/step - loss:
0.0128 - val_loss: 0.0354
Epoch 35/100
1/1              0s 97ms/step - loss:
0.0122 - val_loss: 0.0386
Epoch 36/100
1/1              0s 90ms/step - loss:
0.0115 - val_loss: 0.0424
Epoch 37/100
1/1              0s 87ms/step - loss:
0.0110 - val_loss: 0.0464
Epoch 38/100
1/1              0s 88ms/step - loss:
0.0108 - val_loss: 0.0501
Epoch 39/100
1/1              0s 89ms/step - loss:
0.0108 - val_loss: 0.0529
Epoch 40/100
```

```
1/1            0s 90ms/step - loss:
0.0108 - val_loss: 0.0546
Epoch 41/100
1/1            0s 92ms/step - loss:
0.0108 - val_loss: 0.0550
Epoch 42/100
1/1            0s 103ms/step - loss:
0.0108 - val_loss: 0.0539
Epoch 43/100
1/1            0s 110ms/step - loss:
0.0106 - val_loss: 0.0517
Epoch 44/100
1/1            0s 112ms/step - loss:
0.0102 - val_loss: 0.0487
Epoch 45/100
1/1            0s 215ms/step - loss:
0.0099 - val_loss: 0.0452
Epoch 46/100
1/1            0s 215ms/step - loss:
0.0097 - val_loss: 0.0419
Epoch 47/100
1/1            0s 91ms/step - loss:
0.0096 - val_loss: 0.0391
Epoch 48/100
1/1            0s 139ms/step - loss:
0.0096 - val_loss: 0.0372
Epoch 49/100
1/1            0s 85ms/step - loss:
0.0097 - val_loss: 0.0365
Epoch 50/100
1/1            0s 100ms/step - loss:
0.0097 - val_loss: 0.0369
Epoch 51/100
1/1            0s 99ms/step - loss:
0.0096 - val_loss: 0.0382
Epoch 52/100
1/1            0s 101ms/step - loss:
0.0095 - val_loss: 0.0403
Epoch 53/100
1/1            0s 94ms/step - loss:
0.0093 - val_loss: 0.0427
Epoch 54/100
1/1            0s 90ms/step - loss:
0.0093 - val_loss: 0.0449
Epoch 55/100
1/1            0s 90ms/step - loss:
0.0093 - val_loss: 0.0465
Epoch 56/100
```

```
1/1            0s 103ms/step - loss:
0.0094 - val_loss: 0.0472
Epoch 57/100
1/1            0s 100ms/step - loss:
0.0094 - val_loss: 0.0470
Epoch 58/100
1/1            0s 90ms/step - loss:
0.0094 - val_loss: 0.0459
Epoch 59/100
1/1            0s 94ms/step - loss:
0.0094 - val_loss: 0.0442
Epoch 60/100
1/1            0s 91ms/step - loss:
0.0093 - val_loss: 0.0424
Epoch 61/100
1/1            0s 95ms/step - loss:
0.0093 - val_loss: 0.0407
Epoch 62/100
1/1            0s 98ms/step - loss:
0.0093 - val_loss: 0.0395
Epoch 63/100
1/1            0s 99ms/step - loss:
0.0093 - val_loss: 0.0390
Epoch 64/100
1/1            0s 99ms/step - loss:
0.0093 - val_loss: 0.0393
Epoch 65/100
1/1            0s 96ms/step - loss:
0.0093 - val_loss: 0.0401
Epoch 66/100
1/1            0s 103ms/step - loss:
0.0093 - val_loss: 0.0412
Epoch 67/100
1/1            0s 102ms/step - loss:
0.0092 - val_loss: 0.0425
Epoch 68/100
1/1            0s 103ms/step - loss:
0.0092 - val_loss: 0.0434
Epoch 69/100
1/1            0s 104ms/step - loss:
0.0092 - val_loss: 0.0440
Epoch 70/100
1/1            0s 104ms/step - loss:
0.0092 - val_loss: 0.0440
Epoch 71/100
1/1            0s 107ms/step - loss:
0.0092 - val_loss: 0.0435
Epoch 72/100
```

```
1/1              0s 92ms/step - loss:
0.0092 - val_loss: 0.0426
Epoch 73/100
1/1              0s 91ms/step - loss:
0.0091 - val_loss: 0.0415
Epoch 74/100
1/1              0s 89ms/step - loss:
0.0091 - val_loss: 0.0406
Epoch 75/100
1/1              0s 92ms/step - loss:
0.0091 - val_loss: 0.0398
Epoch 76/100
1/1              0s 96ms/step - loss:
0.0091 - val_loss: 0.0395
Epoch 77/100
1/1              0s 97ms/step - loss:
0.0091 - val_loss: 0.0395
Epoch 78/100
1/1              0s 93ms/step - loss:
0.0091 - val_loss: 0.0399
Epoch 79/100
1/1              0s 92ms/step - loss:
0.0091 - val_loss: 0.0405
Epoch 80/100
1/1              0s 96ms/step - loss:
0.0090 - val_loss: 0.0411
Epoch 81/100
1/1              0s 92ms/step - loss:
0.0090 - val_loss: 0.0416
Epoch 82/100
1/1              0s 93ms/step - loss:
0.0090 - val_loss: 0.0418
Epoch 83/100
1/1              0s 89ms/step - loss:
0.0090 - val_loss: 0.0418
Epoch 84/100
1/1              0s 94ms/step - loss:
0.0090 - val_loss: 0.0415
Epoch 85/100
1/1              0s 96ms/step - loss:
0.0090 - val_loss: 0.0410
Epoch 86/100
1/1              0s 91ms/step - loss:
0.0090 - val_loss: 0.0404
Epoch 87/100
1/1              0s 88ms/step - loss:
0.0090 - val_loss: 0.0398
Epoch 88/100
```

```
1/1            0s 99ms/step - loss:
0.0090 - val_loss: 0.0395
Epoch 89/100
1/1            0s 106ms/step - loss:
0.0090 - val_loss: 0.0393
Epoch 90/100
1/1            0s 103ms/step - loss:
0.0089 - val_loss: 0.0393
Epoch 91/100
1/1            0s 132ms/step - loss:
0.0089 - val_loss: 0.0395
Epoch 92/100
1/1            0s 93ms/step - loss:
0.0089 - val_loss: 0.0398
Epoch 93/100
1/1            0s 94ms/step - loss:
0.0089 - val_loss: 0.0401
Epoch 94/100
1/1            0s 97ms/step - loss:
0.0089 - val_loss: 0.0402
Epoch 95/100
1/1            0s 96ms/step - loss:
0.0089 - val_loss: 0.0403
Epoch 96/100
1/1            0s 129ms/step - loss:
0.0089 - val_loss: 0.0402
Epoch 97/100
1/1            0s 110ms/step - loss:
0.0089 - val_loss: 0.0399
Epoch 98/100
1/1            0s 129ms/step - loss:
0.0088 - val_loss: 0.0395
Epoch 99/100
1/1            0s 99ms/step - loss:
0.0088 - val_loss: 0.0392
Epoch 100/100
1/1            0s 138ms/step - loss:
0.0088 - val_loss: 0.0389
2/2            1s 500ms/step
1/1            0s 29ms/step
```

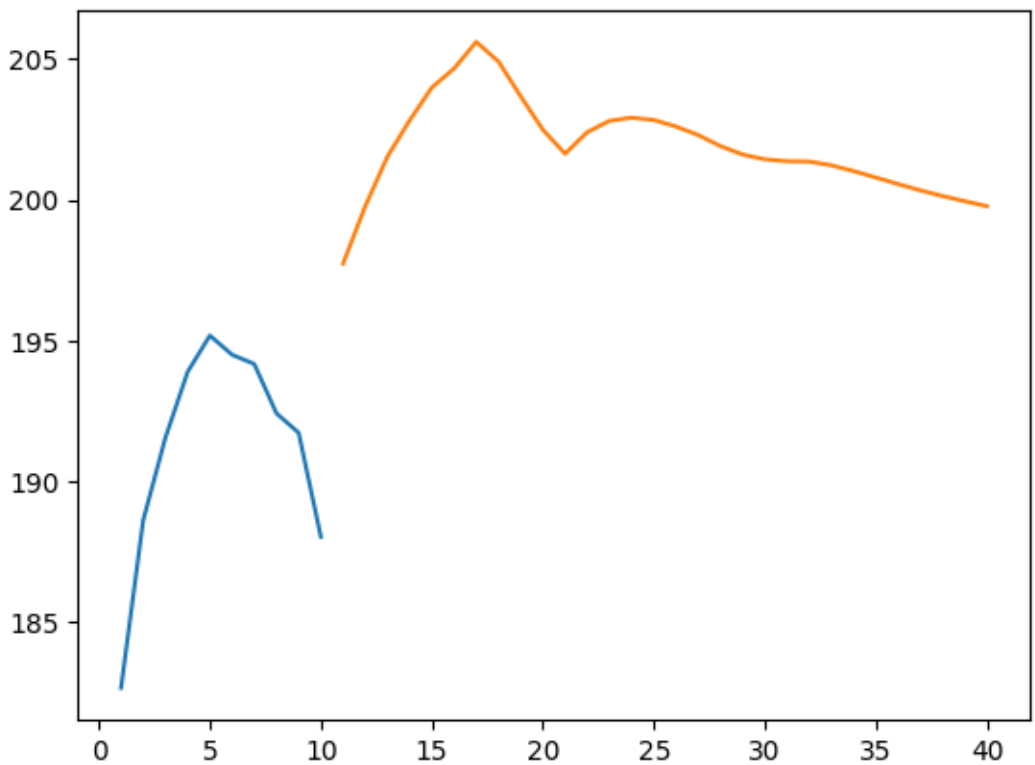



```
Test data len: 41
[0.66473925]
11
1 day input [0.59606011 0.62692933 0.59870024 0.64764419 0.57108042 0.85601137
 0.97623883 1.         0.96445979 0.66473925]
1 day output [[0.70616925]]
2 day input [0.62692933 0.59870024 0.64764419 0.57108042 0.85601137 0.97623883
 1.         0.96445979 0.66473925 0.70616925]
2 day output [[0.7418335]]
3 day input [0.59870024 0.64764419 0.57108042 0.85601137 0.97623883 1.
 0.96445979 0.66473925 0.70616925 0.74183351]
3 day output [[0.76797605]]
4 day input [0.64764419 0.57108042 0.85601137 0.97623883 1.         0.96445979
 0.66473925 0.70616925 0.74183351 0.76797605]
4 day output [[0.7916016]]
5 day input [0.57108042 0.85601137 0.97623883 1.         0.96445979 0.66473925
 0.70616925 0.74183351 0.76797605 0.7916016 ]
5 day output [[0.80531293]]
6 day input [0.85601137 0.97623883 1.         0.96445979 0.66473925 0.70616925
 0.74183351 0.76797605 0.7916016  0.80531293]
6 day output [[0.8244939]]
7 day input [0.97623883 1.         0.96445979 0.66473925 0.70616925 0.74183351
```

```
 0.76797605 0.7916016  0.80531293 0.82449389]
7 day output [[0.81035566]]
8 day input [1.         0.96445979 0.66473925 0.70616925 0.74183351 0.76797605
 0.7916016  0.80531293 0.82449389 0.81035566]
8 day output [[0.7853109]]
9 day input [0.96445979 0.66473925 0.70616925 0.74183351 0.76797605 0.7916016
 0.80531293 0.82449389 0.81035566 0.78531092]
9 day output [[0.76123077]]
10 day input [0.66473925 0.70616925 0.74183351 0.76797605 0.7916016  0.80531293
 0.82449389 0.81035566 0.78531092 0.76123077]
10 day output [[0.7439083]]
11 day input [0.70616925 0.74183351 0.76797605 0.7916016  0.80531293 0.82449389
 0.81035566 0.78531092 0.76123077 0.74390829]
11 day output [[0.759455]]
12 day input [0.74183351 0.76797605 0.7916016  0.80531293 0.82449389 0.81035566
 0.78531092 0.76123077 0.74390829 0.75945503]
12 day output [[0.7676115]]
13 day input [0.76797605 0.7916016  0.80531293 0.82449389 0.81035566 0.78531092
 0.76123077 0.74390829 0.75945503 0.7676115 ]
13 day output [[0.7698845]]
14 day input [0.7916016  0.80531293 0.82449389 0.81035566 0.78531092 0.76123077
 0.74390829 0.75945503 0.7676115  0.76988453]
14 day output [[0.7681975]]
15 day input [0.80531293 0.82449389 0.81035566 0.78531092 0.76123077 0.74390829
 0.75945503 0.7676115  0.76988453 0.76819748]
15 day output [[0.76350844]]
16 day input [0.82449389 0.81035566 0.78531092 0.76123077 0.74390829 0.75945503
 0.7676115  0.76988453 0.76819748 0.76350844]
16 day output [[0.7573668]]
17 day input [0.81035566 0.78531092 0.76123077 0.74390829 0.75945503 0.7676115
 0.76988453 0.76819748 0.76350844 0.75736678]
17 day output [[0.74940896]]
18 day input [0.78531092 0.76123077 0.74390829 0.75945503 0.7676115  0.76988453
 0.76819748 0.76350844 0.75736678 0.74940896]
18 day output [[0.74331445]]
19 day input [0.76123077 0.74390829 0.75945503 0.7676115  0.76988453 0.76819748
 0.76350844 0.75736678 0.74940896 0.74331445]
19 day output [[0.7398741]]
20 day input [0.74390829 0.75945503 0.7676115  0.76988453 0.76819748 0.76350844
 0.75736678 0.74940896 0.74331445 0.73987412]
20 day output [[0.7385418]]
21 day input [0.75945503 0.7676115  0.76988453 0.76819748 0.76350844 0.75736678
 0.74940896 0.74331445 0.73987412 0.73854178]
21 day output [[0.73829144]]
22 day input [0.7676115  0.76988453 0.76819748 0.76350844 0.75736678 0.74940896
 0.74331445 0.73987412 0.73854178 0.73829144]
22 day output [[0.7355423]]
23 day input [0.76988453 0.76819748 0.76350844 0.75736678 0.74940896 0.74331445
```

```
 0.73987412 0.73854178 0.73829144 0.7355423 ]
23 day output [[0.7314266]]
24 day input [0.76819748 0.76350844 0.75736678 0.74940896 0.74331445 0.73987412
 0.73854178 0.73829144 0.7355423  0.7314266 ]
24 day output [[0.7267658]]
25 day input [0.76350844 0.75736678 0.74940896 0.74331445 0.73987412 0.73854178
 0.73829144 0.7355423  0.7314266  0.72676581]
25 day output [[0.7220465]]
26 day input [0.75736678 0.74940896 0.74331445 0.73987412 0.73854178 0.73829144
 0.7355423  0.7314266  0.72676581 0.72204649]
26 day output [[0.71756756]]
27 day input [0.74940896 0.74331445 0.73987412 0.73854178 0.73829144 0.7355423
 0.7314266  0.72676581 0.72204649 0.71756756]
27 day output [[0.71341455]]
28 day input [0.74331445 0.73987412 0.73854178 0.73829144 0.7355423  0.7314266
 0.72676581 0.72204649 0.71756756 0.71341455]
28 day output [[0.70969975]]
29 day input [0.73987412 0.73854178 0.73829144 0.7355423  0.7314266  0.72676581
 0.72204649 0.71756756 0.71341455 0.70969975]
29 day output [[0.70614505]]
[[0.6647392511367798], [0.7061692476272583], [0.741833508014679],
[0.7679760456085205], [0.7916015982627869], [0.8053129315376282],
[0.8244938850402832], [0.8103556632995605], [0.7853109240531921],
[0.7612307667732239], [0.7439082860946655], [0.7594550251960754],
[0.7676115036010742], [0.7698845267295837], [0.7681974768638611],
[0.7635084390640259], [0.7573667764663696], [0.7494089603424072],
[0.7433144450187683], [0.7398741245269775], [0.7385417819023132],
[0.7382914423942566], [0.7355422973632812], [0.7314265966415405],
[0.7267658114433289], [0.7220464944839478], [0.7175675630569458],
[0.7134145498275757], [0.7096997499465942], [0.7061450481414795]]
```

[71]:
```
[<matplotlib.lines.Line2D at 0x7fab4fa78d00>]
```



3.4 Machine Learning Language Translation

[2]:
```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input,LSTM,Dense
import numpy as np

batch_size=64  #batch size for training
epochs=100   #no.of epochs to train for
latent_dim=256  #Latent dimensionality of the encoding space
num_samples=10000  #no of samples to train
#data path
data_path='fra-eng/fra.txt'

# Vectorize the data.
input_texts = []
target_texts = []
input_characters = set()
target_characters = set()
with open(data_path, "r", encoding="utf-8") as f:
    lines = f.read().split("\n")
for line in lines[: min(num_samples, len(lines) - 1)]:
    input_text, target_text, _ = line.split("\t")
```

```
    # We use "tab" as the "start sequence" character
    # for the targets, and "\n" as "end sequence" character.
    target_text = "\t" + target_text + "\n"
    input_texts.append(input_text)
    target_texts.append(target_text)
    for char in input_text:
        if char not in input_characters:
            input_characters.add(char)
    for char in target_text:
        if char not in target_characters:
            target_characters.add(char)


input_characters=sorted(list(input_characters))
target_characters=sorted(list(target_characters))
num_encoder_tokens=len(input_characters)
num_decoder_tokens=len(target_characters)
max_encoder_seq_length=max([len(txt) for txt in input_texts])
max_decoder_seq_length=max([len(txt) for txt in target_texts])

print('No.of samples:',len(input_texts))
print('No.of unique input_tokens:',num_encoder_tokens)
print('No.of unique output_tokens:',num_decoder_tokens)
print('Max sequence length for inputs:',max_encoder_seq_length)
print('Max sequence length for outputs:',max_decoder_seq_length)

input_token_index=dict([(char,i) for i ,char in enumerate(input_characters)])
target_token_index=dict([(char,i) for i,char in enumerate(target_characters)])

encoder_input_data=np.
 ↪zeros((len(input_texts),max_encoder_seq_length,num_encoder_tokens),dtype='float32')
decoder_input_data=np.
 ↪zeros((len(input_texts),max_decoder_seq_length,num_decoder_tokens),dtype='float32')
decoder_target_data=np.
 ↪zeros((len(input_texts),max_decoder_seq_length,num_decoder_tokens),dtype='float32')

for i,(input_text,target_text) in enumerate(zip(input_texts,target_texts)):
    for t,char in enumerate(input_text):
        encoder_input_data[i,t,input_token_index[char]]=1.
    encoder_input_data[i,t+1:,input_token_index[' ']]=1.
    for t,char in enumerate(target_text):
        #decoder_target_data is ahead of decoder_input_data by ome timestep
        decoder_input_data[i,t,target_token_index[char]]=1.
        if t>0:
            #decoder_target_data will be ahead by one timestep not include␣
 ↪start character
            decoder_target_data[i,t-1,target_token_index[char]]=1.
```

```
    decoder_input_data[i,t+1:,target_token_index[' ']]=1.
    decoder_target_data[i,t:,target_token_index[' ']]=1.

print("encoder input data shape:",encoder_input_data[0].shape)

#define an input sequence and process it
encoder_inputs=Input(shape=(None,num_encoder_tokens))
encoder=LSTM(latent_dim,return_state=True)
encoder_outputs,state_h,state_c=encoder(encoder_inputs)
#we discard encoder_outputs and only keep states
encoder_states=[state_h,state_c]


#set up decoder using encoder_states as initial state.
decoder_inputs=Input(shape=(None,num_decoder_tokens))
#set up decoder to return full output sequences and return internal states as␣
 ↪well
#dont use return states in training model,but we will use them in inference
decoder_lstm=LSTM(latent_dim,return_sequences=True,return_state=True)
decoder_outputs,_,_=decoder_lstm(decoder_inputs,initial_state=encoder_states)
decoder_dense=Dense(num_decoder_tokens,activation='softmax')
decoder_outputs=decoder_dense(decoder_outputs)

#define model turn encoder_input_data & decoder_input_data into␣
 ↪decoder_target_data
model=Model([encoder_inputs,decoder_inputs],decoder_outputs)

model.
 ↪compile(optimizer='rmsprop',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit([encoder_input_data,decoder_input_data],decoder_target_data,
          batch_size=batch_size,
          epochs=epochs,
          validation_split=0.2)

# Save model
model.save("s2s_model.keras")

#next: interference mode (sampling)
#1.encode input & retrieve initial decoder state
#2.run one step of decoder with this initial state & 'start of sequence' token␣
 ↪as target
#output next target token
#3.repeat with current target token & current states

#define sampling models
encoder_model= Model(encoder_inputs,encoder_states)
decoder_state_input_h=Input(shape=(latent_dim,))
```

```
decoder_state_input_c=Input(shape=(latent_dim,))
decoder_states_inputs=[decoder_state_input_h,decoder_state_input_c]
decoder_outputs,states_h,state_c=decoder_lstm(decoder_inputs,initial_state=decoder_states_inpu
decoder_states=[states_h,state_c]
decoder_outputs=decoder_dense(decoder_outputs)
decoder_model=Model([decoder_inputs]+decoder_states_inputs,
                    [decoder_outputs]+decoder_states)

#reverse-look up token index to decode sequence back to readable
reverse_input_char_index=dict((i,char) for char,i in input_token_index.items())
reverse_target_char_index=dict((i,char) for char,i in target_token_index.
 ↪items())

def decode_sequence(input_seq):
    #encode input as state vectors
    states_value=encoder_model.predict(input_seq)
    #generate empty target sequence of length 1
    target_seq=np.zeros((1,1,num_decoder_tokens))
    #populate first character of target sequences with start character
    target_seq[0,0,target_token_index['\t']]=1.

    #sampling loop for batch of statements ..bath size is 1
    stop_condition=False
    decoded_sentence=' '
    while not stop_condition:
        output_tokens,h,c=decoder_model.predict([target_seq]+states_value)
        #sample a token
         #sample a token
        sampled_token_index=np.argmax(output_tokens[0,-1,:])
        sampled_char=reverse_target_char_index[sampled_token_index]
        decoded_sentence+=sampled_char

        #exit:hit max length or find stop character
        if sampled_char== '\n' or len(decoded_sentence) >␣
 ↪max_decoder_seq_length:
            stop_condition=True
        #update target sequence (of length 1)
        target_seq=np.zeros((1,1,num_decoder_tokens))
        target_seq[0,0,sampled_token_index]=1.
        #update sates
        states_value=[h,c]
    return decoded_sentence

for seq_index in range(100):
    #take 1 sequence (part of training set) trying to decoding
    input_seq=encoder_input_data[seq_index:seq_index+1]
    decoded_sentence=decode_sequence(input_seq)
```

```
    print('-')
    print('input sentence:',input_texts[seq_index])
    print('decoded sentence:',decoded_sentence)
```

```
No.of samples: 10000
No.of unique input_tokens: 70
No.of unique output_tokens: 91
Max sequence length for inputs: 14
Max sequence length for outputs: 59
encoder input data shape: (14, 70)

2024-12-28 13:47:11.088903: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
171808000 exceeds 10% of free system memory.

Epoch 1/100

2024-12-28 13:47:11.891911: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
171808000 exceeds 10% of free system memory.

125/125              59s 449ms/step -
accuracy: 0.7077 - loss: 1.5173 - val_accuracy: 0.7141 - val_loss: 1.0562
Epoch 2/100
125/125              55s 440ms/step -
accuracy: 0.7483 - loss: 0.9440 - val_accuracy: 0.7319 - val_loss: 0.9544
Epoch 3/100
125/125              56s 448ms/step -
accuracy: 0.7648 - loss: 0.8462 - val_accuracy: 0.7574 - val_loss: 0.8494
Epoch 4/100
125/125              56s 448ms/step -
accuracy: 0.7899 - loss: 0.7495 - val_accuracy: 0.7815 - val_loss: 0.7666
Epoch 5/100
125/125              78s 412ms/step -
accuracy: 0.8068 - loss: 0.6711 - val_accuracy: 0.7927 - val_loss: 0.7128
Epoch 6/100
125/125              43s 345ms/step -
accuracy: 0.8191 - loss: 0.6240 - val_accuracy: 0.8019 - val_loss: 0.6797
Epoch 7/100
125/125              43s 348ms/step -
accuracy: 0.8266 - loss: 0.5938 - val_accuracy: 0.8084 - val_loss: 0.6569
Epoch 8/100
125/125              47s 373ms/step -
accuracy: 0.8314 - loss: 0.5751 - val_accuracy: 0.8160 - val_loss: 0.6296
Epoch 9/100
125/125              87s 413ms/step -
accuracy: 0.8373 - loss: 0.5535 - val_accuracy: 0.8200 - val_loss: 0.6132
Epoch 10/100
125/125              83s 424ms/step -
```

```
accuracy: 0.8410 - loss: 0.5403 - val_accuracy: 0.8296 - val_loss: 0.5927
Epoch 11/100
125/125             50s 398ms/step -
accuracy: 0.8470 - loss: 0.5209 - val_accuracy: 0.8313 - val_loss: 0.5819
Epoch 12/100
125/125             49s 391ms/step -
accuracy: 0.8506 - loss: 0.5100 - val_accuracy: 0.8279 - val_loss: 0.5805
Epoch 13/100
125/125             47s 378ms/step -
accuracy: 0.8549 - loss: 0.4935 - val_accuracy: 0.8395 - val_loss: 0.5577
Epoch 14/100
125/125             47s 375ms/step -
accuracy: 0.8595 - loss: 0.4794 - val_accuracy: 0.8391 - val_loss: 0.5530
Epoch 15/100
125/125             81s 371ms/step -
accuracy: 0.8629 - loss: 0.4656 - val_accuracy: 0.8419 - val_loss: 0.5395
Epoch 16/100
125/125             82s 371ms/step -
accuracy: 0.8657 - loss: 0.4553 - val_accuracy: 0.8463 - val_loss: 0.5321
Epoch 17/100
125/125             44s 351ms/step -
accuracy: 0.8677 - loss: 0.4479 - val_accuracy: 0.8468 - val_loss: 0.5227
Epoch 18/100
125/125             84s 369ms/step -
accuracy: 0.8701 - loss: 0.4384 - val_accuracy: 0.8493 - val_loss: 0.5159
Epoch 19/100
125/125             44s 352ms/step -
accuracy: 0.8737 - loss: 0.4279 - val_accuracy: 0.8500 - val_loss: 0.5114
Epoch 20/100
125/125             45s 356ms/step -
accuracy: 0.8750 - loss: 0.4223 - val_accuracy: 0.8534 - val_loss: 0.5029
Epoch 21/100
125/125             84s 374ms/step -
accuracy: 0.8773 - loss: 0.4129 - val_accuracy: 0.8530 - val_loss: 0.4979
Epoch 22/100
125/125             82s 372ms/step -
accuracy: 0.8786 - loss: 0.4086 - val_accuracy: 0.8533 - val_loss: 0.4961
Epoch 23/100
125/125             44s 353ms/step -
accuracy: 0.8818 - loss: 0.3988 - val_accuracy: 0.8553 - val_loss: 0.4902
Epoch 24/100
125/125             84s 368ms/step -
accuracy: 0.8834 - loss: 0.3917 - val_accuracy: 0.8571 - val_loss: 0.4866
Epoch 25/100
125/125             45s 356ms/step -
accuracy: 0.8853 - loss: 0.3850 - val_accuracy: 0.8582 - val_loss: 0.4791
Epoch 26/100
125/125             84s 374ms/step -
```

```
accuracy: 0.8873 - loss: 0.3785 - val_accuracy: 0.8594 - val_loss: 0.4777
Epoch 27/100
125/125             44s 354ms/step -
accuracy: 0.8895 - loss: 0.3711 - val_accuracy: 0.8604 - val_loss: 0.4754
Epoch 28/100
125/125             45s 356ms/step -
accuracy: 0.8906 - loss: 0.3675 - val_accuracy: 0.8613 - val_loss: 0.4721
Epoch 29/100
125/125             84s 372ms/step -
accuracy: 0.8913 - loss: 0.3644 - val_accuracy: 0.8634 - val_loss: 0.4680
Epoch 30/100
125/125             82s 373ms/step -
accuracy: 0.8935 - loss: 0.3563 - val_accuracy: 0.8637 - val_loss: 0.4670
Epoch 31/100
125/125             82s 370ms/step -
accuracy: 0.8952 - loss: 0.3525 - val_accuracy: 0.8651 - val_loss: 0.4608
Epoch 32/100
125/125             44s 355ms/step -
accuracy: 0.8978 - loss: 0.3437 - val_accuracy: 0.8655 - val_loss: 0.4596
Epoch 33/100
125/125             84s 370ms/step -
accuracy: 0.8978 - loss: 0.3427 - val_accuracy: 0.8664 - val_loss: 0.4573
Epoch 34/100
125/125             45s 355ms/step -
accuracy: 0.9009 - loss: 0.3336 - val_accuracy: 0.8668 - val_loss: 0.4566
Epoch 35/100
125/125             45s 358ms/step -
accuracy: 0.9019 - loss: 0.3298 - val_accuracy: 0.8670 - val_loss: 0.4564
Epoch 36/100
125/125             45s 356ms/step -
accuracy: 0.9027 - loss: 0.3265 - val_accuracy: 0.8658 - val_loss: 0.4567
Epoch 37/100
125/125             44s 355ms/step -
accuracy: 0.9038 - loss: 0.3234 - val_accuracy: 0.8672 - val_loss: 0.4559
Epoch 38/100
125/125             85s 378ms/step -
accuracy: 0.9066 - loss: 0.3139 - val_accuracy: 0.8691 - val_loss: 0.4501
Epoch 39/100
125/125             81s 373ms/step -
accuracy: 0.9063 - loss: 0.3139 - val_accuracy: 0.8695 - val_loss: 0.4500
Epoch 40/100
125/125             45s 357ms/step -
accuracy: 0.9081 - loss: 0.3098 - val_accuracy: 0.8712 - val_loss: 0.4459
Epoch 41/100
125/125             1167s 380ms/step
- accuracy: 0.9105 - loss: 0.3017 - val_accuracy: 0.8703 - val_loss: 0.4477
Epoch 42/100
125/125             80s 369ms/step -
```

```
accuracy: 0.9112 - loss: 0.2982 - val_accuracy: 0.8707 - val_loss: 0.4482
Epoch 43/100
125/125             44s 354ms/step -
accuracy: 0.9126 - loss: 0.2936 - val_accuracy: 0.8719 - val_loss: 0.4442
Epoch 44/100
125/125             84s 369ms/step -
accuracy: 0.9142 - loss: 0.2879 - val_accuracy: 0.8714 - val_loss: 0.4457
Epoch 45/100
125/125             82s 368ms/step -
accuracy: 0.9148 - loss: 0.2836 - val_accuracy: 0.8724 - val_loss: 0.4433
Epoch 46/100
125/125             82s 369ms/step -
accuracy: 0.9158 - loss: 0.2817 - val_accuracy: 0.8725 - val_loss: 0.4440
Epoch 47/100
125/125             44s 353ms/step -
accuracy: 0.9176 - loss: 0.2765 - val_accuracy: 0.8734 - val_loss: 0.4443
Epoch 48/100
125/125             44s 354ms/step -
accuracy: 0.9184 - loss: 0.2745 - val_accuracy: 0.8730 - val_loss: 0.4432
Epoch 49/100
125/125             45s 359ms/step -
accuracy: 0.9195 - loss: 0.2696 - val_accuracy: 0.8730 - val_loss: 0.4456
Epoch 50/100
125/125             84s 377ms/step -
accuracy: 0.9204 - loss: 0.2676 - val_accuracy: 0.8720 - val_loss: 0.4479
Epoch 51/100
125/125             45s 356ms/step -
accuracy: 0.9222 - loss: 0.2614 - val_accuracy: 0.8741 - val_loss: 0.4433
Epoch 52/100
125/125             45s 361ms/step -
accuracy: 0.9231 - loss: 0.2579 - val_accuracy: 0.8725 - val_loss: 0.4475
Epoch 53/100
125/125             44s 356ms/step -
accuracy: 0.9230 - loss: 0.2565 - val_accuracy: 0.8741 - val_loss: 0.4443
Epoch 54/100
125/125             44s 356ms/step -
accuracy: 0.9255 - loss: 0.2499 - val_accuracy: 0.8749 - val_loss: 0.4463
Epoch 55/100
125/125             44s 350ms/step -
accuracy: 0.9268 - loss: 0.2466 - val_accuracy: 0.8738 - val_loss: 0.4452
Epoch 56/100
125/125             45s 358ms/step -
accuracy: 0.9271 - loss: 0.2426 - val_accuracy: 0.8747 - val_loss: 0.4481
Epoch 57/100
125/125             44s 356ms/step -
accuracy: 0.9281 - loss: 0.2400 - val_accuracy: 0.8748 - val_loss: 0.4481
Epoch 58/100
125/125             84s 371ms/step -
```

```
accuracy: 0.9285 - loss: 0.2403 - val_accuracy: 0.8743 - val_loss: 0.4482
Epoch 59/100
125/125             44s 353ms/step -
accuracy: 0.9310 - loss: 0.2320 - val_accuracy: 0.8754 - val_loss: 0.4476
Epoch 60/100
125/125             45s 356ms/step -
accuracy: 0.9314 - loss: 0.2292 - val_accuracy: 0.8767 - val_loss: 0.4482
Epoch 61/100
125/125             44s 354ms/step -
accuracy: 0.9325 - loss: 0.2260 - val_accuracy: 0.8763 - val_loss: 0.4497
Epoch 62/100
125/125             2123s 17s/step -
accuracy: 0.9340 - loss: 0.2217 - val_accuracy: 0.8750 - val_loss: 0.4538
Epoch 63/100
125/125             48s 380ms/step -
accuracy: 0.9339 - loss: 0.2216 - val_accuracy: 0.8763 - val_loss: 0.4540
Epoch 64/100
125/125             74s 320ms/step -
accuracy: 0.9356 - loss: 0.2160 - val_accuracy: 0.8756 - val_loss: 0.4557
Epoch 65/100
125/125             41s 324ms/step -
accuracy: 0.9360 - loss: 0.2142 - val_accuracy: 0.8766 - val_loss: 0.4578
Epoch 66/100
125/125             40s 323ms/step -
accuracy: 0.9368 - loss: 0.2107 - val_accuracy: 0.8763 - val_loss: 0.4588
Epoch 67/100
125/125             41s 325ms/step -
accuracy: 0.9376 - loss: 0.2077 - val_accuracy: 0.8754 - val_loss: 0.4598
Epoch 68/100
125/125             40s 323ms/step -
accuracy: 0.9389 - loss: 0.2047 - val_accuracy: 0.8762 - val_loss: 0.4599
Epoch 69/100
125/125             42s 333ms/step -
accuracy: 0.9399 - loss: 0.2003 - val_accuracy: 0.8758 - val_loss: 0.4642
Epoch 70/100
125/125             41s 325ms/step -
accuracy: 0.9405 - loss: 0.1988 - val_accuracy: 0.8755 - val_loss: 0.4644
Epoch 71/100
125/125             41s 324ms/step -
accuracy: 0.9415 - loss: 0.1967 - val_accuracy: 0.8757 - val_loss: 0.4674
Epoch 72/100
125/125             41s 329ms/step -
accuracy: 0.9420 - loss: 0.1945 - val_accuracy: 0.8769 - val_loss: 0.4679
Epoch 73/100
125/125             41s 326ms/step -
accuracy: 0.9427 - loss: 0.1914 - val_accuracy: 0.8742 - val_loss: 0.4743
Epoch 74/100
125/125             41s 328ms/step -
```

```
accuracy: 0.9437 - loss: 0.1877 - val_accuracy: 0.8756 - val_loss: 0.4721
Epoch 75/100
125/125             41s 325ms/step -
accuracy: 0.9443 - loss: 0.1852 - val_accuracy: 0.8757 - val_loss: 0.4759
Epoch 76/100
125/125             41s 328ms/step -
accuracy: 0.9448 - loss: 0.1831 - val_accuracy: 0.8751 - val_loss: 0.4787
Epoch 77/100
125/125             41s 330ms/step -
accuracy: 0.9455 - loss: 0.1809 - val_accuracy: 0.8765 - val_loss: 0.4787
Epoch 78/100
125/125             41s 327ms/step -
accuracy: 0.9465 - loss: 0.1775 - val_accuracy: 0.8756 - val_loss: 0.4831
Epoch 79/100
125/125             41s 327ms/step -
accuracy: 0.9478 - loss: 0.1737 - val_accuracy: 0.8746 - val_loss: 0.4838
Epoch 80/100
125/125             41s 328ms/step -
accuracy: 0.9477 - loss: 0.1736 - val_accuracy: 0.8757 - val_loss: 0.4853
Epoch 81/100
125/125             82s 328ms/step -
accuracy: 0.9488 - loss: 0.1710 - val_accuracy: 0.8760 - val_loss: 0.4853
Epoch 82/100
125/125             41s 329ms/step -
accuracy: 0.9492 - loss: 0.1690 - val_accuracy: 0.8762 - val_loss: 0.4900
Epoch 83/100
125/125             41s 330ms/step -
accuracy: 0.9506 - loss: 0.1649 - val_accuracy: 0.8751 - val_loss: 0.4936
Epoch 84/100
125/125             41s 326ms/step -
accuracy: 0.9507 - loss: 0.1644 - val_accuracy: 0.8749 - val_loss: 0.4944
Epoch 85/100
125/125             41s 329ms/step -
accuracy: 0.9518 - loss: 0.1602 - val_accuracy: 0.8749 - val_loss: 0.4986
Epoch 86/100
125/125             41s 328ms/step -
accuracy: 0.9515 - loss: 0.1602 - val_accuracy: 0.8738 - val_loss: 0.5030
Epoch 87/100
125/125             41s 329ms/step -
accuracy: 0.9526 - loss: 0.1567 - val_accuracy: 0.8752 - val_loss: 0.5020
Epoch 88/100
125/125             41s 328ms/step -
accuracy: 0.9533 - loss: 0.1548 - val_accuracy: 0.8744 - val_loss: 0.5061
Epoch 89/100
125/125             81s 324ms/step -
accuracy: 0.9538 - loss: 0.1539 - val_accuracy: 0.8744 - val_loss: 0.5088
Epoch 90/100
125/125             42s 330ms/step -
```

```
accuracy: 0.9549 - loss: 0.1503 - val_accuracy: 0.8750 - val_loss: 0.5081
Epoch 91/100
125/125             40s 322ms/step -
accuracy: 0.9548 - loss: 0.1491 - val_accuracy: 0.8746 - val_loss: 0.5136
Epoch 92/100
125/125             41s 325ms/step -
accuracy: 0.9564 - loss: 0.1457 - val_accuracy: 0.8760 - val_loss: 0.5124
Epoch 93/100
125/125             41s 327ms/step -
accuracy: 0.9574 - loss: 0.1426 - val_accuracy: 0.8750 - val_loss: 0.5152
Epoch 94/100
125/125             41s 327ms/step -
accuracy: 0.9569 - loss: 0.1429 - val_accuracy: 0.8736 - val_loss: 0.5235
Epoch 95/100
125/125             41s 325ms/step -
accuracy: 0.9574 - loss: 0.1407 - val_accuracy: 0.8740 - val_loss: 0.5197
Epoch 96/100
125/125             41s 326ms/step -
accuracy: 0.9582 - loss: 0.1393 - val_accuracy: 0.8741 - val_loss: 0.5264
Epoch 97/100
125/125             41s 329ms/step -
accuracy: 0.9589 - loss: 0.1369 - val_accuracy: 0.8725 - val_loss: 0.5290
Epoch 98/100
125/125             43s 343ms/step -
accuracy: 0.9587 - loss: 0.1359 - val_accuracy: 0.8742 - val_loss: 0.5315
Epoch 99/100
125/125             80s 330ms/step -
accuracy: 0.9603 - loss: 0.1317 - val_accuracy: 0.8740 - val_loss: 0.5338
Epoch 100/100
125/125             41s 329ms/step -
accuracy: 0.9608 - loss: 0.1297 - val_accuracy: 0.8745 - val_loss: 0.5376
1/1             0s 122ms/step
1/1             0s 205ms/step
1/1             0s 22ms/step
1/1             0s 22ms/step
1/1             0s 59ms/step
1/1             0s 34ms/step
1/1             0s 21ms/step
1/1             0s 21ms/step
-
input sentence: Go.
decoded sentence:  Pars !

1/1             0s 25ms/step
1/1             0s 21ms/step
1/1             0s 22ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
```

```
1/1              0s 21ms/step
1/1              0s 23ms/step
1/1              0s 23ms/step
-
input sentence: Go.
decoded sentence:  Pars !

1/1              0s 26ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 45ms/step
1/1              0s 28ms/step
1/1              0s 31ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
-
input sentence: Go.
decoded sentence:  Pars !

1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 29ms/step
-
input sentence: Go.
decoded sentence:  Pars !

1/1              0s 30ms/step
1/1              0s 55ms/step
1/1              0s 30ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 23ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
-
input sentence: Hi.
decoded sentence:  Salut !

1/1              0s 31ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
1/1              0s 28ms/step
```

```
1/1              0s 36ms/step
1/1              0s 33ms/step
1/1              0s 32ms/step
1/1              0s 29ms/step
1/1              0s 25ms/step
-
input sentence: Hi.
decoded sentence:  Salut !

1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
1/1              0s 31ms/step
1/1              0s 43ms/step
1/1              0s 36ms/step
-
input sentence: Run!
decoded sentence:  Cours !

1/1              0s 68ms/step
1/1              0s 34ms/step
1/1              0s 30ms/step
1/1              0s 31ms/step
1/1              0s 27ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
-
input sentence: Run!
decoded sentence:  Cours !

1/1              0s 24ms/step
1/1              0s 26ms/step
1/1              0s 42ms/step
1/1              0s 38ms/step
1/1              0s 28ms/step
1/1              0s 27ms/step
1/1              0s 31ms/step
1/1              0s 26ms/step
1/1              0s 25ms/step
-
input sentence: Run!
decoded sentence:  Cours !
```

```
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 29ms/step
1/1              0s 54ms/step
1/1              0s 35ms/step
1/1              0s 28ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
1/1              0s 26ms/step
-
input sentence: Run!
decoded sentence:  Cours !

1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 23ms/step
1/1              0s 22ms/step
1/1              0s 22ms/step
1/1              0s 25ms/step
1/1              0s 50ms/step
1/1              0s 32ms/step
1/1              0s 31ms/step
-
input sentence: Run!
decoded sentence:  Cours !

1/1              0s 32ms/step
1/1              0s 31ms/step
1/1              0s 27ms/step
1/1              0s 23ms/step
1/1              0s 23ms/step
1/1              0s 39ms/step
1/1              0s 32ms/step
1/1              0s 31ms/step
1/1              0s 27ms/step
-
input sentence: Run!
decoded sentence:  Cours !

1/1              0s 30ms/step
1/1              0s 29ms/step
1/1              0s 28ms/step
1/1              0s 31ms/step
1/1              0s 28ms/step
1/1              0s 29ms/step
1/1              0s 56ms/step
1/1              0s 59ms/step
1/1              0s 28ms/step
```

```
-
input sentence: Run!
decoded sentence:  Cours !

1/1             0s 26ms/step
1/1             0s 24ms/step
1/1             0s 25ms/step
1/1             0s 34ms/step
1/1             0s 34ms/step
1/1             0s 59ms/step
1/1             0s 34ms/step
1/1             0s 27ms/step
1/1             0s 28ms/step
-
input sentence: Run!
decoded sentence:  Cours !

1/1             0s 26ms/step
1/1             0s 32ms/step
1/1             0s 31ms/step
1/1             0s 29ms/step
1/1             0s 29ms/step
1/1             0s 33ms/step
1/1             0s 53ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1             0s 29ms/step
1/1             0s 28ms/step
1/1             0s 30ms/step
1/1             0s 29ms/step
1/1             0s 27ms/step
1/1             0s 51ms/step
1/1             0s 39ms/step
1/1             0s 29ms/step
1/1             0s 27ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 25ms/step
1/1             0s 26ms/step
1/1             0s 50ms/step
```

```
1/1              0s 36ms/step
1/1              0s 24ms/step
1/1              0s 28ms/step
1/1              0s 28ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1              0s 26ms/step
1/1              0s 26ms/step
1/1              0s 27ms/step
1/1              0s 43ms/step
1/1              0s 35ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 21ms/step
1/1              0s 25ms/step
1/1              0s 45ms/step
1/1              0s 39ms/step
1/1              0s 34ms/step
1/1              0s 27ms/step
1/1              0s 27ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1              0s 25ms/step
1/1              0s 23ms/step
1/1              0s 24ms/step
1/1              0s 29ms/step
1/1              0s 38ms/step
1/1              0s 23ms/step
1/1              0s 27ms/step
1/1              0s 20ms/step
1/1              0s 23ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1              0s 29ms/step
```

```
1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 26ms/step
1/1              0s 49ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1              0s 23ms/step
1/1              0s 22ms/step
1/1              0s 23ms/step
1/1              0s 26ms/step
1/1              0s 41ms/step
1/1              0s 42ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 23ms/step
-
input sentence: Run.
decoded sentence:  Fuyez !

1/1              0s 23ms/step
1/1              0s 26ms/step
1/1              0s 23ms/step
1/1              0s 28ms/step
1/1              0s 37ms/step
1/1              0s 28ms/step
1/1              0s 26ms/step
1/1              0s 28ms/step
1/1              0s 25ms/step
1/1              0s 30ms/step
1/1              0s 28ms/step
1/1              0s 28ms/step
1/1              0s 43ms/step
1/1              0s 35ms/step
1/1              0s 27ms/step
1/1              0s 31ms/step
1/1              0s 30ms/step
1/1              0s 28ms/step
1/1              0s 32ms/step
-
input sentence: Who?
decoded sentence:  Qui sommes-lous ?
```

```
1/1              0s 33ms/step
1/1              0s 29ms/step
1/1              0s 36ms/step
1/1              0s 45ms/step
1/1              0s 21ms/step
1/1              0s 22ms/step
-
input sentence: Wow!
decoded sentence:  Ça !

1/1              0s 25ms/step
1/1              0s 23ms/step
1/1              0s 21ms/step
1/1              0s 23ms/step
1/1              0s 25ms/step
1/1              0s 37ms/step
-
input sentence: Wow!
decoded sentence:  Ça !

1/1              0s 36ms/step
1/1              0s 28ms/step
1/1              0s 31ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 28ms/step
-
input sentence: Wow!
decoded sentence:  Ça !

1/1              0s 25ms/step
1/1              0s 25ms/step
1/1              0s 35ms/step
1/1              0s 43ms/step
1/1              0s 27ms/step
1/1              0s 32ms/step
1/1              0s 30ms/step
1/1              0s 34ms/step
1/1              0s 30ms/step
1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
-
input sentence: Duck!
decoded sentence:  Casse-toi !

1/1              0s 53ms/step
```

```
1/1              0s 48ms/step
1/1              0s 21ms/step
1/1              0s 22ms/step
1/1              0s 29ms/step
1/1              0s 30ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 55ms/step
1/1              0s 52ms/step
1/1              0s 30ms/step
1/1              0s 29ms/step
1/1              0s 26ms/step
-
input sentence: Duck!
decoded sentence:  Casse-toi !

1/1              0s 28ms/step
1/1              0s 27ms/step
1/1              0s 21ms/step
1/1              0s 57ms/step
1/1              0s 40ms/step
1/1              0s 26ms/step
1/1              0s 26ms/step
1/1              0s 30ms/step
1/1              0s 30ms/step
1/1              0s 27ms/step
1/1              0s 27ms/step
1/1              0s 58ms/step
1/1              0s 31ms/step
-
input sentence: Duck!
decoded sentence:  Casse-toi !

1/1              0s 21ms/step
1/1              0s 23ms/step
1/1              0s 36ms/step
1/1              0s 31ms/step
1/1              0s 25ms/step
1/1              0s 35ms/step
1/1              0s 30ms/step
1/1              0s 39ms/step
1/1              0s 28ms/step
1/1              0s 26ms/step
1/1              0s 24ms/step
-
input sentence: Fire!
decoded sentence:  Colment !
```

```
1/1             0s 24ms/step
1/1             0s 30ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 61ms/step
1/1             0s 44ms/step
1/1             0s 23ms/step
1/1             0s 22ms/step
1/1             0s 24ms/step
1/1             0s 24ms/step
1/1             0s 22ms/step
1/1             0s 30ms/step
1/1             0s 28ms/step
1/1             0s 26ms/step
1/1             0s 33ms/step
1/1             0s 39ms/step
1/1             0s 29ms/step
1/1             0s 30ms/step
1/1             0s 29ms/step
-
input sentence: Help!
decoded sentence:  Firez le cheacie.

1/1             0s 27ms/step
1/1             0s 26ms/step
1/1             0s 48ms/step
1/1             0s 47ms/step
1/1             0s 22ms/step
1/1             0s 21ms/step
1/1             0s 23ms/step
1/1             0s 21ms/step
1/1             0s 27ms/step
1/1             0s 23ms/step
1/1             0s 26ms/step
-
input sentence: Hide.
decoded sentence:  Attendez.

1/1             0s 65ms/step
1/1             0s 40ms/step
1/1             0s 23ms/step
1/1             0s 22ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 25ms/step
1/1             0s 24ms/step
1/1             0s 47ms/step
1/1             0s 27ms/step
```

```
1/1             0s 27ms/step
-
input sentence: Hide.
decoded sentence:  Attendez.

1/1             0s 28ms/step
1/1             0s 31ms/step
1/1             0s 31ms/step
1/1             0s 30ms/step
1/1             0s 29ms/step
1/1             0s 26ms/step
1/1             0s 23ms/step
1/1             0s 36ms/step
1/1             0s 38ms/step
1/1             0s 21ms/step
-
input sentence: Jump!
decoded sentence:  Sautes !

1/1             0s 24ms/step
1/1             0s 31ms/step
1/1             0s 25ms/step
1/1             0s 29ms/step
1/1             0s 26ms/step
1/1             0s 50ms/step
1/1             0s 41ms/step
1/1             0s 26ms/step
-
input sentence: Jump.
decoded sentence:  Saute.

1/1             0s 29ms/step
1/1             0s 24ms/step
1/1             0s 24ms/step
1/1             0s 28ms/step
1/1             0s 29ms/step
1/1             0s 56ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 22ms/step
1/1             0s 22ms/step
1/1             0s 24ms/step
1/1             0s 31ms/step
-
input sentence: Stop!
decoded sentence:  Attendez !

1/1             0s 25ms/step
```

```
1/1              0s 40ms/step
1/1              0s 33ms/step
1/1              0s 26ms/step
1/1              0s 27ms/step
1/1              0s 28ms/step
1/1              0s 26ms/step
1/1              0s 25ms/step
1/1              0s 39ms/step
1/1              0s 41ms/step
1/1              0s 28ms/step
1/1              0s 27ms/step
-
input sentence: Stop!
decoded sentence:  Attendez !

1/1              0s 25ms/step
1/1              0s 26ms/step
1/1              0s 24ms/step
1/1              0s 39ms/step
1/1              0s 35ms/step
1/1              0s 29ms/step
1/1              0s 27ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 31ms/step
1/1              0s 36ms/step
-
input sentence: Stop!
decoded sentence:  Attendez !

1/1              0s 65ms/step
1/1              0s 27ms/step
1/1              0s 37ms/step
1/1              0s 33ms/step
1/1              0s 29ms/step
1/1              0s 30ms/step
1/1              0s 43ms/step
1/1              0s 38ms/step
1/1              0s 27ms/step
1/1              0s 27ms/step
1/1              0s 26ms/step
-
input sentence: Wait!
decoded sentence:  Attendez.

1/1              0s 29ms/step
1/1              0s 24ms/step
```

```
1/1              0s 47ms/step
1/1              0s 37ms/step
1/1              0s 24ms/step
1/1              0s 26ms/step
1/1              0s 27ms/step
1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 26ms/step
-
input sentence: Wait!
decoded sentence:  Attendez.

1/1              0s 62ms/step
1/1              0s 23ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 30ms/step
1/1              0s 21ms/step
1/1              0s 25ms/step
1/1              0s 28ms/step
1/1              0s 38ms/step
1/1              0s 33ms/step
1/1              0s 22ms/step
-
input sentence: Wait!
decoded sentence:  Attendez.

1/1              0s 21ms/step
1/1              0s 23ms/step
1/1              0s 25ms/step
1/1              0s 23ms/step
1/1              0s 22ms/step
1/1              0s 24ms/step
1/1              0s 50ms/step
1/1              0s 41ms/step
1/1              0s 24ms/step
1/1              0s 28ms/step
1/1              0s 25ms/step
-
input sentence: Wait.
decoded sentence:  Attendez.

1/1              0s 23ms/step
1/1              0s 50ms/step
1/1              0s 31ms/step
1/1              0s 30ms/step
1/1              0s 24ms/step
```

```
1/1              0s 26ms/step
1/1              0s 28ms/step
1/1              0s 35ms/step
1/1              0s 30ms/step
1/1              0s 68ms/step
1/1              0s 37ms/step
-
input sentence: Wait.
decoded sentence:  Attendez.

1/1              0s 29ms/step
1/1              0s 32ms/step
1/1              0s 33ms/step
1/1              0s 31ms/step
1/1              0s 27ms/step
1/1              0s 40ms/step
1/1              0s 45ms/step
1/1              0s 26ms/step
1/1              0s 26ms/step
1/1              0s 29ms/step
1/1              0s 27ms/step
-
input sentence: Wait.
decoded sentence:  Attendez.

1/1              0s 25ms/step
1/1              0s 29ms/step
1/1              0s 43ms/step
1/1              0s 32ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 31ms/step
1/1              0s 28ms/step
1/1              0s 30ms/step
1/1              0s 50ms/step
1/1              0s 39ms/step
-
input sentence: Wait.
decoded sentence:  Attendez.

1/1              0s 23ms/step
1/1              0s 26ms/step
1/1              0s 26ms/step
1/1              0s 29ms/step
1/1              0s 35ms/step
1/1              0s 31ms/step
1/1              0s 50ms/step
1/1              0s 43ms/step
```

```
1/1             0s 27ms/step
1/1             0s 33ms/step
-
input sentence: Begin.
decoded sentence:  On fal !

1/1             0s 30ms/step
1/1             0s 27ms/step
1/1             0s 23ms/step
1/1             0s 23ms/step
1/1             0s 23ms/step
1/1             0s 36ms/step
1/1             0s 27ms/step
1/1             0s 27ms/step
1/1             0s 26ms/step
1/1             0s 31ms/step
-
input sentence: Begin.
decoded sentence:  On fal !

1/1             0s 26ms/step
1/1             0s 31ms/step
1/1             0s 34ms/step
1/1             0s 69ms/step
1/1             0s 52ms/step
1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 34ms/step
1/1             0s 31ms/step
1/1             0s 37ms/step
1/1             0s 31ms/step
-
input sentence: Go on.
decoded sentence:  Allez-y !

1/1             0s 39ms/step
1/1             0s 31ms/step
1/1             0s 27ms/step
1/1             0s 31ms/step
1/1             0s 30ms/step
1/1             0s 29ms/step
1/1             0s 39ms/step
1/1             0s 41ms/step
1/1             0s 33ms/step
1/1             0s 29ms/step
1/1             0s 24ms/step
-
input sentence: Go on.
```

```
decoded sentence:  Allez-y !

1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 31ms/step
1/1              0s 46ms/step
1/1              0s 34ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 26ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
-
input sentence: Go on.
decoded sentence:  Allez-y !

1/1              0s 42ms/step
1/1              0s 52ms/step
1/1              0s 28ms/step
1/1              0s 30ms/step
1/1              0s 31ms/step
1/1              0s 29ms/step
1/1              0s 45ms/step
1/1              0s 32ms/step
-
input sentence: Hello!
decoded sentence:  Salut.

1/1              0s 31ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 25ms/step
1/1              0s 25ms/step
1/1              0s 38ms/step
1/1              0s 39ms/step
-
input sentence: Hello!
decoded sentence:  Salut.

1/1              0s 40ms/step
1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 31ms/step
1/1              0s 28ms/step
1/1              0s 45ms/step
```

```
1/1             0s 44ms/step
-
input sentence: Hello.
decoded sentence:  Salut.

1/1             0s 24ms/step
1/1             0s 28ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 38ms/step
1/1             0s 39ms/step
1/1             0s 25ms/step
-
input sentence: Hello.
decoded sentence:  Salut.

1/1             0s 28ms/step
1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 49ms/step
1/1             0s 25ms/step
1/1             0s 27ms/step
-
input sentence: Hello.
decoded sentence:  Salut.

1/1             0s 26ms/step
1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 45ms/step
1/1             0s 47ms/step
1/1             0s 33ms/step
1/1             0s 31ms/step
1/1             0s 28ms/step
-
input sentence: Hello.
decoded sentence:  Salut.

1/1             0s 27ms/step
1/1             0s 73ms/step
1/1             0s 51ms/step
1/1             0s 33ms/step
1/1             0s 32ms/step
1/1             0s 32ms/step
1/1             0s 27ms/step
```

```
1/1             0s 34ms/step
1/1             0s 59ms/step
1/1             0s 32ms/step
1/1             0s 25ms/step
1/1             0s 26ms/step
1/1             0s 27ms/step
-
input sentence: I see.
decoded sentence:  Je tempreu.

1/1             0s 29ms/step
1/1             0s 26ms/step
1/1             0s 47ms/step
1/1             0s 55ms/step
1/1             0s 30ms/step
1/1             0s 27ms/step
1/1             0s 25ms/step
1/1             0s 31ms/step
1/1             0s 31ms/step
1/1             0s 33ms/step
1/1             0s 69ms/step
1/1             0s 32ms/step
1/1             0s 30ms/step
-
input sentence: I see.
decoded sentence:  Je tempreu.

1/1             0s 29ms/step
1/1             0s 30ms/step
1/1             0s 27ms/step
1/1             0s 32ms/step
1/1             0s 45ms/step
1/1             0s 62ms/step
1/1             0s 37ms/step
1/1             0s 30ms/step
1/1             0s 44ms/step
1/1             0s 32ms/step
1/1             0s 32ms/step
1/1             0s 48ms/step
1/1             0s 52ms/step
1/1             0s 27ms/step
1/1             0s 28ms/step
1/1             0s 27ms/step
-
input sentence: I try.
decoded sentence:  Je me suisais.

1/1             0s 27ms/step
```

```
1/1             0s 27ms/step
1/1             0s 26ms/step
1/1             0s 61ms/step
1/1             0s 38ms/step
1/1             0s 37ms/step
1/1             0s 36ms/step
1/1             0s 27ms/step
1/1             0s 29ms/step
1/1             0s 32ms/step
1/1             0s 79ms/step
1/1             0s 30ms/step
1/1             0s 26ms/step
1/1             0s 30ms/step
1/1             0s 34ms/step
1/1             0s 35ms/step
1/1             0s 33ms/step
1/1             0s 30ms/step
-
input sentence: I won!
decoded sentence:  Je l'ai espotie.

1/1             0s 58ms/step
1/1             0s 30ms/step
1/1             0s 26ms/step
1/1             0s 25ms/step
1/1             0s 23ms/step
1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 41ms/step
1/1             0s 37ms/step
1/1             0s 32ms/step
1/1             0s 31ms/step
1/1             0s 30ms/step
1/1             0s 33ms/step
1/1             0s 31ms/step
1/1             0s 40ms/step
1/1             0s 32ms/step
1/1             0s 28ms/step
1/1             0s 30ms/step
-
input sentence: I won!
decoded sentence:  Je l'ai espotie.

1/1             0s 32ms/step
1/1             0s 36ms/step
1/1             0s 28ms/step
1/1             0s 70ms/step
1/1             0s 27ms/step
```

```
1/1              0s 26ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
1/1              0s 27ms/step
1/1              0s 24ms/step
1/1              0s 38ms/step
-
input sentence: I won.
decoded sentence:  Je perdi.

1/1              0s 31ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 28ms/step
1/1              0s 27ms/step
1/1              0s 32ms/step
1/1              0s 47ms/step
1/1              0s 47ms/step
1/1              0s 28ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 30ms/step
1/1              0s 29ms/step
-
input sentence: Oh no!
decoded sentence:  Chonez-vous !

1/1              0s 40ms/step
1/1              0s 41ms/step
1/1              0s 41ms/step
1/1              0s 27ms/step
1/1              0s 26ms/step
1/1              0s 33ms/step
1/1              0s 32ms/step
1/1              0s 29ms/step
1/1              0s 53ms/step
1/1              0s 36ms/step
1/1              0s 25ms/step
1/1              0s 23ms/step
1/1              0s 33ms/step
1/1              0s 31ms/step
1/1              0s 34ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !
```

```
1/1             0s 58ms/step
1/1             0s 47ms/step
1/1             0s 30ms/step
1/1             0s 26ms/step
1/1             0s 27ms/step
1/1             0s 31ms/step
1/1             0s 31ms/step
1/1             0s 53ms/step
1/1             0s 33ms/step
1/1             0s 27ms/step
1/1             0s 28ms/step
1/1             0s 27ms/step
1/1             0s 43ms/step
1/1             0s 33ms/step
1/1             0s 49ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 31ms/step
1/1             0s 36ms/step
1/1             0s 29ms/step
1/1             0s 29ms/step
1/1             0s 29ms/step
1/1             0s 34ms/step
1/1             0s 35ms/step
1/1             0s 65ms/step
1/1             0s 35ms/step
1/1             0s 32ms/step
1/1             0s 28ms/step
1/1             0s 27ms/step
1/1             0s 30ms/step
1/1             0s 32ms/step
1/1             0s 45ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 48ms/step
1/1             0s 27ms/step
1/1             0s 27ms/step
1/1             0s 30ms/step
1/1             0s 37ms/step
1/1             0s 31ms/step
1/1             0s 44ms/step
1/1             0s 27ms/step
1/1             0s 27ms/step
1/1             0s 33ms/step
```

```
1/1             0s 28ms/step
1/1             0s 25ms/step
1/1             0s 54ms/step
1/1             0s 38ms/step
1/1             0s 30ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 23ms/step
1/1             0s 27ms/step
1/1             0s 26ms/step
1/1             0s 37ms/step
1/1             0s 26ms/step
1/1             0s 39ms/step
1/1             0s 36ms/step
1/1             0s 20ms/step
1/1             0s 25ms/step
1/1             0s 23ms/step
1/1             0s 23ms/step
1/1             0s 27ms/step
1/1             0s 24ms/step
1/1             0s 26ms/step
1/1             0s 27ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 53ms/step
1/1             0s 42ms/step
1/1             0s 30ms/step
1/1             0s 23ms/step
1/1             0s 31ms/step
1/1             0s 35ms/step
1/1             0s 28ms/step
1/1             0s 48ms/step
1/1             0s 43ms/step
1/1             0s 31ms/step
1/1             0s 32ms/step
1/1             0s 25ms/step
1/1             0s 24ms/step
1/1             0s 27ms/step
1/1             0s 40ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 43ms/step
```

```
1/1             0s 42ms/step
1/1             0s 29ms/step
1/1             0s 34ms/step
1/1             0s 32ms/step
1/1             0s 29ms/step
1/1             0s 29ms/step
1/1             0s 49ms/step
1/1             0s 45ms/step
1/1             0s 37ms/step
1/1             0s 31ms/step
1/1             0s 31ms/step
1/1             0s 31ms/step
1/1             0s 36ms/step
1/1             0s 51ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 39ms/step
1/1             0s 28ms/step
1/1             0s 28ms/step
1/1             0s 28ms/step
1/1             0s 31ms/step
1/1             0s 29ms/step
1/1             0s 48ms/step
1/1             0s 30ms/step
1/1             0s 30ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
1/1             0s 25ms/step
1/1             0s 53ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 24ms/step
1/1             0s 35ms/step
1/1             0s 26ms/step
1/1             0s 30ms/step
1/1             0s 29ms/step
1/1             0s 64ms/step
1/1             0s 38ms/step
1/1             0s 34ms/step
1/1             0s 36ms/step
1/1             0s 30ms/step
1/1             0s 24ms/step
```

```
1/1              0s 32ms/step
1/1              0s 53ms/step
1/1              0s 26ms/step
1/1              0s 32ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1              0s 30ms/step
1/1              0s 26ms/step
1/1              0s 25ms/step
1/1              0s 26ms/step
1/1              0s 38ms/step
1/1              0s 40ms/step
1/1              0s 30ms/step
1/1              0s 29ms/step
1/1              0s 27ms/step
1/1              0s 31ms/step
1/1              0s 33ms/step
1/1              0s 72ms/step
1/1              0s 26ms/step
1/1              0s 31ms/step
1/1              0s 32ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1              0s 28ms/step
1/1              0s 31ms/step
1/1              0s 45ms/step
1/1              0s 48ms/step
1/1              0s 39ms/step
1/1              0s 24ms/step
1/1              0s 27ms/step
1/1              0s 26ms/step
1/1              0s 26ms/step
1/1              0s 41ms/step
1/1              0s 44ms/step
1/1              0s 33ms/step
1/1              0s 25ms/step
1/1              0s 23ms/step
1/1              0s 24ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1              0s 21ms/step
1/1              0s 32ms/step
```

```
1/1             0s 43ms/step
1/1             0s 38ms/step
1/1             0s 25ms/step
1/1             0s 24ms/step
1/1             0s 24ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
1/1             0s 26ms/step
1/1             0s 36ms/step
1/1             0s 54ms/step
1/1             0s 42ms/step
1/1             0s 28ms/step
1/1             0s 29ms/step
-
input sentence: Relax.
decoded sentence:  Détends-toi !

1/1             0s 28ms/step
1/1             0s 27ms/step
1/1             0s 43ms/step
1/1             0s 48ms/step
1/1             0s 39ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
1/1             0s 24ms/step
1/1             0s 60ms/step
-
input sentence: Smile.
decoded sentence:  Souriez !

1/1             0s 28ms/step
1/1             0s 30ms/step
1/1             0s 29ms/step
1/1             0s 33ms/step
1/1             0s 31ms/step
1/1             0s 30ms/step
1/1             0s 40ms/step
1/1             0s 30ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
1/1             0s 29ms/step
-
input sentence: Smile.
decoded sentence:  Souriez !

1/1             0s 25ms/step
```

```
1/1             0s 30ms/step
1/1             0s 30ms/step
1/1             0s 50ms/step
1/1             0s 27ms/step
1/1             0s 30ms/step
1/1             0s 23ms/step
1/1             0s 22ms/step
1/1             0s 29ms/step
1/1             0s 27ms/step
1/1             0s 28ms/step
-
input sentence: Smile.
decoded sentence:  Souriez !

1/1             0s 43ms/step
1/1             0s 32ms/step
1/1             0s 27ms/step
1/1             0s 31ms/step
1/1             0s 33ms/step
1/1             0s 31ms/step
1/1             0s 27ms/step
1/1             0s 43ms/step
1/1             0s 33ms/step
1/1             0s 28ms/step
1/1             0s 26ms/step
-
input sentence: Sorry?
decoded sentence:  Pard-il ?

1/1             0s 26ms/step
1/1             0s 33ms/step
1/1             0s 27ms/step
1/1             0s 32ms/step
1/1             0s 44ms/step
1/1             0s 43ms/step
1/1             0s 34ms/step
1/1             0s 34ms/step
1/1             0s 29ms/step
1/1             0s 26ms/step
1/1             0s 42ms/step
1/1             0s 38ms/step
1/1             0s 28ms/step
1/1             0s 28ms/step
-
input sentence: Attack!
decoded sentence:  Fillez-les !

1/1             0s 27ms/step
```

```
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 51ms/step
1/1              0s 39ms/step
1/1              0s 27ms/step
1/1              0s 24ms/step
1/1              0s 28ms/step
1/1              0s 27ms/step
1/1              0s 38ms/step
1/1              0s 24ms/step
1/1              0s 19ms/step
1/1              0s 19ms/step
1/1              0s 22ms/step
-
input sentence: Attack!
decoded sentence:  Fillez-les !

1/1              0s 29ms/step
1/1              0s 30ms/step
1/1              0s 29ms/step
1/1              0s 30ms/step
1/1              0s 58ms/step
1/1              0s 38ms/step
1/1              0s 31ms/step
1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
1/1              0s 25ms/step
1/1              0s 46ms/step
1/1              0s 28ms/step
-
input sentence: Attack!
decoded sentence:  Fillez-les !

1/1              0s 27ms/step
1/1              0s 26ms/step
1/1              0s 34ms/step
1/1              0s 30ms/step
1/1              0s 33ms/step
1/1              0s 32ms/step
1/1              0s 22ms/step
1/1              0s 42ms/step
1/1              0s 32ms/step
1/1              0s 29ms/step
1/1              0s 32ms/step
1/1              0s 32ms/step
1/1              0s 29ms/step
```

```
1/1             0s 24ms/step
-
input sentence: Attack!
decoded sentence:  Fillez-les !

1/1             0s 69ms/step
1/1             0s 27ms/step
1/1             0s 25ms/step
1/1             0s 24ms/step
1/1             0s 28ms/step
1/1             0s 25ms/step
1/1             0s 24ms/step
1/1             0s 41ms/step
1/1             0s 26ms/step
1/1             0s 26ms/step
1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 31ms/step
1/1             0s 28ms/step
-
input sentence: Buy it.
decoded sentence:  Achetez-le !

1/1             0s 26ms/step
1/1             0s 53ms/step
1/1             0s 32ms/step
1/1             0s 24ms/step
1/1             0s 25ms/step
1/1             0s 27ms/step
1/1             0s 26ms/step
1/1             0s 33ms/step
1/1             0s 44ms/step
1/1             0s 41ms/step
1/1             0s 25ms/step
1/1             0s 30ms/step
1/1             0s 31ms/step
1/1             0s 38ms/step
-
input sentence: Buy it.
decoded sentence:  Achetez-le !

1/1             0s 28ms/step
1/1             0s 46ms/step
1/1             0s 45ms/step
1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 31ms/step
1/1             0s 30ms/step
```

```
1/1              0s 27ms/step
1/1              0s 37ms/step
1/1              0s 29ms/step
1/1              0s 23ms/step
1/1              0s 27ms/step
1/1              0s 31ms/step
1/1              0s 23ms/step
-
input sentence: Buy it.
decoded sentence:  Achetez-le !

1/1              0s 22ms/step
1/1              0s 23ms/step
1/1              0s 23ms/step
1/1              0s 51ms/step
1/1              0s 32ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 26ms/step
1/1              0s 28ms/step
1/1              0s 36ms/step
1/1              0s 53ms/step
1/1              0s 43ms/step
1/1              0s 35ms/step
1/1              0s 32ms/step
-
input sentence: Buy it.
decoded sentence:  Achetez-le !

1/1              0s 29ms/step
1/1              0s 30ms/step
1/1              0s 65ms/step
1/1              0s 32ms/step
1/1              0s 29ms/step
1/1              0s 27ms/step
1/1              0s 21ms/step
1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 23ms/step
-
input sentence: Cheers!
decoded sentence:  Centie !

1/1              0s 50ms/step
1/1              0s 31ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 31ms/step
```

```
1/1              0s 26ms/step
1/1              0s 25ms/step
1/1              0s 25ms/step
1/1              0s 62ms/step
1/1              0s 51ms/step
-
input sentence: Cheers!
decoded sentence:  Centie !

1/1              0s 34ms/step
1/1              0s 31ms/step
1/1              0s 23ms/step
1/1              0s 24ms/step
1/1              0s 30ms/step
1/1              0s 58ms/step
1/1              0s 24ms/step
1/1              0s 43ms/step
1/1              0s 25ms/step
1/1              0s 25ms/step
-
input sentence: Cheers!
decoded sentence:  Centie !

1/1              0s 28ms/step
1/1              0s 28ms/step
1/1              0s 29ms/step
1/1              0s 41ms/step
1/1              0s 43ms/step
1/1              0s 23ms/step
1/1              0s 33ms/step
1/1              0s 39ms/step
1/1              0s 34ms/step
1/1              0s 35ms/step
-
input sentence: Cheers!
decoded sentence:  Centie !

1/1              0s 29ms/step
1/1              0s 47ms/step
1/1              0s 26ms/step
1/1              0s 26ms/step
1/1              0s 33ms/step
1/1              0s 31ms/step
1/1              0s 30ms/step
1/1              0s 48ms/step
1/1              0s 45ms/step
1/1              0s 27ms/step
1/1              0s 25ms/step
```

```
-
input sentence: Eat it.
decoded sentence:  Mange-le.

1/1             0s 22ms/step
1/1             0s 24ms/step
1/1             0s 27ms/step
1/1             0s 62ms/step
1/1             0s 22ms/step
1/1             0s 23ms/step
1/1             0s 22ms/step
1/1             0s 24ms/step
1/1             0s 27ms/step
1/1             0s 31ms/step
1/1             0s 64ms/step
-
input sentence: Eat it.
decoded sentence:  Mange-le.

1/1             0s 28ms/step
1/1             0s 30ms/step
1/1             0s 25ms/step
1/1             0s 31ms/step
1/1             0s 28ms/step
1/1             0s 49ms/step
1/1             0s 23ms/step
1/1             0s 22ms/step
1/1             0s 26ms/step
1/1             0s 27ms/step
1/1             0s 29ms/step
1/1             0s 27ms/step
1/1             0s 26ms/step
1/1             0s 38ms/step
1/1             0s 31ms/step
1/1             0s 23ms/step
1/1             0s 24ms/step
1/1             0s 23ms/step
-
input sentence: Exhale.
decoded sentence:  Chance avec moi.

1/1             0s 25ms/step
1/1             0s 25ms/step
1/1             0s 26ms/step
1/1             0s 46ms/step
1/1             0s 45ms/step
1/1             0s 31ms/step
1/1             0s 24ms/step
```

```
1/1              0s 22ms/step
1/1              0s 24ms/step
1/1              0s 24ms/step
-
input sentence: Get up.
decoded sentence:  Dégage !

1/1              0s 27ms/step
1/1              0s 51ms/step
1/1              0s 39ms/step
1/1              0s 28ms/step
1/1              0s 21ms/step
1/1              0s 26ms/step
1/1              0s 28ms/step
1/1              0s 26ms/step
1/1              0s 24ms/step
1/1              0s 50ms/step
-
input sentence: Get up.
decoded sentence:  Dégage !

1/1              0s 32ms/step
1/1              0s 38ms/step
1/1              0s 33ms/step
1/1              0s 25ms/step
1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 43ms/step
1/1              0s 35ms/step
1/1              0s 31ms/step
1/1              0s 32ms/step
-
input sentence: Get up.
decoded sentence:  Dégage !

1/1              0s 33ms/step
1/1              0s 30ms/step
1/1              0s 38ms/step
1/1              0s 69ms/step
1/1              0s 38ms/step
1/1              0s 31ms/step
1/1              0s 30ms/step
1/1              0s 29ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 27ms/step
-
input sentence: Go now.
```

```
decoded sentence:  Plendiz !

1/1              0s 65ms/step
1/1              0s 29ms/step
1/1              0s 29ms/step
1/1              0s 25ms/step
1/1              0s 27ms/step
1/1              0s 30ms/step
1/1              0s 28ms/step
1/1              0s 28ms/step
1/1              0s 34ms/step
1/1              0s 39ms/step
1/1              0s 32ms/step
-
input sentence: Go now.
decoded sentence:  Plendiz !

1/1              0s 26ms/step
1/1              0s 32ms/step
1/1              0s 27ms/step
1/1              0s 24ms/step
1/1              0s 23ms/step
1/1              0s 50ms/step
1/1              0s 33ms/step
1/1              0s 26ms/step
1/1              0s 27ms/step
1/1              0s 28ms/step
1/1              0s 28ms/step
-
input sentence: Go now.
decoded sentence:  Plendiz !
```

4 AlexNet

[19]:
```
import tensorflow.keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import␣
↪Dense,Activation,Dropout,Flatten,Conv2D,MaxPooling2D,BatchNormalization

#input 227*227 rgb image

#create sequential model
model=Sequential([
    #1st conv layer
   ␣
↪Conv2D(filters=96,input_shape=(227,227,3),kernel_size=(11,11),strides=(4,4),padding='valid'
```

```
    MaxPooling2D(pool_size=(3,3),strides=(2,2),padding='valid'),
    BatchNormalization() ,

    #2nd conv layer
   ␣
↪Conv2D(filters=256,kernel_size=(5,5),strides=(1,1),padding='same'),Activation('relu'),
    MaxPooling2D(pool_size=(3,3),strides=(2,2),padding='valid'),
    BatchNormalization() ,

    #3rd conv layer
   ␣
↪Conv2D(filters=384,kernel_size=(3,3),strides=(1,1),padding='same'),Activation('relu'),
    BatchNormalization() ,

    #4th conv layer
   ␣
↪Conv2D(filters=384,kernel_size=(3,3),strides=(1,1),padding='same'),Activation('relu'),
    BatchNormalization() ,

    #5th conv layer
   ␣
↪Conv2D(filters=256,kernel_size=(3,3),strides=(1,1),padding='same'),Activation('relu'),
    MaxPooling2D(pool_size=(3,3),strides=(2,2),padding='valid'),
    BatchNormalization() ,

    #6th --fully connected layer
      #pass to dense layer
    Flatten(),
    Dense(4096,input_shape=(224*224*3,)),Activation('relu'),
    Dropout(0.5),
    BatchNormalization() ,

    #7th --fully connected layer
      #pass to dense layer
    Dense(4096),Activation('relu'),
    Dropout(0.5),
    BatchNormalization() ,

    #8th --fully connected layer
      #pass to dense layer
    Dense(1000),Activation('relu'),
    Dropout(0.5),
    BatchNormalization() ,

    #output layer
    Dense(17),  # Assuming 17 classes
    Activation('softmax')
```

```
])

model.summary()
```

```
2024-12-28 22:40:01.276354: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
150994944 exceeds 10% of free system memory.
2024-12-28 22:40:01.419783: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
150994944 exceeds 10% of free system memory.
2024-12-28 22:40:01.479969: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
150994944 exceeds 10% of free system memory.
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_25 (Conv2D)	(None, 55, 55, 96)	34,944
activation_37 (Activation)	(None, 55, 55, 96)	0
max_pooling2d_15 (MaxPooling2D)	(None, 27, 27, 96)	0
batch_normalization_33 (BatchNormalization)	(None, 27, 27, 96)	384
conv2d_26 (Conv2D)	(None, 27, 27, 256)	614,656
activation_38 (Activation)	(None, 27, 27, 256)	0
max_pooling2d_16 (MaxPooling2D)	(None, 13, 13, 256)	0
batch_normalization_34 (BatchNormalization)	(None, 13, 13, 256)	1,024
conv2d_27 (Conv2D)	(None, 13, 13, 384)	885,120
activation_39 (Activation)	(None, 13, 13, 384)	0
batch_normalization_35 (BatchNormalization)	(None, 13, 13, 384)	1,536
conv2d_28 (Conv2D)	(None, 13, 13, 384)	1,327,488

activation_40 (Activation)	(None, 13, 13, 384)	0
batch_normalization_36 (BatchNormalization)	(None, 13, 13, 384)	1,536
conv2d_29 (Conv2D)	(None, 13, 13, 256)	884,992
activation_41 (Activation)	(None, 13, 13, 256)	0
max_pooling2d_17 (MaxPooling2D)	(None, 6, 6, 256)	0
batch_normalization_37 (BatchNormalization)	(None, 6, 6, 256)	1,024
flatten_4 (Flatten)	(None, 9216)	0
dense_13 (Dense)	(None, 4096)	37,752,832
activation_42 (Activation)	(None, 4096)	0
dropout_10 (Dropout)	(None, 4096)	0
batch_normalization_38 (BatchNormalization)	(None, 4096)	16,384
dense_14 (Dense)	(None, 4096)	16,781,312
activation_43 (Activation)	(None, 4096)	0
dropout_11 (Dropout)	(None, 4096)	0
batch_normalization_39 (BatchNormalization)	(None, 4096)	16,384
dense_15 (Dense)	(None, 1000)	4,097,000
activation_44 (Activation)	(None, 1000)	0
dropout_12 (Dropout)	(None, 1000)	0
batch_normalization_40 (BatchNormalization)	(None, 1000)	4,000
dense_16 (Dense)	(None, 17)	17,017
activation_45 (Activation)	(None, 17)	0

```
Total params: 62,437,633 (238.18 MB)

Trainable params: 62,416,497 (238.10 MB)

Non-trainable params: 21,136 (82.56 KB)
```

5 VGG16

[2]:
```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,␣
↪Dropout

def build_vgg16(input_shape=(224, 224, 3), num_classes=1000):
    # Initialize the Sequential model
    model = Sequential()

    # Block 1
    model.add(Conv2D(64, (3, 3), activation='relu', padding='same',␣
↪input_shape=input_shape))
    model.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
    model.add(MaxPooling2D((2, 2), strides=(2, 2)))

    # Block 2
    model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
    model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
    model.add(MaxPooling2D((2, 2), strides=(2, 2)))

    # Block 3
    model.add(Conv2D(256, (3, 3), activation='relu', padding='same'))
    model.add(Conv2D(256, (3, 3), activation='relu', padding='same'))
    model.add(Conv2D(256, (3, 3), activation='relu', padding='same'))
    model.add(MaxPooling2D((2, 2), strides=(2, 2)))

    # Block 4
    model.add(Conv2D(512, (3, 3), activation='relu', padding='same'))
    model.add(Conv2D(512, (3, 3), activation='relu', padding='same'))
    model.add(Conv2D(512, (3, 3), activation='relu', padding='same'))
    model.add(MaxPooling2D((2, 2), strides=(2, 2)))

    # Block 5
    model.add(Conv2D(512, (3, 3), activation='relu', padding='same'))
    model.add(Conv2D(512, (3, 3), activation='relu', padding='same'))
```

```
    model.add(Conv2D(512, (3, 3), activation='relu', padding='same'))
    model.add(MaxPooling2D((2, 2), strides=(2, 2)))

    # Fully connected layers
    model.add(Flatten())
    model.add(Dense(4096, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(4096, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(num_classes, activation='softmax'))

    return model

# Build the model
vgg16_model = build_vgg16(input_shape=(224, 224, 3), num_classes=1000)

# Print the model summary
vgg16_model.summary()
```

```
2024-12-28 22:54:07.971355: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
2024-12-28 22:54:08.006170: E
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8454] Unable to register
cuDNN factory: Attempting to register factory for plugin cuDNN when one has
already been registered
2024-12-28 22:54:08.016328: E
external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1452] Unable to
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
2024-12-28 22:54:08.041880: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2024-12-28 22:54:09.803707: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not
find TensorRT
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2024-12-28 22:54:12.049068: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
```

```
411041792 exceeds 10% of free system memory.
2024-12-28 22:54:12.471310: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
411041792 exceeds 10% of free system memory.
2024-12-28 22:54:12.626461: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
411041792 exceeds 10% of free system memory.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 64)	1,792
conv2d_1 (Conv2D)	(None, 224, 224, 64)	36,928
max_pooling2d (MaxPooling2D)	(None, 112, 112, 64)	0
conv2d_2 (Conv2D)	(None, 112, 112, 128)	73,856
conv2d_3 (Conv2D)	(None, 112, 112, 128)	147,584
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 128)	0
conv2d_4 (Conv2D)	(None, 56, 56, 256)	295,168
conv2d_5 (Conv2D)	(None, 56, 56, 256)	590,080
conv2d_6 (Conv2D)	(None, 56, 56, 256)	590,080
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 256)	0
conv2d_7 (Conv2D)	(None, 28, 28, 512)	1,180,160
conv2d_8 (Conv2D)	(None, 28, 28, 512)	2,359,808
conv2d_9 (Conv2D)	(None, 28, 28, 512)	2,359,808
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 512)	0
conv2d_10 (Conv2D)	(None, 14, 14, 512)	2,359,808
conv2d_11 (Conv2D)	(None, 14, 14, 512)	2,359,808
conv2d_12 (Conv2D)	(None, 14, 14, 512)	2,359,808
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 512)	0

```
 flatten (Flatten)                    (None, 25088)                        0

 dense (Dense)                        (None, 4096)                102,764,544

 dropout (Dropout)                    (None, 4096)                        0

 dense_1 (Dense)                      (None, 4096)                16,781,312

 dropout_1 (Dropout)                  (None, 4096)                        0

 dense_2 (Dense)                      (None, 1000)                 4,097,000


Total params: 138,357,544 (527.79 MB)


Trainable params: 138,357,544 (527.79 MB)


Non-trainable params: 0 (0.00 B)
```

6 LSTM model for Univariate Time Series Forecasting

[16]:
```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM,Dense,Flatten

# preparing independent and dependent features
def prepare_data(timeseries_data, n_features):
        X, y =[],[]
        for i in range(len(timeseries_data)):
                # find the end of this pattern
                end_ix = i + n_features
                # check if we are beyond the sequence
                if end_ix > len(timeseries_data)-1:
                        break
                # gather input and output parts of the pattern
                seq_x, seq_y = timeseries_data[i:end_ix],␣
 ↪timeseries_data[end_ix]
                X.append(seq_x)
                y.append(seq_y)
        return np.array(X), np.array(y)

# define input sequence
timeseries_data = [110, 125, 133, 146, 158, 172, 187, 196, 210]
```

```
# choose a number of time steps
n_steps = 3
# split into samples
X, y = prepare_data(timeseries_data, n_steps)

print(X),print(y)
print("X shape:",X.shape)

# reshape from [samples, timesteps] into [samples, timesteps, features]
n_features = 1
X = X.reshape((X.shape[0], X.shape[1], n_features))

## building LSTM model
# define model
model = Sequential()
model.add(LSTM(50, activation='relu', return_sequences=True,␣
 ↪input_shape=(n_steps, n_features)))
model.add(LSTM(50, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=300, verbose=1)


## predicting for next 10 days
# demonstrate prediction for next 10 days
x_input = np.array([187, 196, 210])
temp_input=list(x_input)
lst_output=[]
i=0
while(i<10):

    if(len(temp_input)>3):
        x_input=np.array(temp_input[1:])
        print("{} day input {}".format(i,x_input))
        #print(x_input)
        x_input = x_input.reshape((1, n_steps, n_features))
        #print(x_input)
        yhat = model.predict(x_input, verbose=0)
        print("{} day output {}".format(i,yhat))
        temp_input.append(yhat[0][0])
        temp_input=temp_input[1:]
        #print(temp_input)
        lst_output.append(yhat[0][0])
        i=i+1
    else:
        x_input = x_input.reshape((1, n_steps, n_features))
```

```
        yhat = model.predict(x_input, verbose=0)
        print(yhat[0])
        temp_input.append(yhat[0][0])
        lst_output.append(yhat[0][0])
        i=i+1


print(lst_output)

## Visualizaing The Output

import matplotlib.pyplot as plt
day_new=np.arange(1,10)
day_pred=np.arange(10,20)
plt.plot(day_new,timeseries_data)
plt.plot(day_pred,lst_output)
```

```
[[110 125 133]
 [125 133 146]
 [133 146 158]
 [146 158 172]
 [158 172 187]
 [172 187 196]]
[146 158 172 187 196 210]
X shape: (6, 3)
Epoch 1/300
```

```
/home/rgukt/.local/lib/python3.10/site-packages/keras/src/layers/rnn/rnn.py:204:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
```

```
1/1              4s 4s/step - loss:
32740.0840
Epoch 2/300
1/1              0s 67ms/step - loss:
32269.8594
Epoch 3/300
1/1              0s 50ms/step - loss:
31848.8613
Epoch 4/300
1/1              0s 60ms/step - loss:
31462.9043
Epoch 5/300
1/1              0s 54ms/step - loss:
31091.5000
Epoch 6/300
```

```
1/1             0s 135ms/step - loss:
30725.2754
Epoch 7/300
1/1             0s 51ms/step - loss:
30354.1973
Epoch 8/300
1/1             0s 64ms/step - loss:
29968.4824
Epoch 9/300
1/1             0s 58ms/step - loss:
29556.0781
Epoch 10/300
1/1             0s 62ms/step - loss:
29104.8906
Epoch 11/300
1/1             0s 67ms/step - loss:
28606.3438
Epoch 12/300
1/1             0s 133ms/step - loss:
28063.8809
Epoch 13/300
1/1             0s 96ms/step - loss:
27492.9219
Epoch 14/300
1/1             0s 121ms/step - loss:
26932.7500
Epoch 15/300
1/1             0s 131ms/step - loss:
26174.9434
Epoch 16/300
1/1             0s 91ms/step - loss:
25078.2598
Epoch 17/300
1/1             0s 59ms/step - loss:
23704.9238
Epoch 18/300
1/1             0s 128ms/step - loss:
22596.6406
Epoch 19/300
1/1             0s 62ms/step - loss:
21475.0371
Epoch 20/300
1/1             0s 55ms/step - loss:
20303.8301
Epoch 21/300
1/1             0s 56ms/step - loss:
19295.3379
Epoch 22/300
```

```
1/1             0s 58ms/step - loss:
18288.7676
Epoch 23/300
1/1             0s 154ms/step - loss:
17216.8652
Epoch 24/300
1/1             0s 70ms/step - loss:
16065.0752
Epoch 25/300
1/1             0s 134ms/step - loss:
14880.8477
Epoch 26/300
1/1             0s 136ms/step - loss:
13469.4951
Epoch 27/300
1/1             0s 57ms/step - loss:
11448.4600
Epoch 28/300
1/1             0s 58ms/step - loss:
9298.1748
Epoch 29/300
1/1             0s 56ms/step - loss:
7251.7749
Epoch 30/300
1/1             0s 62ms/step - loss:
5266.2646
Epoch 31/300
1/1             0s 66ms/step - loss:
3919.4128
Epoch 32/300
1/1             0s 72ms/step - loss:
2935.3770
Epoch 33/300
1/1             0s 65ms/step - loss:
2113.7141
Epoch 34/300
1/1             0s 64ms/step - loss:
1408.5514
Epoch 35/300
1/1             0s 66ms/step - loss:
805.2756
Epoch 36/300
1/1             0s 63ms/step - loss:
350.4561
Epoch 37/300
1/1             0s 135ms/step - loss:
43.6228
Epoch 38/300
```

```
1/1             0s 57ms/step - loss:
98.3279
Epoch 39/300
1/1             0s 60ms/step - loss:
325.7769
Epoch 40/300
1/1             0s 61ms/step - loss:
521.5362
Epoch 41/300
1/1             0s 54ms/step - loss:
659.5475
Epoch 42/300
1/1             0s 53ms/step - loss:
726.4727
Epoch 43/300
1/1             0s 93ms/step - loss:
703.8489
Epoch 44/300
1/1             0s 85ms/step - loss:
607.8985
Epoch 45/300
1/1             0s 95ms/step - loss:
462.7606
Epoch 46/300
1/1             0s 55ms/step - loss:
304.6051
Epoch 47/300
1/1             0s 77ms/step - loss:
175.6073
Epoch 48/300
1/1             0s 46ms/step - loss:
85.2028
Epoch 49/300
1/1             0s 56ms/step - loss:
32.0723
Epoch 50/300
1/1             0s 48ms/step - loss:
14.6539
Epoch 51/300
1/1             0s 52ms/step - loss:
26.9871
Epoch 52/300
1/1             0s 53ms/step - loss:
58.2968
Epoch 53/300
1/1             0s 60ms/step - loss:
95.1582
Epoch 54/300
```

```
1/1             0s 51ms/step - loss:
125.3383
Epoch 55/300
1/1             0s 49ms/step - loss:
141.0433
Epoch 56/300
1/1             0s 49ms/step - loss:
139.9550
Epoch 57/300
1/1             0s 55ms/step - loss:
124.4052
Epoch 58/300
1/1             0s 63ms/step - loss:
99.6060
Epoch 59/300
1/1             0s 54ms/step - loss:
71.7406
Epoch 60/300
1/1             0s 58ms/step - loss:
46.3718
Epoch 61/300
1/1             0s 52ms/step - loss:
27.4651
Epoch 62/300
1/1             0s 118ms/step - loss:
16.9819
Epoch 63/300
1/1             0s 112ms/step - loss:
14.8706
Epoch 64/300
1/1             0s 95ms/step - loss:
19.3718
Epoch 65/300
1/1             0s 50ms/step - loss:
27.6064
Epoch 66/300
1/1             0s 51ms/step - loss:
36.2869
Epoch 67/300
1/1             0s 47ms/step - loss:
42.2960
Epoch 68/300
1/1             0s 54ms/step - loss:
42.6962
Epoch 69/300
1/1             0s 59ms/step - loss:
37.0818
Epoch 70/300
```

```
1/1              0s 64ms/step - loss:
30.9601
Epoch 71/300
1/1              0s 60ms/step - loss:
26.2327
Epoch 72/300
1/1              0s 50ms/step - loss:
21.8507
Epoch 73/300
1/1              0s 53ms/step - loss:
18.1162
Epoch 74/300
1/1              0s 58ms/step - loss:
15.5389
Epoch 75/300
1/1              0s 56ms/step - loss:
14.3487
Epoch 76/300
1/1              0s 59ms/step - loss:
14.4365
Epoch 77/300
1/1              0s 59ms/step - loss:
15.4403
Epoch 78/300
1/1              0s 53ms/step - loss:
16.8621
Epoch 79/300
1/1              0s 51ms/step - loss:
18.2060
Epoch 80/300
1/1              0s 52ms/step - loss:
19.0904
Epoch 81/300
1/1              0s 59ms/step - loss:
19.3141
Epoch 82/300
1/1              0s 55ms/step - loss:
18.8711
Epoch 83/300
1/1              0s 52ms/step - loss:
17.9187
Epoch 84/300
1/1              0s 61ms/step - loss:
16.7090
Epoch 85/300
1/1              0s 57ms/step - loss:
15.5209
Epoch 86/300
```

```
1/1             0s 60ms/step - loss:
14.5832
Epoch 87/300
1/1             0s 46ms/step - loss:
14.0297
Epoch 88/300
1/1             0s 45ms/step - loss:
13.8826
Epoch 89/300
1/1             0s 49ms/step - loss:
14.0626
Epoch 90/300
1/1             0s 49ms/step - loss:
14.4307
Epoch 91/300
1/1             0s 55ms/step - loss:
14.8294
Epoch 92/300
1/1             0s 56ms/step - loss:
15.1316
Epoch 93/300
1/1             0s 58ms/step - loss:
15.2400
Epoch 94/300
1/1             0s 49ms/step - loss:
15.1312
Epoch 95/300
1/1             0s 61ms/step - loss:
14.8453
Epoch 96/300
1/1             0s 50ms/step - loss:
14.4599
Epoch 97/300
1/1             0s 47ms/step - loss:
14.0577
Epoch 98/300
1/1             0s 49ms/step - loss:
13.6878
Epoch 99/300
1/1             0s 57ms/step - loss:
13.3247
Epoch 100/300
1/1             0s 60ms/step - loss:
12.9377
Epoch 101/300
1/1             0s 51ms/step - loss:
13.3317
Epoch 102/300
```

```
1/1             0s 53ms/step - loss:
12.7544
Epoch 103/300
1/1             0s 137ms/step - loss:
13.3052
Epoch 104/300
1/1             0s 49ms/step - loss:
13.6860
Epoch 105/300
1/1             0s 54ms/step - loss:
13.7501
Epoch 106/300
1/1             0s 49ms/step - loss:
13.3714
Epoch 107/300
1/1             0s 51ms/step - loss:
12.4404
Epoch 108/300
1/1             0s 48ms/step - loss:
12.7257
Epoch 109/300
1/1             0s 48ms/step - loss:
12.0283
Epoch 110/300
1/1             0s 57ms/step - loss:
11.8709
Epoch 111/300
1/1             0s 47ms/step - loss:
11.9931
Epoch 112/300
1/1             0s 52ms/step - loss:
11.7285
Epoch 113/300
1/1             0s 59ms/step - loss:
11.1205
Epoch 114/300
1/1             0s 55ms/step - loss:
10.8416
Epoch 115/300
1/1             0s 48ms/step - loss:
11.1975
Epoch 116/300
1/1             0s 59ms/step - loss:
10.2022
Epoch 117/300
1/1             0s 62ms/step - loss:
10.5981
Epoch 118/300
```

```
1/1              0s 60ms/step - loss:
10.5696
Epoch 119/300
1/1              0s 54ms/step - loss:
9.9520
Epoch 120/300
1/1              0s 59ms/step - loss:
9.5621
Epoch 121/300
1/1              0s 48ms/step - loss:
9.7941
Epoch 122/300
1/1              0s 49ms/step - loss:
8.7029
Epoch 123/300
1/1              0s 44ms/step - loss:
8.3585
Epoch 124/300
1/1              0s 47ms/step - loss:
7.9360
Epoch 125/300
1/1              0s 62ms/step - loss:
8.1831
Epoch 126/300
1/1              0s 44ms/step - loss:
7.5506
Epoch 127/300
1/1              0s 47ms/step - loss:
7.4032
Epoch 128/300
1/1              0s 43ms/step - loss:
6.7061
Epoch 129/300
1/1              0s 46ms/step - loss:
6.7492
Epoch 130/300
1/1              0s 62ms/step - loss:
6.8302
Epoch 131/300
1/1              0s 58ms/step - loss:
6.1502
Epoch 132/300
1/1              0s 53ms/step - loss:
7.1845
Epoch 133/300
1/1              0s 58ms/step - loss:
6.4890
Epoch 134/300
```

```
1/1             0s 44ms/step - loss:
6.9106
Epoch 135/300
1/1             0s 41ms/step - loss:
5.3561
Epoch 136/300
1/1             0s 60ms/step - loss:
7.2382
Epoch 137/300
1/1             0s 44ms/step - loss:
6.8196
Epoch 138/300
1/1             0s 47ms/step - loss:
8.1169
Epoch 139/300
1/1             0s 58ms/step - loss:
5.5264
Epoch 140/300
1/1             0s 43ms/step - loss:
8.6631
Epoch 141/300
1/1             0s 43ms/step - loss:
5.3243
Epoch 142/300
1/1             0s 63ms/step - loss:
7.1341
Epoch 143/300
1/1             0s 62ms/step - loss:
6.5692
Epoch 144/300
1/1             0s 57ms/step - loss:
5.1512
Epoch 145/300
1/1             0s 45ms/step - loss:
7.0436
Epoch 146/300
1/1             0s 59ms/step - loss:
5.1683
Epoch 147/300
1/1             0s 56ms/step - loss:
5.8381
Epoch 148/300
1/1             0s 45ms/step - loss:
6.1720
Epoch 149/300
1/1             0s 53ms/step - loss:
5.3483
Epoch 150/300
```

```
1/1              0s 43ms/step - loss:
5.3962
Epoch 151/300
1/1              0s 55ms/step - loss:
5.9005
Epoch 152/300
1/1              0s 53ms/step - loss:
5.1210
Epoch 153/300
1/1              0s 53ms/step - loss:
5.4662
Epoch 154/300
1/1              0s 47ms/step - loss:
5.6368
Epoch 155/300
1/1              0s 53ms/step - loss:
5.2073
Epoch 156/300
1/1              0s 57ms/step - loss:
5.1825
Epoch 157/300
1/1              0s 47ms/step - loss:
5.4852
Epoch 158/300
1/1              0s 49ms/step - loss:
5.0637
Epoch 159/300
1/1              0s 44ms/step - loss:
5.1500
Epoch 160/300
1/1              0s 47ms/step - loss:
5.2866
Epoch 161/300
1/1              0s 63ms/step - loss:
5.0382
Epoch 162/300
1/1              0s 50ms/step - loss:
4.9934
Epoch 163/300
1/1              0s 46ms/step - loss:
5.1539
Epoch 164/300
1/1              0s 51ms/step - loss:
4.9017
Epoch 165/300
1/1              0s 54ms/step - loss:
4.9584
Epoch 166/300
```

```
1/1             0s 59ms/step - loss:
5.0048
Epoch 167/300
1/1             0s 58ms/step - loss:
4.8275
Epoch 168/300
1/1             0s 61ms/step - loss:
4.8883
Epoch 169/300
1/1             0s 41ms/step - loss:
4.8735
Epoch 170/300
1/1             0s 54ms/step - loss:
4.7574
Epoch 171/300
1/1             0s 56ms/step - loss:
4.8393
Epoch 172/300
1/1             0s 45ms/step - loss:
4.7511
Epoch 173/300
1/1             0s 45ms/step - loss:
4.7170
Epoch 174/300
1/1             0s 63ms/step - loss:
4.7554
Epoch 175/300
1/1             0s 44ms/step - loss:
4.6489
Epoch 176/300
1/1             0s 60ms/step - loss:
4.6938
Epoch 177/300
1/1             0s 60ms/step - loss:
4.6331
Epoch 178/300
1/1             0s 43ms/step - loss:
4.6076
Epoch 179/300
1/1             0s 53ms/step - loss:
4.6136
Epoch 180/300
1/1             0s 59ms/step - loss:
4.5459
Epoch 181/300
1/1             0s 53ms/step - loss:
4.5695
Epoch 182/300
```

```
1/1             0s 57ms/step - loss:
4.5048
Epoch 183/300
1/1             0s 54ms/step - loss:
4.5154
Epoch 184/300
1/1             0s 44ms/step - loss:
4.4714
Epoch 185/300
1/1             0s 46ms/step - loss:
4.4595
Epoch 186/300
1/1             0s 44ms/step - loss:
4.4390
Epoch 187/300
1/1             0s 59ms/step - loss:
4.4071
Epoch 188/300
1/1             0s 136ms/step - loss:
4.3976
Epoch 189/300
1/1             0s 56ms/step - loss:
4.3577
Epoch 190/300
1/1             0s 56ms/step - loss:
4.3468
Epoch 191/300
1/1             0s 56ms/step - loss:
4.2901
Epoch 192/300
1/1             0s 56ms/step - loss:
4.2704
Epoch 193/300
1/1             0s 41ms/step - loss:
4.2882
Epoch 194/300
1/1             0s 49ms/step - loss:
4.2875
Epoch 195/300
1/1             0s 46ms/step - loss:
4.2204
Epoch 196/300
1/1             0s 46ms/step - loss:
4.2116
Epoch 197/300
1/1             0s 55ms/step - loss:
4.1751
Epoch 198/300
```

```
1/1             0s 47ms/step - loss:
4.1365
Epoch 199/300
1/1             0s 49ms/step - loss:
4.1331
Epoch 200/300
1/1             0s 46ms/step - loss:
4.0966
Epoch 201/300
1/1             0s 43ms/step - loss:
4.0913
Epoch 202/300
1/1             0s 50ms/step - loss:
4.0726
Epoch 203/300
1/1             0s 54ms/step - loss:
4.0445
Epoch 204/300
1/1             0s 61ms/step - loss:
4.0357
Epoch 205/300
1/1             0s 55ms/step - loss:
4.0045
Epoch 206/300
1/1             0s 52ms/step - loss:
3.9797
Epoch 207/300
1/1             0s 52ms/step - loss:
3.9709
Epoch 208/300
1/1             0s 52ms/step - loss:
3.9485
Epoch 209/300
1/1             0s 59ms/step - loss:
3.9205
Epoch 210/300
1/1             0s 48ms/step - loss:
3.9080
Epoch 211/300
1/1             0s 61ms/step - loss:
3.8978
Epoch 212/300
1/1             0s 62ms/step - loss:
3.8725
Epoch 213/300
1/1             0s 50ms/step - loss:
3.8469
Epoch 214/300
```

```
1/1             0s 46ms/step - loss:
3.8294
Epoch 215/300
1/1             0s 59ms/step - loss:
3.8164
Epoch 216/300
1/1             0s 116ms/step - loss:
3.8025
Epoch 217/300
1/1             0s 83ms/step - loss:
3.7818
Epoch 218/300
1/1             0s 88ms/step - loss:
3.7591
Epoch 219/300
1/1             0s 124ms/step - loss:
3.7333
Epoch 220/300
1/1             0s 51ms/step - loss:
3.7093
Epoch 221/300
1/1             0s 46ms/step - loss:
3.6873
Epoch 222/300
1/1             0s 60ms/step - loss:
3.6669
Epoch 223/300
1/1             0s 46ms/step - loss:
3.6476
Epoch 224/300
1/1             0s 59ms/step - loss:
3.6295
Epoch 225/300
1/1             0s 46ms/step - loss:
3.6162
Epoch 226/300
1/1             0s 136ms/step - loss:
3.6157
Epoch 227/300
1/1             0s 45ms/step - loss:
3.6603
Epoch 228/300
1/1             0s 50ms/step - loss:
3.7582
Epoch 229/300
1/1             0s 60ms/step - loss:
4.0037
Epoch 230/300
```

```
1/1             0s 63ms/step - loss:
3.7955
Epoch 231/300
1/1             0s 45ms/step - loss:
3.6249
Epoch 232/300
1/1             0s 46ms/step - loss:
3.4796
Epoch 233/300
1/1             0s 49ms/step - loss:
3.5557
Epoch 234/300
1/1             0s 43ms/step - loss:
3.7286
Epoch 235/300
1/1             0s 45ms/step - loss:
3.5923
Epoch 236/300
1/1             0s 45ms/step - loss:
3.4554
Epoch 237/300
1/1             0s 49ms/step - loss:
3.4095
Epoch 238/300
1/1             0s 54ms/step - loss:
3.4799
Epoch 239/300
1/1             0s 51ms/step - loss:
3.5571
Epoch 240/300
1/1             0s 60ms/step - loss:
3.4413
Epoch 241/300
1/1             0s 49ms/step - loss:
3.3508
Epoch 242/300
1/1             0s 62ms/step - loss:
3.3461
Epoch 243/300
1/1             0s 58ms/step - loss:
3.3906
Epoch 244/300
1/1             0s 46ms/step - loss:
3.4230
Epoch 245/300
1/1             0s 53ms/step - loss:
3.3466
Epoch 246/300
```

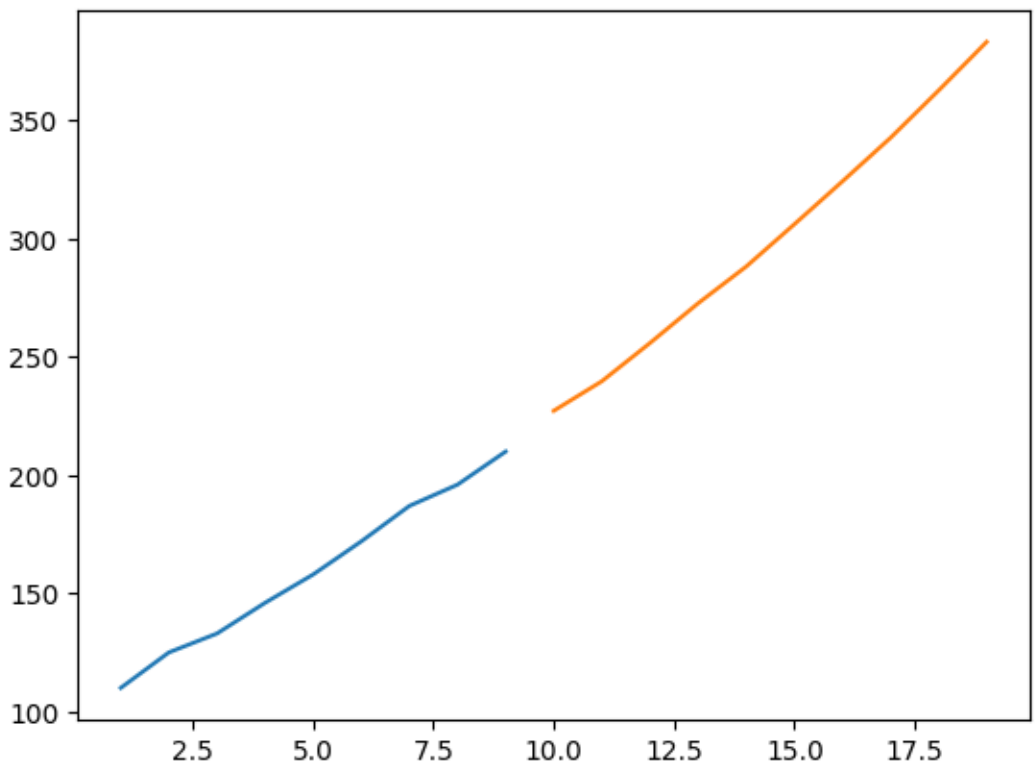
```
1/1             0s 42ms/step - loss:
3.2818
Epoch 247/300
1/1             0s 50ms/step - loss:
3.2605
Epoch 248/300
1/1             0s 61ms/step - loss:
3.2799
Epoch 249/300
1/1             0s 62ms/step - loss:
3.3118
Epoch 250/300
1/1             0s 59ms/step - loss:
3.2931
Epoch 251/300
1/1             0s 45ms/step - loss:
3.2685
Epoch 252/300
1/1             0s 41ms/step - loss:
3.2149
Epoch 253/300
1/1             0s 42ms/step - loss:
3.1769
Epoch 254/300
1/1             0s 45ms/step - loss:
3.1545
Epoch 255/300
1/1             0s 41ms/step - loss:
3.1467
Epoch 256/300
1/1             0s 40ms/step - loss:
3.1516
Epoch 257/300
1/1             0s 63ms/step - loss:
3.1666
Epoch 258/300
1/1             0s 56ms/step - loss:
3.2258
Epoch 259/300
1/1             0s 46ms/step - loss:
3.2976
Epoch 260/300
1/1             0s 57ms/step - loss:
3.5820
Epoch 261/300
1/1             0s 60ms/step - loss:
3.5964
Epoch 262/300
```

```
1/1             0s 50ms/step - loss:
3.9458
Epoch 263/300
1/1             0s 52ms/step - loss:
3.2911
Epoch 264/300
1/1             0s 43ms/step - loss:
3.0203
Epoch 265/300
1/1             0s 60ms/step - loss:
3.0985
Epoch 266/300
1/1             0s 46ms/step - loss:
3.2876
Epoch 267/300
1/1             0s 41ms/step - loss:
3.4853
Epoch 268/300
1/1             0s 42ms/step - loss:
3.0937
Epoch 269/300
1/1             0s 49ms/step - loss:
2.9579
Epoch 270/300
1/1             0s 42ms/step - loss:
3.1038
Epoch 271/300
1/1             0s 48ms/step - loss:
3.1609
Epoch 272/300
1/1             0s 47ms/step - loss:
3.1313
Epoch 273/300
1/1             0s 58ms/step - loss:
2.9280
Epoch 274/300
1/1             0s 43ms/step - loss:
2.9214
Epoch 275/300
1/1             0s 137ms/step - loss:
3.0568
Epoch 276/300
1/1             0s 48ms/step - loss:
3.0295
Epoch 277/300
1/1             0s 49ms/step - loss:
2.9545
Epoch 278/300
```

```
1/1             0s 60ms/step - loss:
2.8366
Epoch 279/300
1/1             0s 43ms/step - loss:
2.8265
Epoch 280/300
1/1             0s 40ms/step - loss:
2.8963
Epoch 281/300
1/1             0s 42ms/step - loss:
2.9141
Epoch 282/300
1/1             0s 45ms/step - loss:
2.9457
Epoch 283/300
1/1             0s 42ms/step - loss:
2.8608
Epoch 284/300
1/1             0s 59ms/step - loss:
2.8255
Epoch 285/300
1/1             0s 48ms/step - loss:
2.7575
Epoch 286/300
1/1             0s 48ms/step - loss:
2.7635
Epoch 287/300
1/1             0s 49ms/step - loss:
2.8370
Epoch 288/300
1/1             0s 59ms/step - loss:
3.2931
Epoch 289/300
1/1             0s 45ms/step - loss:
3.7081
Epoch 290/300
1/1             0s 50ms/step - loss:
5.4738
Epoch 291/300
1/1             0s 48ms/step - loss:
2.6471
Epoch 292/300
1/1             0s 52ms/step - loss:
3.6660
Epoch 293/300
1/1             0s 51ms/step - loss:
6.6825
Epoch 294/300
```

```
1/1              0s 50ms/step - loss:
2.8921
Epoch 295/300
1/1              0s 60ms/step - loss:
9.9881
Epoch 296/300
1/1              0s 52ms/step - loss:
8.4289
Epoch 297/300
1/1              0s 58ms/step - loss:
8.9559
Epoch 298/300
1/1              0s 56ms/step - loss:
4.0930
Epoch 299/300
1/1              0s 52ms/step - loss:
7.9612
Epoch 300/300
1/1              0s 57ms/step - loss:
7.7836
[227.2468]
1 day input [196.         210.         227.24679565]
1 day output [[239.75008]]
2 day input [210.         227.24679565 239.75007629]
2 day output [[255.90955]]
3 day input [227.2468  239.75008 255.90955]
3 day output [[272.71838]]
4 day input [239.75008 255.90955 272.71838]
4 day output [[288.273]]
5 day input [255.90955 272.71838 288.273  ]
5 day output [[306.1518]]
6 day input [272.71838 288.273   306.1518 ]
6 day output [[324.30753]]
7 day input [288.273   306.1518  324.30753]
7 day output [[342.73947]]
8 day input [306.1518  324.30753 342.73947]
8 day output [[362.76816]]
9 day input [324.30753 342.73947 362.76816]
9 day output [[383.34955]]
[227.2468, 239.75008, 255.90955, 272.71838, 288.273, 306.1518, 324.30753,
342.73947, 362.76816, 383.34955]
```

[16]:
```
[<matplotlib.lines.Line2D at 0x7f8a6c7af1f0>]
```



7 Audio Classification EDA(Explorartory Data Analysis)

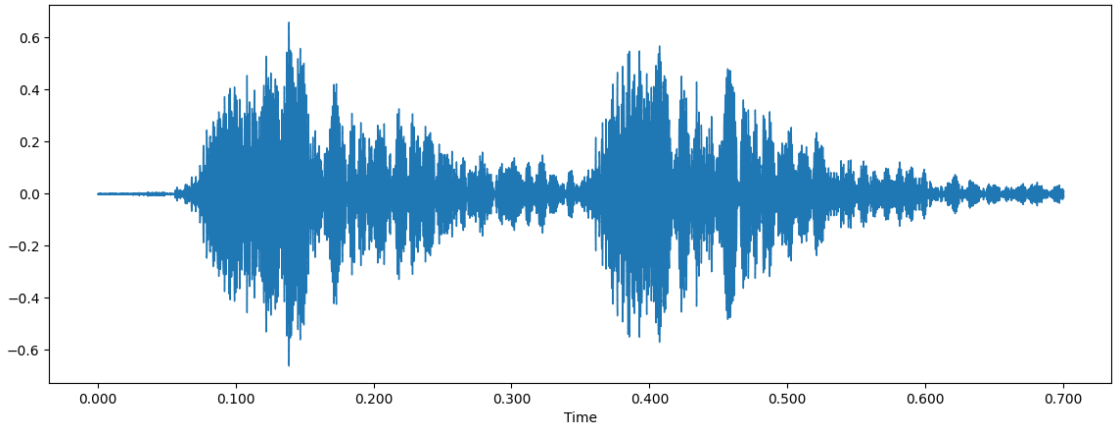
[9]:
```
import matplotlib.pyplot as plt
%matplotlib inline
### Let's read a sample audio using librosa
import IPython.display as ipd
import librosa
import librosa.display


## Dog sound
filename='UrbanSound8K/dog_bark.wav'
print("Dog")
plt.figure(figsize=(14,5))
data,sample_rate=librosa.load(filename)
librosa.display.waveshow(data,sr=sample_rate)
ipd.Audio(filename)

print("Sample rate:",sample_rate)-

print("Sound")
```

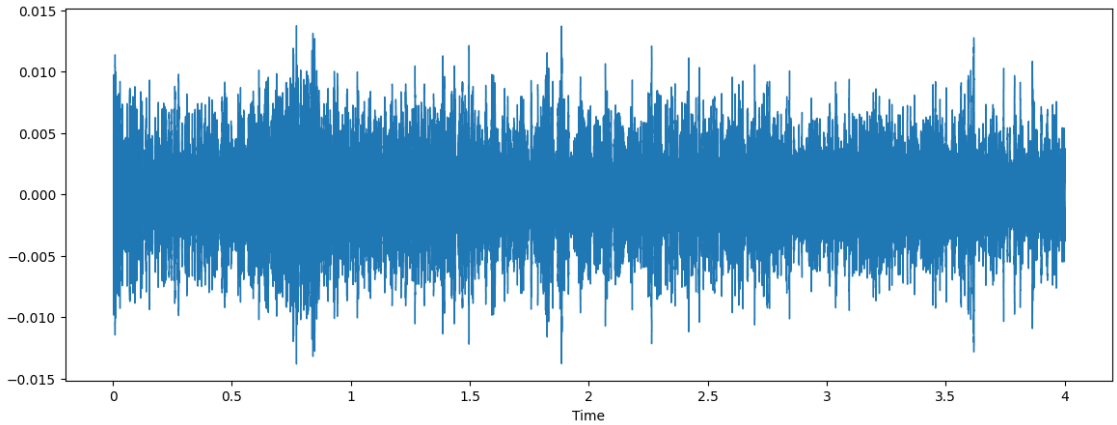
```python
### Sound
filename='UrbanSound8K/100263-2-0-3.wav'
plt.figure(figsize=(14,5))
data,sample_rate=librosa.load(filename)
librosa.display.waveshow(data,sr=sample_rate)
ipd.Audio(filename)


### Let's read a sample audio using scipy
from scipy.io import wavfile as wav
wave_sample_rate, wave_audio=wav.read(filename)
print("Wave sample rate:",wave_sample_rate)
print("Wave audio:",wave_audio)
print("Data:",data)


import pandas as pd

metadata=pd.read_csv('UrbanSound8K/metadata/UrbanSound8K.csv')
metadata.head(10)

### Check whether the dataset is imbalanced
metadata['class'].value_counts()
```

```
Dog
Sample rate: 22050
Sound
Wave sample rate: 44100
Wave audio: [[ 194  100]
 [ 179  113]
 [ 160  124]
 …
 [-143  -87]
 [-134  -91]
 [-110  -98]]
Data: [ 0.00331575  0.00467553  0.00361099 … -0.00376796 -0.00347471
 -0.00357828]
```

[9]:
```
class
dog_bark            1000
children_playing    1000
air_conditioner     1000
street_music        1000
engine_idling       1000
jackhammer          1000
drilling            1000
siren                929
```

```
car_horn             429
gun_shot             374
Name: count, dtype: int64
```





[6]:
```
!pip install librosa
!pip install resampy
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: librosa in
/home/rgukt/.local/lib/python3.10/site-packages (0.10.2.post1)
Requirement already satisfied: audioread>=2.1.9 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (3.0.1)
Requirement already satisfied: numba>=0.51.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (0.60.0)
Requirement already satisfied: soxr>=0.3.2 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (0.5.0.post1)
Requirement already satisfied: scikit-learn>=0.20.0 in
```

```
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (1.5.1)
Requirement already satisfied: decorator>=4.3.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (5.1.1)
Requirement already satisfied: pooch>=1.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (1.8.2)
Requirement already satisfied: lazy-loader>=0.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (0.4)
Requirement already satisfied: typing-extensions>=4.1.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (4.12.2)
Requirement already satisfied: msgpack>=1.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (1.1.0)
Requirement already satisfied: joblib>=0.14 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (1.4.2)
Requirement already satisfied: soundfile>=0.12.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (0.12.1)
Requirement already satisfied: scipy>=1.2.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (1.14.0)
Requirement already satisfied: numpy!=1.22.0,!=1.22.1,!=1.22.2,>=1.20.3 in
/home/rgukt/.local/lib/python3.10/site-packages (from librosa) (1.26.4)
Requirement already satisfied: packaging in
/home/rgukt/.local/lib/python3.10/site-packages (from lazy-loader>=0.1->librosa)
(24.1)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/home/rgukt/.local/lib/python3.10/site-packages (from numba>=0.51.0->librosa)
(0.43.0)
Requirement already satisfied: platformdirs>=2.5.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from pooch>=1.1->librosa)
(4.2.2)
Requirement already satisfied: requests>=2.19.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from pooch>=1.1->librosa)
(2.32.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from scikit-
learn>=0.20.0->librosa) (3.5.0)
Requirement already satisfied: cffi>=1.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from
soundfile>=0.12.1->librosa) (1.16.0)
Requirement already satisfied: pycparser in
/home/rgukt/.local/lib/python3.10/site-packages (from
cffi>=1.0->soundfile>=0.12.1->librosa) (2.22)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages
(from requests>=2.19.0->pooch>=1.1->librosa) (3.3)
Requirement already satisfied: charset-normalizer<4,>=2 in
/home/rgukt/.local/lib/python3.10/site-packages (from
requests>=2.19.0->pooch>=1.1->librosa) (3.3.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/lib/python3/dist-
packages (from requests>=2.19.0->pooch>=1.1->librosa) (1.26.5)
Requirement already satisfied: certifi>=2017.4.17 in
```

```
/home/rgukt/.local/lib/python3.10/site-packages (from
requests>=2.19.0->pooch>=1.1->librosa) (2024.8.30)
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: resampy in
/home/rgukt/.local/lib/python3.10/site-packages (0.4.3)
Requirement already satisfied: numpy>=1.17 in
/home/rgukt/.local/lib/python3.10/site-packages (from resampy) (1.26.4)
Requirement already satisfied: numba>=0.53 in
/home/rgukt/.local/lib/python3.10/site-packages (from resampy) (0.60.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/home/rgukt/.local/lib/python3.10/site-packages (from numba>=0.53->resampy)
(0.43.0)
```

7.1 Data Preprocessing

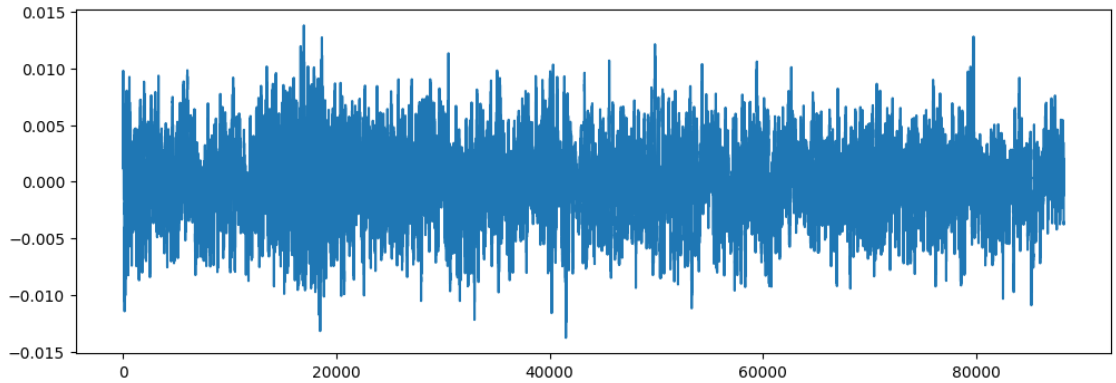
1.Here Librosa converts the signal to mono, meaning the channel will always be 1.

2.Mel-Frequency Cepstral Coefficients(MFCC) from the audio samples. The MFCC summarises the frequency distribution across the window size, so it is possible to analyse both the frequency and time characteristics of the sound. These audio representations will allow us to identify features for classification.

[2]:
```python
### Let's read a sample audio using librosa
import librosa
audio_file_path='UrbanSound8K/100263-2-0-3.wav'
librosa_audio_data,librosa_sample_rate=librosa.load(audio_file_path)
print(librosa_audio_data)

### Lets plot the librosa audio data
import matplotlib.pyplot as plt
# Original audio with 1 channel
plt.figure(figsize=(12, 4))
plt.plot(librosa_audio_data)
```

```
[ 0.00331575  0.00467553  0.00361099 … -0.00376796 -0.00347471
 -0.00357828]
```

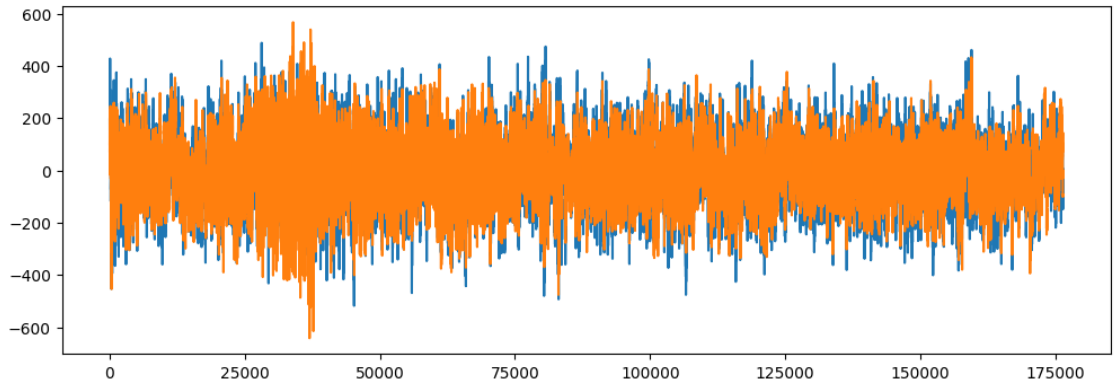
[2]:
```
[<matplotlib.lines.Line2D at 0x7f23749e2770>]
```



[3]:
```
### Lets read with scipy
from scipy.io import wavfile as wav
wave_sample_rate, wave_audio = wav.read(audio_file_path)
print(wave_audio)

import matplotlib.pyplot as plt

# Original audio with 2 channels
plt.figure(figsize=(12, 4))
plt.plot(wave_audio)
```

```
[[ 194  100]
 [ 179  113]
 [ 160  124]
 …
 [-143  -87]
 [-134  -91]
 [-110  -98]]
```

[3]:
```
[<matplotlib.lines.Line2D at 0x7f23726d7070>,
 <matplotlib.lines.Line2D at 0x7f23726d7160>]
```



[15]:
```
mfccs = librosa.feature.mfcc(y=librosa_audio_data, sr=librosa_sample_rate,␣
↪n_mfcc=40)
print("MFCCS shape:",mfccs.shape)

# Extracting MFCC's For every audio file
import pandas as pd
import numpy as np
import os
import librosa

audio_dataset_path='UrbanSound8K/audio/'
metadata=pd.read_csv('UrbanSound8K/metadata/UrbanSound8K.csv')
print(metadata.head())

def features_extractor(file):
    audio, sample_rate = librosa.load(file_name, res_type='kaiser_fast')
    mfccs_features = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=40)
    mfccs_scaled_features = np.mean(mfccs_features.T,axis=0)
    return mfccs_scaled_features

from tqdm import tqdm
### Now we iterate through every audio file and extract features
### using Mel-Frequency Cepstral Coefficients
extracted_features=[]
for index_num,row in tqdm(metadata.iterrows()):
    file_name = os.path.join(os.path.
↪abspath(audio_dataset_path),'fold'+str(row["fold"])+'/
↪',str(row["slice_file_name"]))
    final_class_labels=row["class"]
    data=features_extractor(file_name)
    extracted_features.append([data,final_class_labels])

# converting extracted_features to Pandas dataframe
print("Extracted features to DataFrame:")
extracted_features_df=pd.
↪DataFrame(extracted_features,columns=['feature','class'])
print(extracted_features_df.head())

# Split the dataset into independent and dependent dataset
X=np.array(extracted_features_df['feature'].tolist())
y=np.array(extracted_features_df['class'].tolist())

print("X shape:",X.shape)
```

```python
# Label Encoding
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder
labelencoder=LabelEncoder()
y=to_categorical(labelencoder.fit_transform(y))

print("y shape:",y.shape)

# Train Test Split
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)

print("X train:",X_train.shape)

rint("X test:",X_test.shape)
print("y train:",y_train.shape)
print("y test:",y_test.shape)
```

```
MFCCS shape: (40, 173)
      slice_file_name    fsID  start        end  salience  fold  classID  \
0    100032-3-0-0.wav  100032    0.0   0.317551         1     5        3
1  100263-2-0-117.wav  100263   58.5  62.500000         1     5        2
2  100263-2-0-121.wav  100263   60.5  64.500000         1     5        2
3  100263-2-0-126.wav  100263   63.0  67.000000         1     5        2
4  100263-2-0-137.wav  100263   68.5  72.500000         1     5        2

              class
0          dog_bark
1  children_playing
2  children_playing
3  children_playing
4  children_playing
```

```
3554it [04:30, 13.86it/s]/home/rgukt/.local/lib/python3.10/site-
packages/librosa/core/spectrum.py:266: UserWarning: n_fft=2048 is too large for
input signal of length=1323
  warnings.warn(
8325it [10:27, 18.49it/s]/home/rgukt/.local/lib/python3.10/site-
packages/librosa/core/spectrum.py:266: UserWarning: n_fft=2048 is too large for
input signal of length=1103
  warnings.warn(
/home/rgukt/.local/lib/python3.10/site-packages/librosa/core/spectrum.py:266:
UserWarning: n_fft=2048 is too large for input signal of length=1523
  warnings.warn(
8732it [10:55, 13.32it/s]
```

```
Extracted features to DataFrame:
                                                feature              class
```

```
0  [-217.35526, 70.22339, -130.38527, -53.282898,…          dog_bark
1  [-424.09818, 109.34077, -52.919525, 60.86475, …  children_playing
2  [-458.79114, 121.38419, -46.520657, 52.00812, …  children_playing
3  [-413.89984, 101.66371, -35.42945, 53.036354, …  children_playing
4  [-446.60352, 113.68541, -52.402214, 60.302044,…  children_playing
X shape: (8732, 40)
y shape: (8732, 10)
X train: (6985, 40)
X test: (1747, 40)
y train: (6985, 10)
y test: (1747, 10)
```

7.2 Model Creation

[16]:
```
print(y)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout,Activation,Flatten
from tensorflow.keras.optimizers import Adam
from sklearn import metrics

#No of classes
num_labels=y.shape[1]
print(y)

model=Sequential()
#first layer
model.add(Dense(100,input_shape=(40,)))
model.add(Activation('relu'))
model.add(Dropout(0.5))
#second layer
model.add(Dense(200))
model.add(Activation('relu'))
model.add(Dropout(0.5))
#third layer
model.add(Dense(100))
model.add(Activation('relu'))
model.add(Dropout(0.5))

#final layer
model.add(Dense(num_labels))
model.add(Activation('softmax'))

model.summary()

model.
 ↪compile(loss='categorical_crossentropy',metrics=['accuracy'],optimizer='adam')
```

```python
# Trianing my model
from tensorflow.keras.callbacks import ModelCheckpoint
from datetime import datetime

num_epochs = 100
num_batch_size = 32
checkpointer = ModelCheckpoint(filepath='saved_models/audio_classification.
 ↪keras', verbose=1, save_best_only=True)
start = datetime.now()
model.fit(X_train, y_train, batch_size=num_batch_size, epochs=num_epochs,␣
 ↪validation_data=(X_test, y_test), callbacks=[checkpointer], verbose=1)


duration = datetime.now() - start
print("Training completed in time: ", duration)

model.save("urbansoundmodel.h5")

test_accuracy=model.evaluate(X_test,y_test,verbose=0)
print(test_accuracy[1])
```

```
[[0. 0. 0. … 0. 0. 0.]
 [0. 0. 1. … 0. 0. 0.]
 [0. 0. 1. … 0. 0. 0.]
 …
 [0. 1. 0. … 0. 0. 0.]
 [0. 1. 0. … 0. 0. 0.]
 [0. 1. 0. … 0. 0. 0.]]
[[0. 0. 0. … 0. 0. 0.]
 [0. 0. 1. … 0. 0. 0.]
 [0. 0. 1. … 0. 0. 0.]
 …
 [0. 1. 0. … 0. 0. 0.]
 [0. 1. 0. … 0. 0. 0.]
 [0. 1. 0. … 0. 0. 0.]]
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 100)	4,100

```
 activation_8 (Activation)        (None, 100)                        0

 dropout_6 (Dropout)              (None, 100)                        0

 dense_9 (Dense)                  (None, 200)                   20,200

 activation_9 (Activation)        (None, 200)                        0

 dropout_7 (Dropout)              (None, 200)                        0

 dense_10 (Dense)                 (None, 100)                   20,100

 activation_10 (Activation)       (None, 100)                        0

 dropout_8 (Dropout)              (None, 100)                        0

 dense_11 (Dense)                 (None, 10)                     1,010

 activation_11 (Activation)       (None, 10)                         0


 Total params: 45,410 (177.38 KB)

 Trainable params: 45,410 (177.38 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/100
215/219             0s 5ms/step -
accuracy: 0.1138 - loss: 20.7341
Epoch 1: val_loss improved from inf to 2.27619, saving model to
saved_models/audio_classification.keras
219/219             4s 7ms/step -
accuracy: 0.1139 - loss: 20.4902 - val_accuracy: 0.1179 - val_loss: 2.2762
Epoch 2/100
209/219             0s 4ms/step -
accuracy: 0.1313 - loss: 2.6411
Epoch 2: val_loss improved from 2.27619 to 2.25930, saving model to
saved_models/audio_classification.keras
219/219             1s 5ms/step -
accuracy: 0.1312 - loss: 2.6356 - val_accuracy: 0.1179 - val_loss: 2.2593
Epoch 3/100
215/219             0s 4ms/step -
accuracy: 0.1331 - loss: 2.3393
Epoch 3: val_loss improved from 2.25930 to 2.18422, saving model to
```

```
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.1332 - loss: 2.3386 - val_accuracy: 0.1671 - val_loss: 2.1842
Epoch 4/100
211/219              0s 4ms/step -
accuracy: 0.1546 - loss: 2.2752
Epoch 4: val_loss improved from 2.18422 to 2.13591, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.1550 - loss: 2.2737 - val_accuracy: 0.1689 - val_loss: 2.1359
Epoch 5/100
210/219              0s 4ms/step -
accuracy: 0.1806 - loss: 2.1998
Epoch 5: val_loss improved from 2.13591 to 2.03048, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.1808 - loss: 2.1985 - val_accuracy: 0.2690 - val_loss: 2.0305
Epoch 6/100
208/219              0s 4ms/step -
accuracy: 0.2076 - loss: 2.1364
Epoch 6: val_loss improved from 2.03048 to 1.96907, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.2076 - loss: 2.1352 - val_accuracy: 0.2742 - val_loss: 1.9691
Epoch 7/100
217/219              0s 4ms/step -
accuracy: 0.2098 - loss: 2.0580
Epoch 7: val_loss improved from 1.96907 to 1.93606, saving model to
saved_models/audio_classification.keras
219/219              1s 5ms/step -
accuracy: 0.2100 - loss: 2.0579 - val_accuracy: 0.2753 - val_loss: 1.9361
Epoch 8/100
208/219              0s 4ms/step -
accuracy: 0.2270 - loss: 2.0108
Epoch 8: val_loss improved from 1.93606 to 1.86194, saving model to
saved_models/audio_classification.keras
219/219              1s 5ms/step -
accuracy: 0.2277 - loss: 2.0099 - val_accuracy: 0.3309 - val_loss: 1.8619
Epoch 9/100
213/219              0s 5ms/step -
accuracy: 0.2766 - loss: 1.9623
Epoch 9: val_loss improved from 1.86194 to 1.78059, saving model to
saved_models/audio_classification.keras
219/219              1s 5ms/step -
accuracy: 0.2767 - loss: 1.9616 - val_accuracy: 0.3818 - val_loss: 1.7806
Epoch 10/100
208/219              0s 4ms/step -
accuracy: 0.2927 - loss: 1.8876
```

```
Epoch 10: val_loss improved from 1.78059 to 1.67523, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.2931 - loss: 1.8872 - val_accuracy: 0.4442 - val_loss: 1.6752
Epoch 11/100
206/219            0s 4ms/step -
accuracy: 0.3431 - loss: 1.8198
Epoch 11: val_loss improved from 1.67523 to 1.62053, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.3430 - loss: 1.8195 - val_accuracy: 0.4419 - val_loss: 1.6205
Epoch 12/100
218/219            0s 4ms/step -
accuracy: 0.3606 - loss: 1.7683
Epoch 12: val_loss improved from 1.62053 to 1.53666, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.3607 - loss: 1.7682 - val_accuracy: 0.4928 - val_loss: 1.5367
Epoch 13/100
207/219            0s 4ms/step -
accuracy: 0.3826 - loss: 1.7018
Epoch 13: val_loss improved from 1.53666 to 1.49681, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.3826 - loss: 1.7020 - val_accuracy: 0.5197 - val_loss: 1.4968
Epoch 14/100
212/219            0s 4ms/step -
accuracy: 0.4034 - loss: 1.6528
Epoch 14: val_loss improved from 1.49681 to 1.44176, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.4035 - loss: 1.6533 - val_accuracy: 0.5295 - val_loss: 1.4418
Epoch 15/100
207/219            0s 4ms/step -
accuracy: 0.4192 - loss: 1.6500
Epoch 15: val_loss improved from 1.44176 to 1.41051, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.4193 - loss: 1.6492 - val_accuracy: 0.5547 - val_loss: 1.4105
Epoch 16/100
206/219            0s 4ms/step -
accuracy: 0.4298 - loss: 1.5868
Epoch 16: val_loss improved from 1.41051 to 1.39376, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.4300 - loss: 1.5871 - val_accuracy: 0.5552 - val_loss: 1.3938
Epoch 17/100
215/219            0s 4ms/step -
```

```
accuracy: 0.4389 - loss: 1.6138
Epoch 17: val_loss improved from 1.39376 to 1.33942, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.4393 - loss: 1.6128 - val_accuracy: 0.5713 - val_loss: 1.3394
Epoch 18/100
206/219            0s 4ms/step -
accuracy: 0.4662 - loss: 1.5575
Epoch 18: val_loss improved from 1.33942 to 1.30860, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.4663 - loss: 1.5567 - val_accuracy: 0.5827 - val_loss: 1.3086
Epoch 19/100
205/219            0s 4ms/step -
accuracy: 0.4744 - loss: 1.4942
Epoch 19: val_loss improved from 1.30860 to 1.29107, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.4750 - loss: 1.4930 - val_accuracy: 0.5890 - val_loss: 1.2911
Epoch 20/100
219/219            0s 4ms/step -
accuracy: 0.5016 - loss: 1.4706
Epoch 20: val_loss improved from 1.29107 to 1.24524, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.5016 - loss: 1.4706 - val_accuracy: 0.6136 - val_loss: 1.2452
Epoch 21/100
212/219            0s 4ms/step -
accuracy: 0.5113 - loss: 1.4119
Epoch 21: val_loss improved from 1.24524 to 1.22485, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.5111 - loss: 1.4127 - val_accuracy: 0.6136 - val_loss: 1.2249
Epoch 22/100
209/219            0s 3ms/step -
accuracy: 0.5162 - loss: 1.4126
Epoch 22: val_loss improved from 1.22485 to 1.19010, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.5159 - loss: 1.4129 - val_accuracy: 0.6297 - val_loss: 1.1901
Epoch 23/100
209/219            0s 4ms/step -
accuracy: 0.5212 - loss: 1.3948
Epoch 23: val_loss improved from 1.19010 to 1.17743, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.5215 - loss: 1.3948 - val_accuracy: 0.6188 - val_loss: 1.1774
Epoch 24/100
```

```
206/219              0s 3ms/step -
accuracy: 0.5245 - loss: 1.3680
Epoch 24: val_loss improved from 1.17743 to 1.17100, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5252 - loss: 1.3674 - val_accuracy: 0.6234 - val_loss: 1.1710
Epoch 25/100
208/219              0s 3ms/step -
accuracy: 0.5398 - loss: 1.3420
Epoch 25: val_loss improved from 1.17100 to 1.14676, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5399 - loss: 1.3421 - val_accuracy: 0.6394 - val_loss: 1.1468
Epoch 26/100
219/219              0s 4ms/step -
accuracy: 0.5345 - loss: 1.3398
Epoch 26: val_loss improved from 1.14676 to 1.13965, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5345 - loss: 1.3398 - val_accuracy: 0.6188 - val_loss: 1.1397
Epoch 27/100
205/219              0s 4ms/step -
accuracy: 0.5445 - loss: 1.3329
Epoch 27: val_loss improved from 1.13965 to 1.10338, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5450 - loss: 1.3321 - val_accuracy: 0.6377 - val_loss: 1.1034
Epoch 28/100
219/219              0s 4ms/step -
accuracy: 0.5622 - loss: 1.2824
Epoch 28: val_loss improved from 1.10338 to 1.10095, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5622 - loss: 1.2824 - val_accuracy: 0.6440 - val_loss: 1.1009
Epoch 29/100
207/219              0s 4ms/step -
accuracy: 0.5679 - loss: 1.2892
Epoch 29: val_loss improved from 1.10095 to 1.04976, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5679 - loss: 1.2885 - val_accuracy: 0.6634 - val_loss: 1.0498
Epoch 30/100
218/219              0s 4ms/step -
accuracy: 0.5716 - loss: 1.2807
Epoch 30: val_loss improved from 1.04976 to 1.04182, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5716 - loss: 1.2805 - val_accuracy: 0.6726 - val_loss: 1.0418
```

```
Epoch 31/100
205/219            0s 4ms/step -
accuracy: 0.5726 - loss: 1.2600
Epoch 31: val_loss improved from 1.04182 to 1.03645, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.5726 - loss: 1.2596 - val_accuracy: 0.6629 - val_loss: 1.0365
Epoch 32/100
206/219            0s 4ms/step -
accuracy: 0.5729 - loss: 1.2198
Epoch 32: val_loss did not improve from 1.03645
219/219            1s 4ms/step -
accuracy: 0.5731 - loss: 1.2203 - val_accuracy: 0.6697 - val_loss: 1.0373
Epoch 33/100
216/219            0s 4ms/step -
accuracy: 0.6019 - loss: 1.1751
Epoch 33: val_loss improved from 1.03645 to 1.01450, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6017 - loss: 1.1757 - val_accuracy: 0.6732 - val_loss: 1.0145
Epoch 34/100
214/219            0s 4ms/step -
accuracy: 0.5874 - loss: 1.2008
Epoch 34: val_loss improved from 1.01450 to 1.00559, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.5874 - loss: 1.2013 - val_accuracy: 0.6714 - val_loss: 1.0056
Epoch 35/100
205/219            0s 4ms/step -
accuracy: 0.5904 - loss: 1.2007
Epoch 35: val_loss did not improve from 1.00559
219/219            1s 4ms/step -
accuracy: 0.5908 - loss: 1.2004 - val_accuracy: 0.6783 - val_loss: 1.0069
Epoch 36/100
206/219            0s 4ms/step -
accuracy: 0.6023 - loss: 1.1705
Epoch 36: val_loss improved from 1.00559 to 1.00093, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6021 - loss: 1.1710 - val_accuracy: 0.6617 - val_loss: 1.0009
Epoch 37/100
209/219            0s 4ms/step -
accuracy: 0.6026 - loss: 1.1911
Epoch 37: val_loss improved from 1.00093 to 0.96828, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6026 - loss: 1.1906 - val_accuracy: 0.6892 - val_loss: 0.9683
Epoch 38/100
```

```
219/219              0s 4ms/step -
accuracy: 0.5960 - loss: 1.1826
Epoch 38: val_loss improved from 0.96828 to 0.95092, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.5961 - loss: 1.1826 - val_accuracy: 0.6949 - val_loss: 0.9509
Epoch 39/100
208/219              0s 4ms/step -
accuracy: 0.6012 - loss: 1.1761
Epoch 39: val_loss improved from 0.95092 to 0.94518, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.6016 - loss: 1.1751 - val_accuracy: 0.6995 - val_loss: 0.9452
Epoch 40/100
217/219              0s 4ms/step -
accuracy: 0.6165 - loss: 1.1573
Epoch 40: val_loss did not improve from 0.94518
219/219              1s 4ms/step -
accuracy: 0.6165 - loss: 1.1571 - val_accuracy: 0.6863 - val_loss: 0.9578
Epoch 41/100
214/219              0s 4ms/step -
accuracy: 0.6181 - loss: 1.1002
Epoch 41: val_loss did not improve from 0.94518
219/219              1s 4ms/step -
accuracy: 0.6180 - loss: 1.1010 - val_accuracy: 0.6795 - val_loss: 0.9676
Epoch 42/100
211/219              0s 4ms/step -
accuracy: 0.6221 - loss: 1.1243
Epoch 42: val_loss improved from 0.94518 to 0.93429, saving model to
saved_models/audio_classification.keras
219/219              1s 5ms/step -
accuracy: 0.6219 - loss: 1.1250 - val_accuracy: 0.7018 - val_loss: 0.9343
Epoch 43/100
206/219              0s 4ms/step -
accuracy: 0.6229 - loss: 1.1276
Epoch 43: val_loss improved from 0.93429 to 0.93400, saving model to
saved_models/audio_classification.keras
219/219              1s 4ms/step -
accuracy: 0.6229 - loss: 1.1275 - val_accuracy: 0.7046 - val_loss: 0.9340
Epoch 44/100
218/219              0s 4ms/step -
accuracy: 0.6358 - loss: 1.0847
Epoch 44: val_loss did not improve from 0.93400
219/219              1s 4ms/step -
accuracy: 0.6356 - loss: 1.0850 - val_accuracy: 0.6938 - val_loss: 0.9490
Epoch 45/100
214/219              0s 4ms/step -
accuracy: 0.6270 - loss: 1.1119
```

```
Epoch 45: val_loss did not improve from 0.93400
219/219            1s 4ms/step -
accuracy: 0.6269 - loss: 1.1121 - val_accuracy: 0.6909 - val_loss: 0.9390
Epoch 46/100
215/219            0s 4ms/step -
accuracy: 0.6235 - loss: 1.1065
Epoch 46: val_loss improved from 0.93400 to 0.91645, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6233 - loss: 1.1068 - val_accuracy: 0.7075 - val_loss: 0.9164
Epoch 47/100
214/219            0s 4ms/step -
accuracy: 0.6267 - loss: 1.1022
Epoch 47: val_loss did not improve from 0.91645
219/219            1s 5ms/step -
accuracy: 0.6267 - loss: 1.1021 - val_accuracy: 0.7029 - val_loss: 0.9284
Epoch 48/100
216/219            0s 4ms/step -
accuracy: 0.6279 - loss: 1.0919
Epoch 48: val_loss improved from 0.91645 to 0.91092, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6278 - loss: 1.0919 - val_accuracy: 0.7069 - val_loss: 0.9109
Epoch 49/100
218/219            0s 4ms/step -
accuracy: 0.6253 - loss: 1.1063
Epoch 49: val_loss did not improve from 0.91092
219/219            1s 4ms/step -
accuracy: 0.6253 - loss: 1.1062 - val_accuracy: 0.7149 - val_loss: 0.9274
Epoch 50/100
205/219            0s 4ms/step -
accuracy: 0.6371 - loss: 1.0648
Epoch 50: val_loss did not improve from 0.91092
219/219            1s 4ms/step -
accuracy: 0.6368 - loss: 1.0660 - val_accuracy: 0.7018 - val_loss: 0.9282
Epoch 51/100
217/219            0s 4ms/step -
accuracy: 0.6307 - loss: 1.0786
Epoch 51: val_loss did not improve from 0.91092
219/219            1s 4ms/step -
accuracy: 0.6307 - loss: 1.0788 - val_accuracy: 0.7115 - val_loss: 0.9130
Epoch 52/100
213/219            0s 4ms/step -
accuracy: 0.6437 - loss: 1.0602
Epoch 52: val_loss improved from 0.91092 to 0.88484, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6435 - loss: 1.0606 - val_accuracy: 0.7201 - val_loss: 0.8848
```

```
Epoch 53/100
218/219            0s 4ms/step -
accuracy: 0.6328 - loss: 1.0863
Epoch 53: val_loss improved from 0.88484 to 0.88056, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6328 - loss: 1.0863 - val_accuracy: 0.7212 - val_loss: 0.8806
Epoch 54/100
217/219            0s 4ms/step -
accuracy: 0.6346 - loss: 1.0838
Epoch 54: val_loss improved from 0.88056 to 0.87681, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6347 - loss: 1.0837 - val_accuracy: 0.7258 - val_loss: 0.8768
Epoch 55/100
216/219            0s 4ms/step -
accuracy: 0.6533 - loss: 1.0341
Epoch 55: val_loss did not improve from 0.87681
219/219            1s 4ms/step -
accuracy: 0.6531 - loss: 1.0346 - val_accuracy: 0.7195 - val_loss: 0.8804
Epoch 56/100
218/219            0s 4ms/step -
accuracy: 0.6535 - loss: 1.0606
Epoch 56: val_loss did not improve from 0.87681
219/219            1s 5ms/step -
accuracy: 0.6535 - loss: 1.0606 - val_accuracy: 0.7155 - val_loss: 0.8780
Epoch 57/100
216/219            0s 4ms/step -
accuracy: 0.6537 - loss: 1.0370
Epoch 57: val_loss improved from 0.87681 to 0.85186, saving model to
saved_models/audio_classification.keras
219/219            1s 4ms/step -
accuracy: 0.6536 - loss: 1.0372 - val_accuracy: 0.7218 - val_loss: 0.8519
Epoch 58/100
213/219            0s 4ms/step -
accuracy: 0.6497 - loss: 1.0443
Epoch 58: val_loss improved from 0.85186 to 0.84789, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6498 - loss: 1.0443 - val_accuracy: 0.7321 - val_loss: 0.8479
Epoch 59/100
210/219            0s 5ms/step -
accuracy: 0.6489 - loss: 1.0164
Epoch 59: val_loss did not improve from 0.84789
219/219            1s 6ms/step -
accuracy: 0.6490 - loss: 1.0169 - val_accuracy: 0.7315 - val_loss: 0.8563
Epoch 60/100
219/219            0s 5ms/step -
```

```
accuracy: 0.6572 - loss: 1.0167
Epoch 60: val_loss improved from 0.84789 to 0.84457, saving model to
saved_models/audio_classification.keras
219/219            1s 6ms/step -
accuracy: 0.6572 - loss: 1.0168 - val_accuracy: 0.7327 - val_loss: 0.8446
Epoch 61/100
212/219            0s 5ms/step -
accuracy: 0.6448 - loss: 1.0392
Epoch 61: val_loss improved from 0.84457 to 0.82949, saving model to
saved_models/audio_classification.keras
219/219            1s 6ms/step -
accuracy: 0.6450 - loss: 1.0392 - val_accuracy: 0.7333 - val_loss: 0.8295
Epoch 62/100
219/219            0s 4ms/step -
accuracy: 0.6577 - loss: 1.0201
Epoch 62: val_loss did not improve from 0.82949
219/219            1s 5ms/step -
accuracy: 0.6576 - loss: 1.0201 - val_accuracy: 0.7401 - val_loss: 0.8306
Epoch 63/100
214/219            0s 5ms/step -
accuracy: 0.6702 - loss: 0.9783
Epoch 63: val_loss improved from 0.82949 to 0.81374, saving model to
saved_models/audio_classification.keras
219/219            1s 6ms/step -
accuracy: 0.6700 - loss: 0.9789 - val_accuracy: 0.7481 - val_loss: 0.8137
Epoch 64/100
210/219            0s 4ms/step -
accuracy: 0.6647 - loss: 1.0075
Epoch 64: val_loss did not improve from 0.81374
219/219            1s 5ms/step -
accuracy: 0.6643 - loss: 1.0090 - val_accuracy: 0.7390 - val_loss: 0.8605
Epoch 65/100
211/219            0s 4ms/step -
accuracy: 0.6569 - loss: 0.9924
Epoch 65: val_loss did not improve from 0.81374
219/219            1s 5ms/step -
accuracy: 0.6569 - loss: 0.9938 - val_accuracy: 0.7172 - val_loss: 0.8619
Epoch 66/100
211/219            0s 4ms/step -
accuracy: 0.6478 - loss: 1.0094
Epoch 66: val_loss did not improve from 0.81374
219/219            1s 5ms/step -
accuracy: 0.6482 - loss: 1.0095 - val_accuracy: 0.7424 - val_loss: 0.8158
Epoch 67/100
219/219            0s 4ms/step -
accuracy: 0.6608 - loss: 1.0243
Epoch 67: val_loss did not improve from 0.81374
219/219            1s 5ms/step -
```

```
accuracy: 0.6608 - loss: 1.0242 - val_accuracy: 0.7315 - val_loss: 0.8446
Epoch 68/100
213/219            0s 4ms/step -
accuracy: 0.6679 - loss: 0.9780
Epoch 68: val_loss did not improve from 0.81374
219/219            1s 6ms/step -
accuracy: 0.6676 - loss: 0.9788 - val_accuracy: 0.7481 - val_loss: 0.8197
Epoch 69/100
218/219            0s 4ms/step -
accuracy: 0.6674 - loss: 1.0097
Epoch 69: val_loss improved from 0.81374 to 0.80547, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6674 - loss: 1.0096 - val_accuracy: 0.7464 - val_loss: 0.8055
Epoch 70/100
215/219            0s 4ms/step -
accuracy: 0.6702 - loss: 0.9970
Epoch 70: val_loss did not improve from 0.80547
219/219            1s 4ms/step -
accuracy: 0.6700 - loss: 0.9973 - val_accuracy: 0.7407 - val_loss: 0.8100
Epoch 71/100
214/219            0s 4ms/step -
accuracy: 0.6664 - loss: 1.0069
Epoch 71: val_loss improved from 0.80547 to 0.79438, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6664 - loss: 1.0067 - val_accuracy: 0.7550 - val_loss: 0.7944
Epoch 72/100
211/219            0s 4ms/step -
accuracy: 0.6674 - loss: 0.9923
Epoch 72: val_loss improved from 0.79438 to 0.78606, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6672 - loss: 0.9927 - val_accuracy: 0.7590 - val_loss: 0.7861
Epoch 73/100
214/219            0s 4ms/step -
accuracy: 0.6719 - loss: 0.9730
Epoch 73: val_loss did not improve from 0.78606
219/219            1s 5ms/step -
accuracy: 0.6718 - loss: 0.9733 - val_accuracy: 0.7562 - val_loss: 0.8070
Epoch 74/100
214/219            0s 4ms/step -
accuracy: 0.6707 - loss: 0.9859
Epoch 74: val_loss did not improve from 0.78606
219/219            1s 5ms/step -
accuracy: 0.6707 - loss: 0.9863 - val_accuracy: 0.7459 - val_loss: 0.8165
Epoch 75/100
213/219            0s 4ms/step -
```

```
accuracy: 0.6702 - loss: 0.9837
Epoch 75: val_loss did not improve from 0.78606
219/219            1s 5ms/step -
accuracy: 0.6703 - loss: 0.9839 - val_accuracy: 0.7424 - val_loss: 0.8089
Epoch 76/100
212/219            0s 4ms/step -
accuracy: 0.6723 - loss: 0.9896
Epoch 76: val_loss improved from 0.78606 to 0.77539, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6721 - loss: 0.9898 - val_accuracy: 0.7682 - val_loss: 0.7754
Epoch 77/100
219/219            0s 4ms/step -
accuracy: 0.6745 - loss: 0.9924
Epoch 77: val_loss did not improve from 0.77539
219/219            1s 4ms/step -
accuracy: 0.6745 - loss: 0.9924 - val_accuracy: 0.7573 - val_loss: 0.7833
Epoch 78/100
219/219            0s 5ms/step -
accuracy: 0.6707 - loss: 0.9813
Epoch 78: val_loss did not improve from 0.77539
219/219            1s 6ms/step -
accuracy: 0.6707 - loss: 0.9813 - val_accuracy: 0.7539 - val_loss: 0.7961
Epoch 79/100
219/219            0s 4ms/step -
accuracy: 0.6725 - loss: 0.9738
Epoch 79: val_loss did not improve from 0.77539
219/219            1s 5ms/step -
accuracy: 0.6725 - loss: 0.9738 - val_accuracy: 0.7562 - val_loss: 0.7784
Epoch 80/100
219/219            0s 4ms/step -
accuracy: 0.6722 - loss: 0.9770
Epoch 80: val_loss improved from 0.77539 to 0.77406, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6721 - loss: 0.9771 - val_accuracy: 0.7647 - val_loss: 0.7741
Epoch 81/100
212/219            0s 4ms/step -
accuracy: 0.6696 - loss: 0.9826
Epoch 81: val_loss did not improve from 0.77406
219/219            1s 5ms/step -
accuracy: 0.6695 - loss: 0.9833 - val_accuracy: 0.7613 - val_loss: 0.7925
Epoch 82/100
219/219            0s 4ms/step -
accuracy: 0.6635 - loss: 0.9739
Epoch 82: val_loss did not improve from 0.77406
219/219            1s 5ms/step -
accuracy: 0.6635 - loss: 0.9740 - val_accuracy: 0.7613 - val_loss: 0.7870
```

```
Epoch 83/100
209/219            0s 4ms/step -
accuracy: 0.6690 - loss: 0.9730
Epoch 83: val_loss improved from 0.77406 to 0.76723, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6692 - loss: 0.9726 - val_accuracy: 0.7693 - val_loss: 0.7672
Epoch 84/100
214/219            0s 4ms/step -
accuracy: 0.6788 - loss: 0.9721
Epoch 84: val_loss did not improve from 0.76723
219/219            1s 4ms/step -
accuracy: 0.6787 - loss: 0.9724 - val_accuracy: 0.7533 - val_loss: 0.7781
Epoch 85/100
214/219            0s 4ms/step -
accuracy: 0.6795 - loss: 0.9531
Epoch 85: val_loss did not improve from 0.76723
219/219            1s 5ms/step -
accuracy: 0.6795 - loss: 0.9533 - val_accuracy: 0.7613 - val_loss: 0.7754
Epoch 86/100
218/219            0s 5ms/step -
accuracy: 0.6775 - loss: 0.9716
Epoch 86: val_loss did not improve from 0.76723
219/219            1s 6ms/step -
accuracy: 0.6774 - loss: 0.9716 - val_accuracy: 0.7584 - val_loss: 0.7777
Epoch 87/100
219/219            0s 4ms/step -
accuracy: 0.6937 - loss: 0.9829
Epoch 87: val_loss did not improve from 0.76723
219/219            1s 5ms/step -
accuracy: 0.6936 - loss: 0.9829 - val_accuracy: 0.7567 - val_loss: 0.7774
Epoch 88/100
208/219            0s 4ms/step -
accuracy: 0.6849 - loss: 0.9365
Epoch 88: val_loss did not improve from 0.76723
219/219            1s 5ms/step -
accuracy: 0.6846 - loss: 0.9377 - val_accuracy: 0.7590 - val_loss: 0.7728
Epoch 89/100
218/219            0s 4ms/step -
accuracy: 0.6701 - loss: 0.9948
Epoch 89: val_loss improved from 0.76723 to 0.75564, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6701 - loss: 0.9946 - val_accuracy: 0.7773 - val_loss: 0.7556
Epoch 90/100
206/219            0s 4ms/step -
accuracy: 0.6869 - loss: 0.9300
Epoch 90: val_loss did not improve from 0.75564
```

```
219/219            1s 5ms/step -
accuracy: 0.6869 - loss: 0.9302 - val_accuracy: 0.7682 - val_loss: 0.7705
Epoch 91/100
210/219            0s 4ms/step -
accuracy: 0.6906 - loss: 0.9265
Epoch 91: val_loss did not improve from 0.75564
219/219            1s 5ms/step -
accuracy: 0.6903 - loss: 0.9276 - val_accuracy: 0.7579 - val_loss: 0.7638
Epoch 92/100
219/219            0s 4ms/step -
accuracy: 0.6871 - loss: 0.9495
Epoch 92: val_loss improved from 0.75564 to 0.75035, saving model to
saved_models/audio_classification.keras
219/219            1s 5ms/step -
accuracy: 0.6871 - loss: 0.9495 - val_accuracy: 0.7722 - val_loss: 0.7504
Epoch 93/100
216/219            0s 4ms/step -
accuracy: 0.6747 - loss: 0.9470
Epoch 93: val_loss did not improve from 0.75035
219/219            1s 5ms/step -
accuracy: 0.6748 - loss: 0.9472 - val_accuracy: 0.7624 - val_loss: 0.7758
Epoch 94/100
211/219            0s 5ms/step -
accuracy: 0.6710 - loss: 0.9645
Epoch 94: val_loss did not improve from 0.75035
219/219            1s 6ms/step -
accuracy: 0.6711 - loss: 0.9642 - val_accuracy: 0.7607 - val_loss: 0.7782
Epoch 95/100
207/219            0s 4ms/step -
accuracy: 0.6948 - loss: 0.9184
Epoch 95: val_loss improved from 0.75035 to 0.74516, saving model to
saved_models/audio_classification.keras
219/219            2s 5ms/step -
accuracy: 0.6944 - loss: 0.9204 - val_accuracy: 0.7676 - val_loss: 0.7452
Epoch 96/100
219/219            0s 4ms/step -
accuracy: 0.6902 - loss: 0.9205
Epoch 96: val_loss did not improve from 0.74516
219/219            1s 5ms/step -
accuracy: 0.6901 - loss: 0.9205 - val_accuracy: 0.7670 - val_loss: 0.7540
Epoch 97/100
219/219            0s 4ms/step -
accuracy: 0.6808 - loss: 0.9520
Epoch 97: val_loss did not improve from 0.74516
219/219            1s 5ms/step -
accuracy: 0.6809 - loss: 0.9519 - val_accuracy: 0.7682 - val_loss: 0.7531
Epoch 98/100
209/219            0s 4ms/step -
```

```
accuracy: 0.6763 - loss: 0.9709
Epoch 98: val_loss did not improve from 0.74516
219/219              1s 5ms/step -
accuracy: 0.6764 - loss: 0.9703 - val_accuracy: 0.7544 - val_loss: 0.7801
Epoch 99/100
219/219              0s 4ms/step -
accuracy: 0.6801 - loss: 0.9580
Epoch 99: val_loss did not improve from 0.74516
219/219              1s 5ms/step -
accuracy: 0.6801 - loss: 0.9580 - val_accuracy: 0.7670 - val_loss: 0.7528
Epoch 100/100
218/219              0s 4ms/step -
accuracy: 0.6989 - loss: 0.9378
Epoch 100: val_loss did not improve from 0.74516
219/219              1s 5ms/step -
accuracy: 0.6988 - loss: 0.9378 - val_accuracy: 0.7710 - val_loss: 0.7494
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

```
Training completed in time:  0:01:54.883922
0.7710360884666443
```

[17]:
```
metadata['class'].unique()
```

[17]:
```
array(['dog_bark', 'children_playing', 'car_horn', 'air_conditioner',
       'street_music', 'gun_shot', 'siren', 'engine_idling', 'jackhammer',
       'drilling'], dtype=object)
```

7.3 Testing some test data

1.preprocess the new audio data

2.predict the classes

3.invere transform your predicted label

[54]:
```
# Load audio file and extract MFCC features
filename = "UrbanSound8K/carhorn.wav"
audio, sample_rate = librosa.load(filename, res_type='kaiser_fast')
mfccs_features = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=40)
mfccs_scaled_features = np.mean(mfccs_features.T, axis=0)

# Print extracted features
print("MFCCs scaled features:", mfccs_scaled_features)
mfccs_scaled_features = mfccs_scaled_features.reshape(1, -1)
print("MFCCs scaled features (reshaped):", mfccs_scaled_features)
```

```
print("MFCCs scaled features shape:", mfccs_scaled_features.shape)

# Predict class
predicted_label = np.argmax(model.predict(mfccs_scaled_features), axis=-1)
print("Predicted label:", predicted_label)

# Map predicted label to class name
prediction_class = labelencoder.inverse_transform(predicted_label)
print("Prediction class:", prediction_class)
```

```
MFCCs scaled features: [-142.23528      64.313705    -65.76922      11.390171
-32.892128
   -6.1799746    -9.712353      1.99971     -22.375465     10.463358
  -24.464956     -5.90166      -5.4398284     9.6334715     5.237608
   26.032831      4.832415      4.7442617   -11.672217      6.153628
   -6.398495     -7.67534     -15.409541      1.8400229     2.262191
   -5.4980874    -9.390764     18.122797     21.13855      -6.1640983
  -27.362368      0.46142736   25.696905     11.829148     -0.5846754
    3.9731686    -9.98214     -31.751429     -2.2294564    29.76882   ]
MFCCs scaled features (reshaped): [[-142.23528      64.313705    -65.76922
11.390171    -32.892128
    -6.1799746    -9.712353      1.99971     -22.375465     10.463358
   -24.464956     -5.90166      -5.4398284     9.6334715     5.237608
    26.032831      4.832415      4.7442617   -11.672217      6.153628
    -6.398495     -7.67534     -15.409541      1.8400229     2.262191
    -5.4980874    -9.390764     18.122797     21.13855      -6.1640983
   -27.362368      0.46142736   25.696905     11.829148     -0.5846754
     3.9731686    -9.98214     -31.751429     -2.2294564    29.76882   ]]
MFCCs scaled features shape: (1, 40)
1/1              0s 28ms/step
Predicted label: [1]
Prediction class: ['car_horn']
```

[55]:
```
#using function but not working
def features_extractor(filename):
    audio, sample_rate = librosa.load(file_name, res_type='kaiser_fast')
    mfccs_features = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=40)
    mfccs_scaled_features = np.mean(mfccs_features.T,axis=0)
    return mfccs_scaled_features

filename="UrbanSound8K/dog_bark.wav"

prediction_feature=features_extractor(filename)
prediction_feature=prediction_feature.reshape(1,-1)

predicted_label = np.argmax(model.predict(prediction_feature), axis=-1)
print("Predicted label:", predicted_label)
```

```
# Map predicted label to class name
prediction_class = labelencoder.inverse_transform(predicted_label)
print("Prediction class:", prediction_class)
```

```
1/1              0s 57ms/step
Predicted label: [1]
Prediction class: ['car_horn']
```

8 Handling Imbalanced Classification Using Neural Network

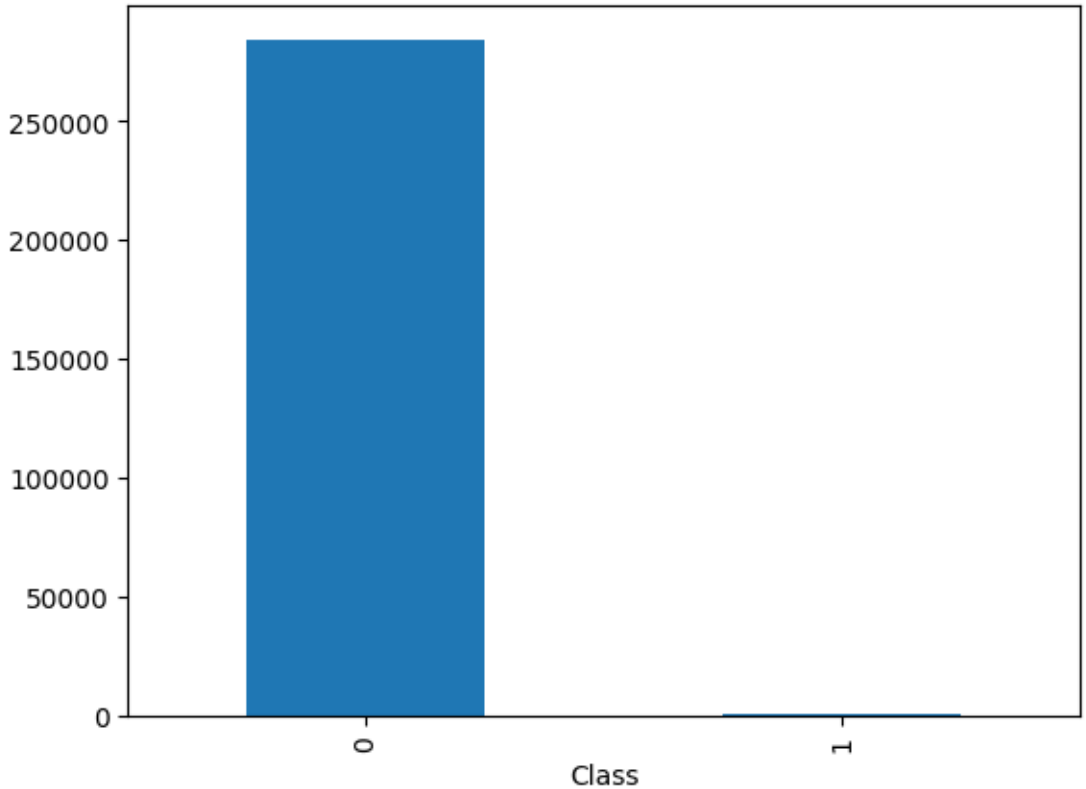
[2]:
```
import pandas as pd
df = pd.read_csv('creditcard.csv')
print(df.head())
print(df['Class'].value_counts())
pd.value_counts(df['Class']).plot.bar()
```

```
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  …       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  … -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  … -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  …  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  … -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  … -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
Class
0    284315
1       492
Name: count, dtype: int64
```

```
/tmp/ipykernel_6466/3243127651.py:5: FutureWarning: pandas.value_counts is
deprecated and will be removed in a future version. Use
pd.Series(obj).value_counts() instead.
```

```
pd.value_counts(df['Class']).plot.bar()
```

[2]: <Axes: xlabel='Class'>



8.0.1 Initially

step 1- we will create an ANN solve this problem

step 2- we will use some weights handling techniquesto solve yhis imbalanced data

[12]:
```
print("Columns:",len(df.columns))
X=df.iloc[:,1:30]
y=df.iloc[:,-1]
print(X.head())
print(y.head())
```

```
Columns: 31
         V1        V2        V3        V4        V5        V6        V7  \
0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941
```

```
         V8        V9       V10  …       V20       V21       V22       V23  \
0  0.098698  0.363787  0.090794  …  0.251412 -0.018307  0.277838 -0.110474
1  0.085102 -0.255425 -0.166974  … -0.069083 -0.225775 -0.638672  0.101288
2  0.247676 -1.514654  0.207643  …  0.524980  0.247998  0.771679  0.909412
3  0.377436 -1.387024 -0.054952  … -0.208038 -0.108300  0.005274 -0.190321
4 -0.270533  0.817739  0.753074  …  0.408542 -0.009431  0.798278 -0.137458

        V24       V25       V26       V27       V28  Amount
0  0.066928  0.128539 -0.189115  0.133558 -0.021053  149.62
1 -0.339846  0.167170  0.125895 -0.008983  0.014724    2.69
2 -0.689281 -0.327642 -0.139097 -0.055353 -0.059752  378.66
3 -1.175575  0.647376 -0.221929  0.062723  0.061458  123.50
4  0.141267 -0.206010  0.502292  0.219422  0.215153   69.99

[5 rows x 29 columns]
0    0
1    0
2    0
3    0
4    0
Name: Class, dtype: int64
```

[14]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.
 ↪30,random_state=42)
y_train.value_counts()
```

[14]:
```
Class
0    199008
1       356
Name: count, dtype: int64
```

[27]:
```
import tensorflow
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
print(len(X.columns))
n_inputs=29

#define model

model=Sequential()
model.
 ↪add(Dense(50,input_dim=n_inputs,activation='relu',kernel_initializer='he_uniform'))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam')
model.fit(X_train,y_train,epochs=10)
```

```
29

/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-01-02 21:59:04.380752: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
23126224 exceeds 10% of free system memory.

Epoch 1/10
6231/6231              13s 2ms/step -
loss: 0.4853
Epoch 2/10
6231/6231              13s 2ms/step -
loss: 0.0101
Epoch 3/10
6231/6231              13s 2ms/step -
loss: 0.0085
Epoch 4/10
6231/6231              12s 2ms/step -
loss: 0.0100
Epoch 5/10
6231/6231              21s 2ms/step -
loss: 0.0117
Epoch 6/10
6231/6231              13s 2ms/step -
loss: 0.0062
Epoch 7/10
6231/6231              12s 2ms/step -
loss: 0.0089
Epoch 8/10
6231/6231              14s 2ms/step -
loss: 0.0058
Epoch 9/10
6231/6231              13s 2ms/step -
loss: 0.0063
Epoch 10/10
6231/6231              13s 2ms/step -
loss: 0.0079
```

[27]:
```
<keras.src.callbacks.history.History at 0x7f425f492860>
```

[28]:
```
from  sklearn.metrics import roc_auc_score
y_pred=model.predict(X_test)

print(roc_auc_score(y_test,y_pred))
```

```
2671/2671              4s 1ms/step
0.9452761100220036
```

8.0.2 weighted Neural Network with Keras

Define weights Fit the model with those specific weights

[32]:
```
weights_assigned={0:1,1:550}

model=Sequential()
model.
 ↪add(Dense(50,input_dim=n_inputs,activation='relu',kernel_initializer='he_uniform'))
model.add(Dense(1,activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam')
model.fit(X_train,y_train,class_weight=weights_assigned,epochs=10)
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[32], line 7
      5 model.add(Dense(1,activation='sigmoid'))
      6 model.compile(loss='binary_crossentropy',optimizer='adam')
----> 7 model.fit(X_train,y_train,class_weight=weights_assigned,epochs=10)

File ~/.local/lib/python3.10/site-packages/keras/src/utils/traceback_utils.py:
 ↪122, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    119     filtered_tb = _process_traceback_frames(e.__traceback__)
    120     # To get the full stack trace, call:
    121     # `keras.config.disable_traceback_filtering()`
--> 122     raise e.with_traceback(filtered_tb) from None
    123 finally:
    124     del filtered_tb

File ~/.local/lib/python3.10/site-packages/pandas/core/series.py:1121, in Series.
 ↪__getitem__(self, key)
   1118     return self._values[key]
   1120 elif key_is_scalar:
-> 1121     return self._get_value(key)
   1123 # Convert generator to list before going through hashable part
   1124 # (We will iterate through the generator there to check for slices)
   1125 if is_iterator(key):
```

```
File ~/.local/lib/python3.10/site-packages/pandas/core/series.py:1237, in Series.
↪_get_value(self, label, takeable)
   1234     return self._values[label]
   1236 # Similar to Index.get_value, but we do not fall back to positional
-> 1237 loc = self.index.get_loc(label)
   1239 if is_integer(loc):
   1240     return self._values[loc]

File ~/.local/lib/python3.10/site-packages/pandas/core/indexes/base.py:3812, in
↪Index.get_loc(self, key)
   3807     if isinstance(casted_key, slice) or (
   3808         isinstance(casted_key, abc.Iterable)
   3809         and any(isinstance(x, slice) for x in casted_key)
   3810     ):
   3811         raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will raise
   3815     #  InvalidIndexError. Otherwise we fall through and re-raise
   3816     #  the TypeError.
   3817     self._check_indexing_error(key)

KeyError: 0
```

[33]:
```
y_predict=model.predict(X_test)
roc_auc_score(y_test,y_pred)
```

```
2671/2671              4s 1ms/step
```

[33]: 0.9452761100220036

9 Convolutional Neural Network using SVM as Final Layer

[2]:
```
from tensorflow.compat.v1 import ConfigProto
from tensorflow.compat.v1 import InteractiveSession

config=ConfigProto()
config.gpu_options.per_process_gpu_memory_fraction=0.5
config.gpu_options.allow_growth=True
session=InteractiveSession(config=config)
```

[14]:
```
#CNN
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

#1.preprocessing
```

```
train_datagen=ImageDataGenerator(rescale=1./255,
                                 shear_range=0.2,
                                 zoom_range=0.2,
                                 horizontal_flip=True)
training_set=train_datagen.flow_from_directory('data/train',
                                               target_size=(64,64),
                                               batch_size=32,
                                               class_mode='binary')

test_datagen=ImageDataGenerator(rescale=1./255)
test_set=test_datagen.flow_from_directory('data/validation',
                                          target_size=(64,64),
                                          batch_size=32,
                                          class_mode='binary')
```

```
Found 20 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
```

[15]:
```
from tensorflow.keras.layers import Conv2D,Dense
from tensorflow.keras.regularizers import l2


#2.build CNN
cnn=tf.keras.models.Sequential()
cnn.add(tf.keras.layers.
 ↪Conv2D(filters=32,padding='same',kernel_size=3,activation='relu',input_shape=[64,64,3]))
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2,strides=2))

cnn.add(tf.keras.layers.
 ↪Conv2D(filters=32,padding='same',kernel_size=3,activation='relu'))
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2,strides=2))

cnn.add(tf.keras.layers.Flatten())

cnn.add(tf.keras.layers.Dense(units=128,activation='relu'))


#output
cnn.add(Dense(1,kernel_regularizer=tf.keras.regularizers.l2(0.
 ↪01),activation='linear'))

#for multi classification
# cnn.add(Dense(2,kernel_regularizer=tf.keras.regularizers.l2(0.
 ↪01,activation='softmax'))
# cnn.compile(optimizer='adam',loss='squared_hinge',metrics=['accuracy'])
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[16]:
```
cnn.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 64, 64, 32)	896
max_pooling2d_7 (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_8 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d_8 (MaxPooling2D)	(None, 16, 16, 32)	0
flatten_3 (Flatten)	(None, 8192)	0
dense_6 (Dense)	(None, 128)	1,048,704
dense_7 (Dense)	(None, 1)	129

Total params: 1,058,977 (4.04 MB)

Trainable params: 1,058,977 (4.04 MB)

Non-trainable params: 0 (0.00 B)

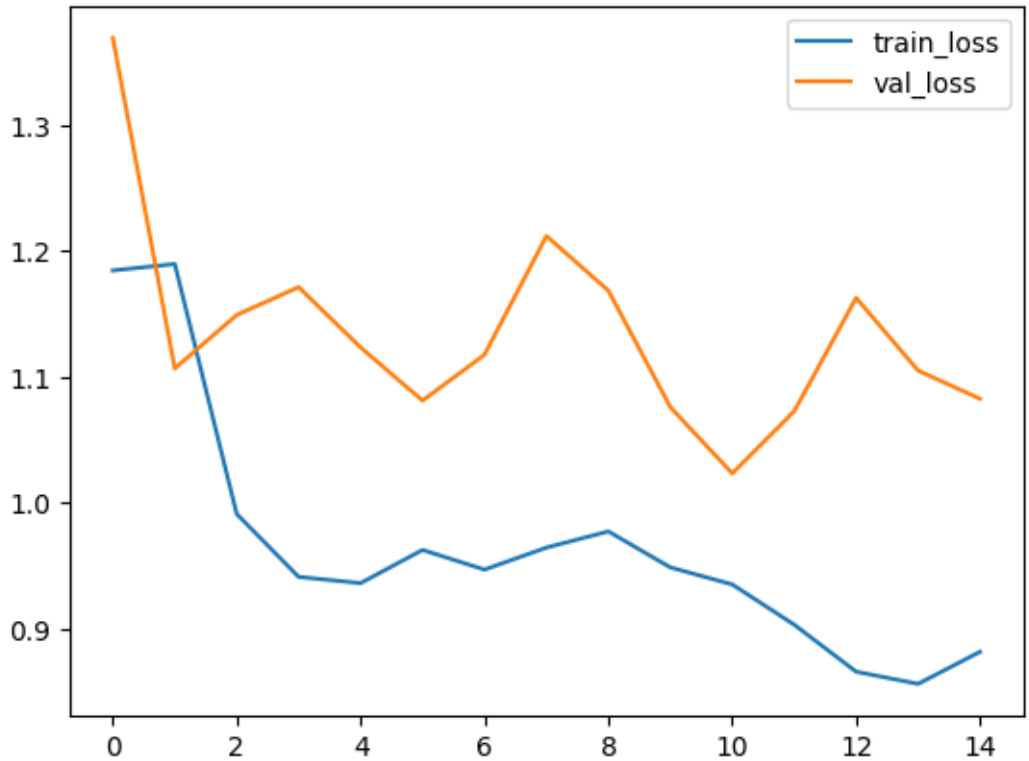
[10]:
```
#train cnn
cnn.compile(optimizer='adam',loss='hinge',metrics=['accuracy'])
r=cnn.fit(x=training_set,validation_data=test_set,epochs=15)
```

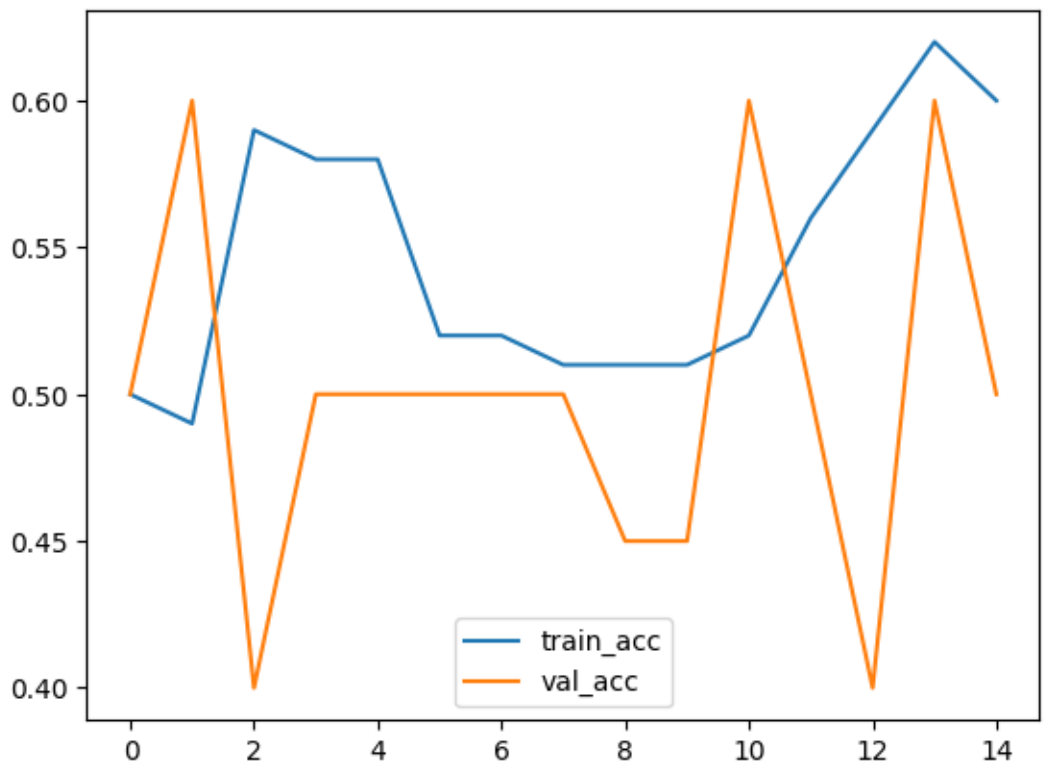
```
Epoch 1/15
```

```
/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122:
UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in
its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
```

```
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be
ignored.
  self._warn_if_super_not_called()
4/4              4s 301ms/step -
accuracy: 0.5196 - loss: 1.0878 - val_accuracy: 0.5000 - val_loss: 1.3693
Epoch 2/15
4/4              1s 171ms/step -
accuracy: 0.4989 - loss: 1.2449 - val_accuracy: 0.6000 - val_loss: 1.1068
Epoch 3/15
4/4              1s 157ms/step -
accuracy: 0.5696 - loss: 0.9890 - val_accuracy: 0.4000 - val_loss: 1.1493
Epoch 4/15
4/4              1s 179ms/step -
accuracy: 0.5996 - loss: 0.8860 - val_accuracy: 0.5000 - val_loss: 1.1714
Epoch 5/15
4/4              1s 164ms/step -
accuracy: 0.5768 - loss: 0.9278 - val_accuracy: 0.5000 - val_loss: 1.1235
Epoch 6/15
4/4              1s 165ms/step -
accuracy: 0.4755 - loss: 1.0566 - val_accuracy: 0.5000 - val_loss: 1.0813
Epoch 7/15
4/4              1s 167ms/step -
accuracy: 0.4955 - loss: 0.9478 - val_accuracy: 0.5000 - val_loss: 1.1177
Epoch 8/15
4/4              1s 164ms/step -
accuracy: 0.5253 - loss: 0.9497 - val_accuracy: 0.5000 - val_loss: 1.2120
Epoch 9/15
4/4              1s 203ms/step -
accuracy: 0.5295 - loss: 0.9187 - val_accuracy: 0.4500 - val_loss: 1.1686
Epoch 10/15
4/4              1s 163ms/step -
accuracy: 0.5288 - loss: 0.9582 - val_accuracy: 0.4500 - val_loss: 1.0762
Epoch 11/15
4/4              1s 159ms/step -
accuracy: 0.5132 - loss: 0.9344 - val_accuracy: 0.6000 - val_loss: 1.0235
Epoch 12/15
4/4              1s 162ms/step -
accuracy: 0.5826 - loss: 0.8922 - val_accuracy: 0.5000 - val_loss: 1.0728
Epoch 13/15
4/4              1s 159ms/step -
accuracy: 0.6141 - loss: 0.8787 - val_accuracy: 0.4000 - val_loss: 1.1629
Epoch 14/15
4/4              1s 160ms/step -
accuracy: 0.6158 - loss: 0.8822 - val_accuracy: 0.6000 - val_loss: 1.1052
Epoch 15/15
4/4              1s 178ms/step -
accuracy: 0.5960 - loss: 0.8159 - val_accuracy: 0.5000 - val_loss: 1.0827
```

[17]:
```
#plot the loss
import matplotlib.pyplot as plt
plt.plot(r.history['loss'],label='train_loss')
plt.plot(r.history['val_loss'],label= 'val_loss')
plt.legend()
plt.show()
plt.savefig('LossVal_loss')

#plot the accuracy
plt.plot(r.history['accuracy'],label='train_acc')
plt.plot(r.history['val_accuracy'],label= 'val_acc')
plt.legend()
plt.show()
plt.savefig('AccVal_acc')
```






```
<Figure size 640x480 with 0 Axes>
```

[18]:
```
cnn.save('reg_catordog.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

[19]:
```
from tensorflow.keras.models import load_model
model=load_model('reg_catordog.h5')
model.summary()
```

```
WARNING:absl:No training configuration found in the save file, so the model was
*not* compiled. Compile it manually.
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 64, 64, 32)	896

```
max_pooling2d_7 (MaxPooling2D)    (None, 32, 32, 32)                  0

conv2d_8 (Conv2D)                 (None, 32, 32, 32)              9,248

max_pooling2d_8 (MaxPooling2D)    (None, 16, 16, 32)                  0

flatten_3 (Flatten)               (None, 8192)                        0

dense_6 (Dense)                   (None, 128)                 1,048,704

dense_7 (Dense)                   (None, 1)                         129
```

Total params: 1,058,977 (4.04 MB)

Trainable params: 1,058,977 (4.04 MB)

Non-trainable params: 0 (0.00 B)

[21]:
```python
#making single prediction
import numpy as np
from tensorflow.keras.preprocessing import image
test_img=image.load_img('cat_or_dog3.png',target_size=[64,64])
test_img=image.img_to_array(test_img)
test_img=test_img/255
test_img=np.expand_dims(test_img,axis=0)
result=cnn.predict(test_img)
result
```

```
1/1              0s 43ms/step
```

[21]:
```
array([[-0.01631583]], dtype=float32)
```

[22]:
```python
if result[0]<0:
    print('dog')
else:
    print('cat')
```

```
dog
```

10 Gradio interface

[13]:
```
!pip install gradio
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: gradio in /home/rgukt/.local/lib/python3.10/site-
packages (5.9.1)
Requirement already satisfied: uvicorn>=0.14.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.34.0)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.1.6)
Requirement already satisfied: huggingface-hub>=0.25.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.27.0)
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.13.2)
Requirement already satisfied: ffmpy in /home/rgukt/.local/lib/python3.10/site-
packages (from gradio) (0.5.0)
Requirement already satisfied: httpx>=0.24.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.27.0)
Requirement already satisfied: jinja2<4.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (3.1.4)
Requirement already satisfied: orjson~=3.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (3.10.13)
Requirement already satisfied: ruff>=0.2.2 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.8.4)
Requirement already satisfied: markupsafe~=2.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (2.1.5)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.115.6)
Requirement already satisfied: aiofiles<24.0,>=22.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (23.2.1)
Requirement already satisfied: pydub in /home/rgukt/.local/lib/python3.10/site-
packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.0.20)
Requirement already satisfied: typer<1.0,>=0.12 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.15.1)
Requirement already satisfied: packaging in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (24.1)
Requirement already satisfied: anyio<5.0,>=3.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (4.4.0)
Requirement already satisfied: pandas<3.0,>=1.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (2.2.3)
Requirement already satisfied: pydantic>=2.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (2.10.4)
Requirement already satisfied: semantic-version~=2.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (2.10.0)
Requirement already satisfied: gradio-client==1.5.2 in
```

```
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (1.5.2)
Requirement already satisfied: numpy<3.0,>=1.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (1.26.4)
Requirement already satisfied: typing-extensions~=4.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (4.12.2)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/lib/python3/dist-
packages (from gradio) (9.0.1)
Requirement already satisfied: starlette<1.0,>=0.40.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio) (0.41.3)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/lib/python3/dist-
packages (from gradio) (5.4.1)
Requirement already satisfied: fsspec in /home/rgukt/.local/lib/python3.10/site-
packages (from gradio-client==1.5.2->gradio) (2024.12.0)
Requirement already satisfied: websockets<15.0,>=10.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from gradio-
client==1.5.2->gradio) (14.1)
Requirement already satisfied: exceptiongroup>=1.0.2 in
/home/rgukt/.local/lib/python3.10/site-packages (from anyio<5.0,>=3.0->gradio)
(1.2.1)
Requirement already satisfied: sniffio>=1.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from anyio<5.0,>=3.0->gradio)
(1.3.1)
Requirement already satisfied: idna>=2.8 in /usr/lib/python3/dist-packages (from
anyio<5.0,>=3.0->gradio) (3.3)
Requirement already satisfied: certifi in
/home/rgukt/.local/lib/python3.10/site-packages (from httpx>=0.24.1->gradio)
(2024.8.30)
Requirement already satisfied: httpcore==1.* in
/home/rgukt/.local/lib/python3.10/site-packages (from httpx>=0.24.1->gradio)
(1.0.5)
Requirement already satisfied: h11<0.15,>=0.13 in
/home/rgukt/.local/lib/python3.10/site-packages (from
httpcore==1.*->httpx>=0.24.1->gradio) (0.14.0)
Requirement already satisfied: tqdm>=4.42.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from huggingface-
hub>=0.25.1->gradio) (4.66.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-
packages (from huggingface-hub>=0.25.1->gradio) (3.16.0)
Requirement already satisfied: requests in
/home/rgukt/.local/lib/python3.10/site-packages (from huggingface-
hub>=0.25.1->gradio) (2.32.3)
Requirement already satisfied: pytz>=2020.1 in /usr/lib/python3/dist-packages
(from pandas<3.0,>=1.0->gradio) (2022.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
/home/rgukt/.local/lib/python3.10/site-packages (from pandas<3.0,>=1.0->gradio)
(2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in
/home/rgukt/.local/lib/python3.10/site-packages (from pandas<3.0,>=1.0->gradio)
```

```
(2024.1)
Requirement already satisfied: annotated-types>=0.6.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from pydantic>=2.0->gradio)
(0.7.0)
Requirement already satisfied: pydantic-core==2.27.2 in
/home/rgukt/.local/lib/python3.10/site-packages (from pydantic>=2.0->gradio)
(2.27.2)
Requirement already satisfied: click>=8.0.0 in /usr/lib/python3/dist-packages
(from typer<1.0,>=0.12->gradio) (8.0.3)
Requirement already satisfied: rich>=10.11.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from typer<1.0,>=0.12->gradio)
(13.8.1)
Requirement already satisfied: shellingham>=1.3.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from typer<1.0,>=0.12->gradio)
(1.5.4)
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from
python-dateutil>=2.8.2->pandas<3.0,>=1.0->gradio) (1.16.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from
rich>=10.11.0->typer<1.0,>=0.12->gradio) (2.18.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/home/rgukt/.local/lib/python3.10/site-packages (from
rich>=10.11.0->typer<1.0,>=0.12->gradio) (3.0.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/lib/python3/dist-
packages (from requests->huggingface-hub>=0.25.1->gradio) (1.26.5)
Requirement already satisfied: charset-normalizer<4,>=2 in
/home/rgukt/.local/lib/python3.10/site-packages (from requests->huggingface-
hub>=0.25.1->gradio) (3.3.2)
Requirement already satisfied: mdurl~=0.1 in
/home/rgukt/.local/lib/python3.10/site-packages (from markdown-it-
py>=2.2.0->rich>=10.11.0->typer<1.0,>=0.12->gradio) (0.1.2)
```

[56]:

```python
import gradio as gr

def greet(name):
    return 'Hello'+ name +"!"

iface=gr.Interface(fn=greet,inputs="text",outputs="text")
iface.launch()
```

```
* Running on local URL:  http://127.0.0.1:7860

To create a public link, set `share=True` in `launch()`.

<IPython.core.display.HTML object>
```

[56]:

[57]:
```
import gradio as gr
import tensorflow as tf
import numpy as np
import requests

#load the model
inception_net=tf.keras.applications.InceptionV3()

#human readable labels for imagenet
response=requests.get("https://git.io/JJKYN")
labels=response.text.split("\n")

def classify_image(inp):
    # inp=inp.reshape((-1,229,229,3))
    # inp=tf.keras.applications.inception_v3.preprocess_input(inp)
    inp = tf.image.resize(inp, (299, 299))  # Resize to 299x299
    inp = np.expand_dims(inp, axis=0)  # Add batch dimension
    inp = tf.keras.applications.inception_v3.preprocess_input(inp)  #␣
 ↪Preprocess the input

    prediction=inception_net.predict(inp).flatten()
    return {labels[i]:float(prediction[i]) for i in range(1000)}

image=gr.Image(type='numpy')
label=gr.Label(num_top_classes=3)

gr.Interface(fn=classify_image,inputs=image,outputs=label).launch()
```

```
* Running on local URL:  http://127.0.0.1:7861

To create a public link, set `share=True` in `launch()`.
```

```
<IPython.core.display.HTML object>
```

[57]:

```
Traceback (most recent call last):
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/queueing.py",
line 625, in process_events
    response = await route_utils.call_process_api(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/route_utils.py",
line 322, in call_process_api
    output = await app.get_blocks().process_api(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/blocks.py", line
2047, in process_api
    result = await self.call_function(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/blocks.py", line
1594, in call_function
```

```
    prediction = await anyio.to_thread.run_sync(  # type: ignore
  File "/home/rgukt/.local/lib/python3.10/site-packages/anyio/to_thread.py",
line 56, in run_sync
    return await get_async_backend().run_sync_in_worker_thread(
  File "/home/rgukt/.local/lib/python3.10/site-
packages/anyio/_backends/_asyncio.py", line 2177, in run_sync_in_worker_thread
    return await future
  File "/home/rgukt/.local/lib/python3.10/site-
packages/anyio/_backends/_asyncio.py", line 859, in run
    result = context.run(func, *args)
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/utils.py", line
869, in wrapper
    response = f(*args, **kwargs)
  File "/tmp/ipykernel_4998/1726201170.py", line 18, in classify_image
    inp = tf.keras.applications.inception_v3.preprocess_input(inp)  # Preprocess
the input
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/inception_v3.py", line 427, in preprocess_input
    return imagenet_utils.preprocess_input(
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/imagenet_utils.py", line 104, in
preprocess_input
    return _preprocess_numpy_input(x, data_format=data_format, mode=mode)
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/imagenet_utils.py", line 185, in
_preprocess_numpy_input
    x /= 127.5
ValueError: output array is read-only
Traceback (most recent call last):
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/queueing.py",
line 625, in process_events
    response = await route_utils.call_process_api(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/route_utils.py",
line 322, in call_process_api
    output = await app.get_blocks().process_api(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/blocks.py", line
2047, in process_api
    result = await self.call_function(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/blocks.py", line
1594, in call_function
    prediction = await anyio.to_thread.run_sync(  # type: ignore
  File "/home/rgukt/.local/lib/python3.10/site-packages/anyio/to_thread.py",
line 56, in run_sync
    return await get_async_backend().run_sync_in_worker_thread(
  File "/home/rgukt/.local/lib/python3.10/site-
packages/anyio/_backends/_asyncio.py", line 2177, in run_sync_in_worker_thread
    return await future
  File "/home/rgukt/.local/lib/python3.10/site-
```

```
packages/anyio/_backends/_asyncio.py", line 859, in run
    result = context.run(func, *args)
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/utils.py", line
869, in wrapper
    response = f(*args, **kwargs)
  File "/tmp/ipykernel_4998/1726201170.py", line 18, in classify_image
    inp = tf.keras.applications.inception_v3.preprocess_input(inp)  # Preprocess
the input
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/inception_v3.py", line 427, in preprocess_input
    return imagenet_utils.preprocess_input(
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/imagenet_utils.py", line 104, in
preprocess_input
    return _preprocess_numpy_input(x, data_format=data_format, mode=mode)
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/imagenet_utils.py", line 185, in
_preprocess_numpy_input
    x /= 127.5
ValueError: output array is read-only
Traceback (most recent call last):
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/queueing.py",
line 625, in process_events
    response = await route_utils.call_process_api(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/route_utils.py",
line 322, in call_process_api
    output = await app.get_blocks().process_api(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/blocks.py", line
2047, in process_api
    result = await self.call_function(
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/blocks.py", line
1594, in call_function
    prediction = await anyio.to_thread.run_sync(  # type: ignore
  File "/home/rgukt/.local/lib/python3.10/site-packages/anyio/to_thread.py",
line 56, in run_sync
    return await get_async_backend().run_sync_in_worker_thread(
  File "/home/rgukt/.local/lib/python3.10/site-
packages/anyio/_backends/_asyncio.py", line 2177, in run_sync_in_worker_thread
    return await future
  File "/home/rgukt/.local/lib/python3.10/site-
packages/anyio/_backends/_asyncio.py", line 859, in run
    result = context.run(func, *args)
  File "/home/rgukt/.local/lib/python3.10/site-packages/gradio/utils.py", line
869, in wrapper
    response = f(*args, **kwargs)
  File "/tmp/ipykernel_4998/1726201170.py", line 18, in classify_image
    inp = tf.keras.applications.inception_v3.preprocess_input(inp)  # Preprocess
the input
```

```
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/inception_v3.py", line 427, in preprocess_input
    return imagenet_utils.preprocess_input(
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/imagenet_utils.py", line 104, in
preprocess_input
    return _preprocess_numpy_input(x, data_format=data_format, mode=mode)
  File "/home/rgukt/.local/lib/python3.10/site-
packages/keras/src/applications/imagenet_utils.py", line 185, in
_preprocess_numpy_input
    x /= 127.5
ValueError: output array is read-only
```

[59]:

```python
from PIL import Image, ImageDraw, ImageFont

# Generate a placeholder image
def create_placeholder_image(text, size=(200, 200), file_name="placeholder.
↪jpg"):
    img = Image.new("RGB", size, color=(150, 150, 150))
    draw = ImageDraw.Draw(img)
    font_size = 20
    try:
        font = ImageFont.truetype("arial.ttf", font_size)
    except:
        font = None  # Use default font if arial is not available

    text_width, text_height = draw.textsize(text, font=font)
    text_x = (size[0] - text_width) // 2
    text_y = (size[1] - text_height) // 2
    draw.text((text_x, text_y), text, fill=(255, 255, 255), font=font)
    img.save(file_name)
    print(f"Image saved as: {file_name}")

# Create example placeholder images
image_names = ["monkey.jpg", "sailboat.jpg", "bicycle.jpg", "download.jpg",␣
↪"fox.jpg"]
for name in image_names:
    create_placeholder_image(name.split('.')[0].capitalize(), file_name=name)
```

```
Image saved as: monkey.jpg
Image saved as: sailboat.jpg
Image saved as: bicycle.jpg
Image saved as: download.jpg
Image saved as: fox.jpg
```

[2]:
```
# def classify_image_with_mobile_net(im):
#     im=Image.fromarray(im.astype('uint8'),'RGB')
#     im=im.resize((224,224))
#     arr=np.array(im).reshape((-1,224,224,3))
#     arr=tf.keras.applications.mobilenet.preprocess_input(arr)
#     prediction=mobile_net.predict(arr).flatten()
#     return {labels[i]:float(prediction[i]) for i in range(1000)}

# def classify_image_with_inception_net(im):
#     im=Image.fromarray(im.astype('uint8'),'RGB')
#     im=im.resize((224,224))
#     arr=np.array(im).reshape((-1,224,224,3))
#     arr=tf.keras.applications.inception_v3.preprocess_input(arr)
#     prediction=mobile_net.predict(arr).flatten()
#     return {labels[i]:float(prediction[i]) for i in range(1000)}

import gradio as gr
import tensorflow as tf
import numpy as np
from PIL import Image
import requests

# Download human-readable labels for ImageNet
response = requests.get("https://git.io/JJkYN")
labels = response.text.split("\n")

# Load models
mobile_net = tf.keras.applications.MobileNetV2()
inception_net = tf.keras.applications.InceptionV3()

def classify_image_with_inception_net(im):
    # Resize the image to
    im = Image.fromarray(im.astype('uint8'), 'RGB')
    im = im.resize((299, 299))
    arr = np.array(im).reshape((-1, 299, 299, 3))
    # arr = tf.keras.applications.inception_v3.preprocess_input(arr)
    prediction = inception_net.predict(arr).flatten()
    return {labels[i]: float(prediction[i]) for i in range(1000)}

def classify_image_with_mobile_net(im):
    im = Image.fromarray(im.astype('uint8'), 'RGB')
    im = im.resize((224, 224))
    arr = np.array(im).reshape((-1, 224, 224, 3))
    # arr = tf.keras.applications.mobilenet.preprocess_input(arr)
    prediction = mobile_net.predict(arr).flatten()
    return {labels[i]: float(prediction[i]) for i in range(1000)}
```

```
# Gradio setup
imagein = gr.Image(type="numpy")
label = gr.Label(num_top_classes=3)

# sample_images = [
#     ['monkey.jpg'],
#     ['sailboat.jpg'],
#     ['bicycle.jpg'],
#     ['fox.jpg']
# ]

# gr.Interface(
#     fn=classify_image_with_both,  # Single callable function
#     inputs=imagein,
#     outputs=label,
#     title="MobileNet vs InceptionNet",
#     description="Lightweight MobileNet (0.704 accuracy) vs Heavier␣
↪InceptionNet (0.779 accuracy).",
#     examples=sample_images
# ).launch()


with gr.Blocks() as demo:
    gr.Markdown("## MobileNet vs. InceptionNet\nCompare predictions from two␣
↪models:")
    with gr.Row():
        for model, title in [(classify_image_with_mobile_net, "MobileNet"),␣
↪(classify_image_with_inception_net, "InceptionNet")]:
            with gr.Column():
                gr.Interface(model, imagein,label, title=title).render()

demo.launch()
```

```
* Running on local URL:  http://127.0.0.1:7863

To create a public link, set `share=True` in `launch()`.
```

```
<IPython.core.display.HTML object>
```

[2]:

```
1/1              2s 2s/step
```

[ ]:

```

```